



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA INFORMÁTICA EN
INGENIERÍA DEL SOFTWARE

TRABAJO FIN DE GRADO

**Análisis y gestión de servicios de red en
infraestructuras 5G de código abierto**

Juan Barco Gil
Septiembre, 2026



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA INFORMÁTICA EN
INGENIERÍA DEL SOFTWARE

TRABAJO FIN DE GRADO

**Análisis y gestión de servicios de red en
infraestructuras 5G de código abierto**

Autor: Juan Barco Gil

Tutor: David Miguel Cortés Polo

Agradecimientos

A mis padres, Fátima y Juan, por haberme enseñado tanto y haberme educado de la mejor forma y con todo el cariño que uno pudiera desear; y por haberme apoyado en todas mis empresas y malos momentos.

A mi hermana, Andrea, y a mi hermano, Mario, por haberme allanado el camino y haberme acompañado en todo momento. Los mejores hermanos que podría haber tenido.

A mi tutor, David, por haberme guiado y ayudado tanto durante el desarrollo de este trabajo, y haberme despertado el interés por las redes y la telemática.

Gracias a todos por haberme ayudado a llegar hasta aquí.

Declaración de uso de herramientas de inteligencia artificial

Durante la realización de este Trabajo de Fin de Grado se han utilizado herramientas de inteligencia artificial como apoyo auxiliar, concretamente Claude y Prism. Su uso se ha orientado a la organización del estudio, la revisión y mejora de la redacción, la detección de posibles erratas, la comprobación de coherencia entre secciones y la resolución de dudas relacionadas con la escritura y estructuración del documento en LaTeX.

El uso de estas herramientas no ha sustituido el análisis propio del autor ni la validación técnica del trabajo. Las afirmaciones teóricas, las citas bibliográficas, los resultados presentados y el funcionamiento de la aplicación han sido revisados a partir de las fuentes consultadas y del código implementado.

Resumen

Este Trabajo de Fin de Grado se centra principalmente en la exploración y documentación del desarrollo de una herramienta que permite y facilita tanto el mantenimiento como la visualización de redes privadas 5G, concretamente aquellas que son implementadas en su totalidad por tecnologías Open Source, como lo son srsRAN y Open5GS. El trabajo realizado no solo abarca el diseño e implementación del *dashboard*, si no que también se encarga del despliegue al completo de la infraestructura de radio (estación base usando srsRAN y una tarjeta SDR) y del núcleo de la red (Open5GS en contenedores).

El proyecto se apoya en parte del desarrollo de un anterior alumno de una aplicación con objetivos parecidos, y logra expandirse de forma eficaz más allá de las limitaciones de este como, por ejemplo, poder arrancar el núcleo de red o dar de alta a un UE desde la *app*; mejora y optimiza mecanismos limitados, como la recuperación de los *logs*; y aporta nuevas funcionalidades al panel de control, permitiendo guardar y cargar configuraciones personalizadas del gNB, entre otras cosas. Además de todo esto, la portabilidad conseguida gracias al uso de Docker permite desplegar la herramienta en otros equipos con facilidad.

Los resultados obtenidos tras el desarrollo son positivos, y es con ellos con los que se puede afirmar que la aplicación facilita y mejora el trabajo de gestión y monitorización de redes privadas de este tipo, y contribuye satisfactoriamente al desarrollo de herramientas para la gestión de redes 5G, haciendo énfasis en el uso de software open source.

Palabras clave: 5G, Open5GS, srsRAN, Docker, red privada, radio, Radio Definida por Software (SDR), dashboard, monitorización, gestión de red.

Abstract

This Bachelor's Thesis focuses primarily on the exploration and documentation of the development of a tool that enables and facilitates both the maintenance and the visualisation of private 5G networks, specifically those implemented entirely with Open Source technologies, such as srsRAN and Open5GS. The work carried out not only covers the design and implementation of the dashboard, but also encompasses the complete deployment of the radio infrastructure (base station using srsRAN and an SDR card) and the network core (Open5GS in containers).

The project builds in part upon the development of a previous student's application with similar objectives, and successfully expands beyond its limitations, such as being able to start the network core or register a UE, both directly from the app; it improves and optimises limited mechanisms, such as log retrieval; and it brings new functionalities to the control panel, allowing custom gNB configurations to be saved and loaded, among other things. In addition to all of this, the portability achieved through the use of Docker allows the tool to be easily deployed on other machines.

The results obtained following the development are positive, and it is with these results that it can be affirmed that the application facilitates and improves the work of managing and monitoring private networks of this kind, and successfully contributes to the development of tools for the management of 5G networks, with an emphasis on the use of open source software.

Keywords: 5G, Open5GS, srsRAN, Docker, private network, radio, Software Defined Radio (SDR), dashboard, monitoring, network management.

Índice general

1. Introducción	1
2. Objetivos	4
3. Estado del Arte	6
Tecnología 5G y sus redes	7
Qué es 5G y en qué mejora a 4G	7
Arquitectura de las redes 5G	9
Arquitecturas <i>Standalone</i> y <i>Non-Standalone</i>	11
Redes 5G privadas de código abierto	12
Plataformas de acceso radio de código abierto	13
Núcleos de red 5G de código abierto	14
Implementaciones del gNB	14
Radio definida por software - SDR	15
Monitorización del <i>Core</i> 5G	17
Antecedentes y punto de partida del trabajo	18
4. Metodología	20
4.1. Selección de tecnologías	21
4.1.1. Elección del núcleo de red: Open5GS	21
4.1.2. Elección del hardware SDR: bladeRF frente a USRP B210	22
4.1.3. Elección del mecanismo de monitorización del <i>Core</i>	23
4.1.4. Elección entre srsRAN Project y OCUDU	25

4.2.	Entorno y recursos empleados	25
4.2.1.	Equipos de desarrollo	26
4.2.2.	Hardware de radio	26
4.2.3.	Equipo de usuario y programación de la SIM	28
4.3.	Diseño del panel de monitorización	30
4.3.1.	Alcance: de la monitorización a la gestión	30
4.3.2.	Arquitectura del <i>backend</i>	31
4.3.3.	Mecanismo de actualización de datos	32
4.3.4.	Seguridad: privilegio mínimo y autenticación configurable	32
5.	Implementación y desarrollo	34
5.1.	Preparación del entorno base	34
5.1.1.	Instalación de los paquetes base	34
5.1.2.	Detección e identificación del modelo	35
5.1.3.	Versiones finales del entorno	36
5.2.	Instalación de SoapySDR	36
5.3.	Obtención y compilación de srsRAN Project	37
5.4.	Despliegue del núcleo Open5GS	38
5.4.1.	Compilación e instalación	38
5.4.2.	Configuración del dispositivo TUN	39
5.4.3.	Configuración del PLMN de pruebas	40
5.4.4.	Automatización del arranque del núcleo	40
5.5.	Configuración del gNB con bladeRF	40
5.5.1.	Fichero de configuración de partida	40
5.6.	Verificación: primer arranque estable	42
5.7.	Validación con un equipo de usuario real	42
5.7.1.	Programación de la SIM y alta del suscriptor	43
5.8.	Migración y configuración con la USRP B210	43
5.8.1.	Configuración con UHD nativo	44
5.9.	Despliegue del núcleo Open5GS en Docker	45

5.9.1. Selección del repositorio base	45
5.9.2. Instalación de Docker en Debian Trixie	45
5.9.3. Adaptación de la configuración	46
5.10. Desarrollo del <i>backend</i>	47
5.10.1. Estructura del proyecto	47
5.10.2. Documentación automática de la API	48
5.10.3. Autenticación y degradación controlada	49
5.10.4. Verificación automatizada con datos reales	49
5.10.5. <i>Scripts</i> de control con privilegio mínimo	50
5.11. Monitorización de cada NF	51
5.12. Verificación de los <i>endpoints</i>	51
5.12.1. Control de Docker con privilegio mínimo	52
5.12.2. Verificación del control de NFs y de suscriptores	53
5.13. Estabilidad y portabilidad del despliegue	53
5.14. Portabilidad del <i>backend</i> y <i>scripts</i> de control	55
5.15. Desarrollo del <i>frontend</i>	56
5.15.1. Estructura del proyecto	56
5.15.2. Política de mismo origen del navegador (CORS)	57
5.15.3. Recuento de gNBs mediante la métrica nativa del AMF	58
5.16. Monitorización de NFs mediante NRF	58
5.16.1. El registro del NRF como fuente de estado	58
5.16.2. El caso particular del UPF	59
5.17. Ampliaciones de gestión del gNB	60
5.17.1. Arranque independiente de la modificación de configuración	60
5.17.2. Confirmación del hardware SDR realmente detectado	60
5.18. Ampliación del modo sin autenticación	61
5.19. Diagrama de topología de red en vivo	62
5.20. Revisión y nuevas funcionalidades	62
5.20.1. Comparativa con el trabajo predecesor	63

5.20.2. Funcionalidades inspiradas en OAIBOX	63
5.21. Histórico de tráfico N3	64
5.21.1. Decisión de arquitectura: réplica del <i>stack</i> del predecesor frente a solución propia	64
5.21.2. Muestreo en segundo plano y cálculo de tasas	64
5.22. Gestión ampliada de las funciones de red	65
5.23. Configuración del gNB mediante <i>presets</i>	66
5.23.1. Límites reales de ganancia de la USRP B210	66
5.23.2. Diseño: <i>presets</i> validados frente a campos libres	67
5.24. Resolución de la conexión del UE	67
5.24.1. Evidencia del registro completo	68
5.24.2. Salida real a la red de datos	68
5.25. Ampliaciones de gestión adicionales	69
5.25.1. Configuraciones del gNB guardadas por el usuario	69
5.25.2. Consulta, búsqueda y descarga de registros de actividad	71
5.25.3. Arranque y parada del <i>core</i> completo	72
6. Resultados	74
6.1. Funcionamiento de la aplicación construida	74
7. Conclusiones y trabajos futuros	89
7.1. Conclusiones	89
7.2. Trabajos futuros	90
Anexos	93
A. Script de orquestación del núcleo Open5GS	94
B. Script de control del gNB	100
C. Script de control de las funciones de red en Docker	105

D. Incidencias, diagnóstico y resolución	110
D.1. Paquete de <i>firmware</i> de la bladeRF en el componente contrib	110
D.2. Rama por defecto del repositorio archivado de srsRAN Project	111
D.3. Incidencias múltiples en la integración del gNB con la bladeRF	111
D.3.1. Valor vacío no admitido en device_driver	112
D.3.2. Fuente de sincronización no soportada	112
D.3.3. <i>Timeouts</i> de los búferes de transmisión y recepción	114
D.4. El terminal no detecta la celda (bladeRF)	115
D.5. <i>Timeouts</i> de transmisión con la USRP B210	116
D.5.1. Réplica del entorno del trabajo predecesor	117
D.5.2. Aislamiento final de la causa: benchmark_rate	117
D.5.3. Causa raíz: parche heredado de la integración de la bladeRF	118
D.6. Corrupción de la red de Docker tras reinicios parciales	119
D.7. Conexión SCTP entre el gNB y el AMF en el despliegue Docker	119
D.7.1. Primera causa: reenvío de direcciones internas en el <i>multi-homing</i> de SCTP	120
D.7.2. Segunda causa: bind_addr obsoleto	121
D.7.3. Confirmación final, y una lección sobre el propio proceso de depuración	122
Bibliografía	123

Índice de tablas

4.1. Especificaciones de los dos equipos empleados durante el desarrollo.	26
5.1. Versiones del entorno bladeRF verificadas en la Fase 1.	36
5.2. Adaptación del fichero de configuración de ejemplo del gNB.	41
6.1. Suscriptores dados de alta para la validación de este trabajo.	84

Índice de figuras

3.1. Elementos de un sistema 5G	9
3.2. Funciones de red del <i>Core</i> 5G	10
3.3. Arquitecturas <i>Standalone</i> y <i>Non-Standalone</i>	12
4.1. Arquitectura del proyecto, planos de radio y de gestión	21
4.2. Tarjeta USRP B210 empleada en este trabajo	27
4.3. Cable USB-A a USB-B con adaptador USB-A a USB-C	27
4.4. Google Pixel 8, UE empleado	28
4.5. Lector de tarjetas SIM empleado	29
4.6. Portátil del laboratorio empleado con GRSIMWrite	29
4.7. GRSIMWrite, empleado para programar la SIM	30
6.1. Formulario de <i>login</i> del <i>dashboard</i>	76
6.2. Panel resumen del <i>dashboard</i>	77
6.3. Panel Topología del <i>dashboard</i>	78
6.4. Panel Núcleo del <i>dashboard</i>	79
6.5. Acciones sobre los <i>Network Functions</i>	79
6.6. Acciones sobre los registros del <i>Core</i>	80
6.7. Panel gNB del <i>dashboard</i> , aún inactivo	81
6.8. Panel gNB del <i>dashboard</i> , mostrando un <i>preset</i> personalizado	82
6.9. Panel gNB del <i>dashboard</i> , ya activo, mostrando el <i>log</i>	83
6.10. Panel Suscriptores del <i>dashboard</i>	84
6.11. Formulario para dar de alta un nuevo UE en el <i>dashboard</i>	84

6.12. Registro del AMF, conexión entre el <i>core</i> y el gNB	85
6.13. Registro del AMF, conexión entre el UE y el gNB	85
6.14. UE conectado a la red privada 5G	86
6.15. Panel Resumen mostrando todos los elementos conectados	86
6.16. Panel Topología del <i>dashboard</i> , mostrando todo activo	87
6.17. Panel Resumen del <i>dashboard</i> , mostrando tráfico del UE	87
6.18. Panel Resumen del <i>dashboard</i> , detalles del tráfico del UE	88

Capítulo 1

Introducción

Con el paso del tiempo, el despliegue y uso de las redes móviles de quinta generación (5G) ha dejado de ser algo exclusivo de operadoras de telecomunicaciones. La consolidación de proyectos *software* de código abierto capaces de implementar tanto la red de acceso radio (*Radio Access Network*, RAN) como el núcleo de red (*Core*) ha permitido que universidades, centros de investigación y empresas puedan construir sus propias redes 5G privadas sobre hardware de radio definida por software (*Software Defined Radio*, SDR) relativamente asequible. Esta democratización facilita que laboratorios, universidades, institutos y otros entornos de experimentación que no cuentan con los recursos que eran necesarios previamente puedan acceder, sin depender de un operador comercial, a aspectos como la gestión de usuarios o el comportamiento de la red ante distintas condiciones de tráfico.

A pesar de esto, disponer de una red 5G privada desplegada no resulta suficiente por sí solo. En estos entornos, resulta necesario poder observar en todo momento qué está ocurriendo dentro de la red: si la estación base está activa, si los distintos servicios que componen el núcleo se encuentran disponibles, o qué equipos de usuario están conectados en cada momento son preguntas recurrentes que, en despliegues de este tipo, no siempre tienen una respuesta fácil y rápida si no se cuenta con las herramientas de monitorización adecuadas. Las soluciones comerciales, como OAIBOX¹, ofrecen un panel de control o

¹Producto de Allbesmart basado en OpenAirInterface, empleado únicamente como referencia de funcionalidad a lo largo de este trabajo, sin relación técnica entre ambos proyectos.

dashboard con la información pertinente, pero de forma cerrada y ligada a hardware y software propios, lo que limita su uso en un contexto enteramente abierto como el que se persigue en este trabajo.

Es por esta razón por la que surge este Trabajo de Fin de Grado, cuyo objetivo es desarrollar una herramienta de **monitorización y gestión** para una red 5G privada construida enteramente con tecnologías de código abierto: **srsRAN Project** como implementación de la estación base, y **Open5GS**, desplegado en contenedores Docker, como núcleo de red. Este proyecto retoma el trabajo previo de otro estudiante, quien desarrolló una primera versión de un panel de monitorización de solo lectura para esta misma combinación de tecnologías, empleando una tarjeta USRP B200 como SDR [1]. Aquel trabajo obtenía toda su información mediante el análisis de los registros del núcleo, limitado a la función de red AMF y únicamente en tiempo real, sin posibilidad de consultar el contenido ya existente ni de buscar información dentro de él; para el tráfico histórico recurría a un conjunto de herramientas de terceros (*ntopng*, *InfluxDB* y *Grafana*); y el alta de suscriptores se realizaba a través de la interfaz web de Open5GS, no desde la app.

Sobre esa base, este trabajo amplía el alcance de la **monitorización** a la **gestión activa** de la infraestructura desde el propio panel, aplicando además principios de seguridad (privilegio mínimo, autenticación de usuarios) ausentes en el trabajo predecesor, al no requerirlos una herramienta de solo lectura. El panel desarrollado, con un *backend* en Python (FastAPI) y un *frontend* en React, ofrece en concreto:

- **Monitorización en tiempo real** del estado de las diez funciones de red del núcleo, de la estación base y de los equipos de usuario conectados, apoyada sobre los mecanismos estructurados que el propio Open5GS expone (métricas en formato Prometheus y una API JSON propia), en lugar del análisis de registros de texto empleado por el trabajo predecesor; un diagrama de topología en vivo de las interfaces reales de la arquitectura 5G SA; y un histórico del tráfico de subida cursado por el equipo de usuario.
- **Gestión activa** del núcleo (arranque, parada y reinicio de cada función de red de forma individual o conjunta, y consulta, búsqueda y descarga de sus registros

de actividad) y de la estación base (arranque, parada y reconfiguración mediante combinaciones de parámetros de radio validadas, o mediante configuraciones personalizadas guardadas por el propio usuario).

- **Administración de suscriptores** (alta y baja) integrada en el propio panel, sin necesidad de recurrir a la interfaz web oficial de Open5GS.
- Un modelo de **seguridad** basado en autenticación mediante *tokens* JWT y en el principio de privilegio mínimo: el *backend* nunca ejecuta directamente ninguna acción sobre el sistema operativo o sobre Docker, sino que delega en *scripts* acotados mediante reglas **sudoers** específicas.

El desarrollo abarca, además del propio panel, el despliegue completo de la infraestructura de radio y núcleo sobre la que este opera, en un equipo con Debian: una estación base real sobre una tarjeta USRP B210, con la que se ha validado el funcionamiento del sistema mediante el registro de un equipo de usuario real en la red desplegada. De forma complementaria, se ha explorado también la viabilidad de emplear una tarjeta bladeRF 2.0 (sin soporte nativo en srsRAN Project), documentándose las incidencias encontradas durante dicha exploración, aunque sin llegar a completarse con éxito. Por último, la portabilidad de la solución desarrollada se ha validado mediante su migración completa a un segundo equipo.

El resto de este documento se organiza de la siguiente forma: el Capítulo 2 especifica los objetivos generales y específicos perseguidos con este trabajo. El Capítulo de Estado del Arte aporta una visión teórica del panorama actual de las redes 5G, con énfasis en las redes privadas de código abierto y en las opciones existentes para su monitorización. A continuación, el Capítulo de Metodología describe el enfoque de trabajo seguido y las decisiones técnicas justificadas antes de comenzar el desarrollo, mientras que el de Implementación y Desarrollo detalla el trabajo realizado en cada una de las fases del proyecto. Por último, los Capítulos de Resultados y de Conclusiones y Trabajos Futuros recogen la validación del sistema desarrollado y sus resultados, así como una reflexión sobre el trabajo realizado y las posibles ampliaciones que se podrían explorar.

Capítulo 2

Objetivos

Como se ha mencionado en la introducción del documento, el uso de implementaciones de código abierto tanto para la estación base (srsRAN Project) como para el núcleo de red (Open5GS) en redes privadas 5G ha hecho que este tipo de despliegues sea mucho más accesible. Sin embargo, aún existe cierta dificultad a la hora de operar y supervisar una infraestructura como esta, que está compuesta por varios elementos que son independientes entre sí.

Teniendo todo esto en cuenta, el **objetivo principal** que se plantea para este trabajo es **desarrollar una herramienta de monitorización y gestión para infraestructuras de red 5G privadas construidas con tecnologías de código abierto, facilitando así estas tareas.**

Para alcanzar este objetivo, también se han planteado una serie de **objetivos específicos**:

- Analizar el estado del arte de las tecnologías de código abierto empleadas en redes 5G privadas (srsRAN Project, Open5GS) y de las herramientas de monitorización y gestión existentes, tomando como referencia un trabajo anterior con el que se comparten objetivos y tecnologías, y las plataformas comerciales que cuenten con una misión similar.
- Desplegar la infraestructura de red 5G privada de forma completa y funcional, compuesta por una estación base sobre una tarjeta de radio definida por software

y un núcleo de red en contenedores, comprobando su funcionamiento mediante el registro real de un equipo de usuario.

- Diseñar e implementar un panel web que permita monitorizar en tiempo real el estado de las funciones de red del núcleo, de la estación base y de los equipos de usuario conectados.
- Ampliar dicho panel con capacidades de gestión sobre la infraestructura desplegada, como, por ejemplo, arranque, parada y reconfiguración de sus componentes.
- Validar la portabilidad y la robustez de la solución desarrollada mediante la migración a otro equipo. Además, resolver cualquier incidencia detectada durante la migración o prueba de la aplicación.

Capítulo 3

Estado del Arte

En este apartado se explicarán en detalle aquellos aspectos relevantes para comprender y desarrollar el trabajo, además de proporcionar al lector unos cimientos teóricos en los que apoyarse de cara a la lectura del documento.

En este capítulo se exponen los avances más recientes y relevantes de las tecnologías empleadas en este trabajo, priorizando en gran medida el 5G y su vertiente de código abierto. Se tratarán los siguientes puntos:

- Qué es el 5G y en qué se diferencia de su antecesor.
- Arquitectura de este tipo de redes, incluyendo sus variantes Standalone y Non-Standalone.
- Redes privadas 5G y la aparición del código abierto en el ámbito.
- Implementaciones del gNB, desde soluciones monolíticas hasta la desagregación propuesta por O-RAN.
- Radio Definida por Software y panorama de opciones de hardware.
- Elementos heredados y base desde la que se parte.



Tecnología 5G y sus redes

Qué es 5G y en qué mejora a 4G

La quinta generación de comunicaciones móviles (5G) no se trata únicamente de una progresión lineal de las redes 4G/LTE en cuanto a velocidad de datos se refiere, sino que se trata de una nueva generación diseñada desde el principio para dar servicio a un conjunto de escenarios de uso mucho más amplio que el de sus predecesoras. La **ITU** (*International Telecommunication Union*), a través de la Recomendación ITU-R M.2083, define el marco IMT-2020 en torno a tres escenarios de uso principales [2]:

- **eMBB** (*enhanced Mobile Broadband*): banda ancha móvil mejorada, dedicado a mejorar en gran medida las tasas de datos, la capacidad y la experiencia del usuario respecto a 4G, especialmente en escenarios de alta densidad de usuarios.
- **URLLC** (*Ultra-Reliable Low-Latency Communications*): comunicaciones ultrafiabiles y de baja latencia, pensadas para aplicaciones críticas tales como la automatización industrial, los vehículos de conducción autónoma o la cirugía remota, todos campos en los que 4G no podía dar un soporte con la certeza que requieren estos.
- **mMTC** (*massive Machine-Type Communications*): comunicaciones masivas tipo máquina, orientadas a dar servicio a una densidad de dispositivos IoT muy superior a la que 4G fue diseñada para soportar.

Para dar soporte a estos tres escenarios, la propia ITU establece un conjunto de capacidades clave en las que 5G debe superar (y supera) de forma notable a 4G. Entre ellas destacan: una mayor tasa de datos máxima y de usuario, una latencia radicalmente menor, una densidad de conexión considerablemente mayor (pensada para escenarios mMTC) y una mayor eficiencia espectral y energética [2]. Es precisamente esta necesidad de satisfacer simultáneamente todas estas capacidades tan dispares entre sí (optimizar para baja latencia no es lo mismo que optimizar para densidad de dispositivos) lo que motiva buena parte de las decisiones de diseño de la arquitectura 5G descritas más adelante,



en el apartado dedicado a la arquitectura de las redes 5G. Además de esto, también se cuenta con la introducción de conceptos que 4G no contemplaba, como puede ser el *network slicing*, es decir, la posibilidad de segmentar (*slice*) una red física en varias redes virtuales, cada una optimizada para un escenario de uso distinto.

Es importante señalar, para evitar una confusión habitual, el papel que desempeña cada organismo a la hora de definir características y aspectos de la tecnología 5G. Por un lado, la ITU no diseña la tecnología en sí, sino que fija el marco de requisitos y las capacidades que cualquier tecnología debe cumplir para poder considerarse de forma oficial IMT-2020, todo esto a través de la ya mencionada recomendación M.2083.

Por otro lado, es el **3GPP** quien diseña y especifica la tecnología concreta, como la interfaz radio NR y la arquitectura de red descrita en el apartado siguiente, fue quien presentó formalmente dicha especificación ante la UIT. Tras su evaluación, la ITU reconoció en 2020 que la propuesta de 3GPP cumplía los requisitos de IMT-2020 [3]. En lo que resta de documento, por tanto, las referencias a la ITU corresponden a la visión y los objetivos de 5G, mientras que las referencias a 3GPP corresponden al diseño técnico concreto de 5G.

En cuanto al espectro radioeléctrico, 3GPP divide las bandas de frecuencia empleadas por 5G NR en dos rangos: **FR1** (*Frequency Range 1*), que abarca de 410 MHz a 7125 MHz, también denominado espectro *sub-7 GHz*, e incluye tanto bandas bajas (por debajo de 1 GHz) como bandas medias; y **FR2** (*Frequency Range 2*), de 24,25 GHz a 71 GHz, correspondiente al espectro de ondas milimétricas (*mmWave*) [4]. Las bandas de FR1 ofrecen mayor alcance y penetración a costa de menor ancho de banda disponible, mientras que las de FR2 permiten anchos de banda de hasta varios gigahercios a costa de un alcance mucho más reducido. Las redes 5G privadas para entornos industriales o de campus, como la desarrollada en este trabajo, se despliegan habitualmente sobre bandas medias de FR1 (concretamente, la banda **n78**, en torno a 3,5 GHz), por ofrecer un equilibrio razonable entre cobertura y ancho de banda disponible sin requerir el equipamiento especializado que exige FR2.

Arquitectura de las redes 5G

Una red 5G se organiza, al igual que sus predecesoras, en dos grandes bloques: la **red de acceso radio** (*Radio Access Network*, RAN), que a su vez está formada por las estaciones base, también llamadas **gNB** en 5G, frente al **eNB** de 4G, que se encargan de la comunicación por radiofrecuencia con los equipos de usuario (*User Equipment*, UE); y el **núcleo de red** (*Core ó 5GC*), responsable de la autenticación, la gestión de la movilidad, el establecimiento de sesiones de datos y el encaminamiento del tráfico hacia redes externas. Esta separación conceptual es uno de los elementos que 5G **mantiene** de la arquitectura de 4G. Véase la figura 3.1.

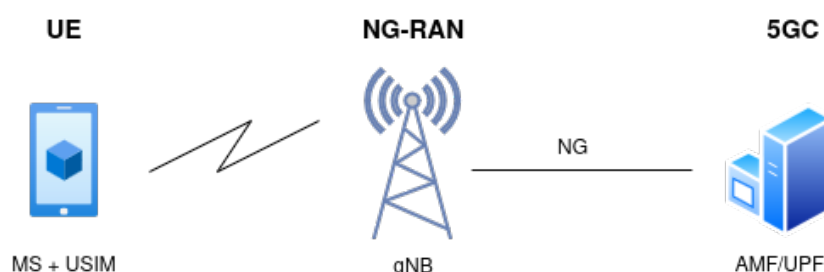


Figura 3.1: Elementos de un sistema 5G.

Fuente: Propia

Lo que cambia de forma sustancial entre ambas tecnologías es el cómo se organiza internamente el núcleo de red o *core*. Mientras que el núcleo de 4G (*Evolved Packet Core*, EPC) se articulaba en torno a un número reducido de nodos monolíticos (la MME para señalización y movilidad, o el par SGW/PGW para el plano de usuario) comunicados mediante interfaces punto a punto, 3GPP especifica el núcleo 5G mediante una **arquitectura basada en servicios** (*Service-Based Architecture*, SBA), un elemento completamente novedoso que no existía en tecnologías anteriores. De esta forma, el núcleo se descompone en un conjunto de **funciones de red** (*Network Functions*, NF) independientes que se comunican entre sí mediante interfaces de servicio (*Service-Based Interfaces*, SBI), haciendo uso de HTTP/2, en lugar de las interfaces punto a punto tradicionales [5]. Entre las NF más relevantes para este trabajo se encuentran:

- **AMF** (*Access and Mobility Management Function*): asume las funciones de registro

y movilidad que en 4G correspondían a la MME (*Mobility Management Entity*).

- **SMF** (*Session Management Function*) y **UPF** (*User Plane Function*): ambas son el resultado de dividir en dos NF independientes las funciones que en 4G utilizaba conjuntamente el par SGW/PGW, separando así de forma eficaz el plano de control del plano de usuario, dentro del propio núcleo.
- **NRF** (*NF Repository Function*): hace las veces de directorio en el que el resto de NF se registran y a través del cual se descubren entre sí. Se trata de un elemento que no cuenta con un equivalente directo en la arquitectura EPC de 4G.
- **AUSF**, **UDM** y **UDR**: los tres intervienen en la autenticación del UE y en el almacenamiento y gestión de sus datos de su suscripción, realizando una tarea similar a la que el HSS (*Home Subscriber Server*) hacía en 4G.

El 3GPP define además otras NF para el *Core* 5G, véase la Figura 3.2.

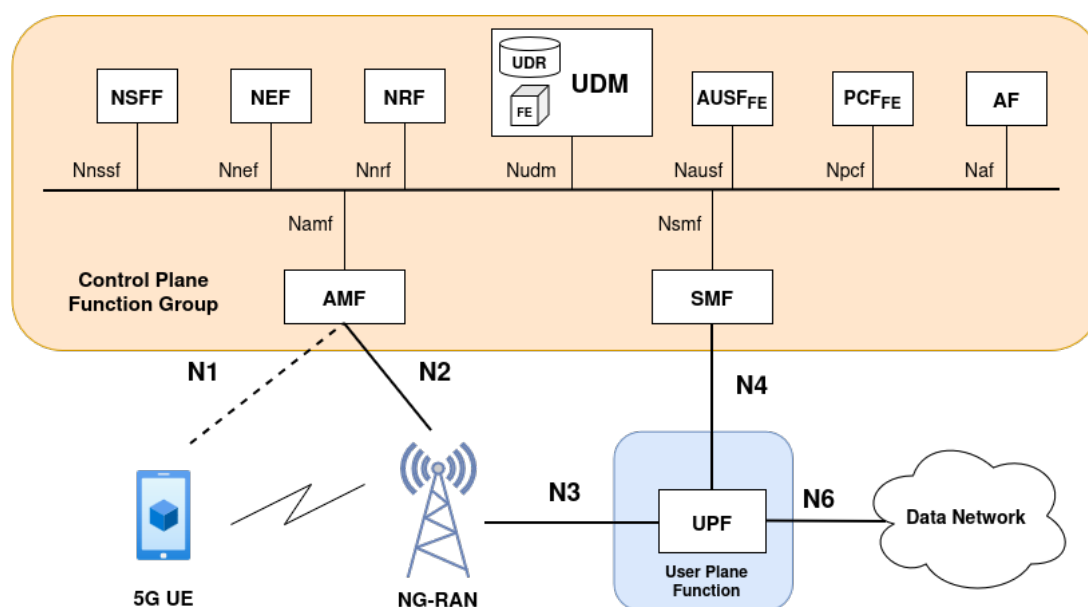


Figura 3.2: Funciones de red del *Core* 5G.

Fuente: Propia

Como se puede observar en la anterior figura, la comunicación entre el gNB y el núcleo se realiza a través de la interfaz **N2** hacia el AMF, y de la interfaz **N3** hacia el



plano de usuario y el UPF. Por otro lado, el UE se comunica con el núcleo mediante la interfaz **N1** [5]. Esta separación entre plano de control y plano de usuario, así como la descomposición del núcleo en las diferentes funciones de red independientes, son elementos novedosos respecto a 4G y aportan la capacidad de que proyectos de código abierto (serán descritos en las secciones siguientes) puedan implementar cada función de red como un proceso independiente, pudiendo ejecutarse en contenedores separados si así se desea, y es también la base sobre la que se apoyan los mecanismos de monitorización descritos en la Sección 3.

En cuanto al tráfico de datos del propio UE, este se organiza mediante **sesiones PDU** (*Protocol Data Unit*), establecidas entre el UE y una red de datos externa concreta a través del SMF y el UPF, identificada mediante un **DNN** (*Data Network Name*, equivalente conceptual al APN de 4G). Dentro de cada sesión PDU, el tráfico se clasifica a su vez en uno o varios **QoS flows**, cada uno con un identificador (*5QI*) que determina el tratamiento que dicho tráfico recibe en la red (prioridad, latencia máxima admisible, tasa de error tolerada), permitiendo así diferenciar, dentro de una misma sesión, tráfico con requisitos de calidad de servicio distintos [5]. Este es, precisamente, el nivel de detalle (DNN, dirección IP asignada, estado de la sesión) que numerosas herramientas de monitorización, incluida la desarrollada en este trabajo, exponen sobre cada equipo de usuario conectado.

Arquitecturas *Standalone* y *Non-Standalone*

Todo lo descrito hasta ahora en esta sección corresponde a una arquitectura ***Standalone*** (SA): un gNB conectado directamente a un 5GC, sin ninguna dependencia de infraestructura 4G. Es el despliegue empleado en este trabajo, al igual que en el proyecto predecesor.

No obstante, conviene mencionar que buena parte de los primeros despliegues comerciales de 5G no siguieron esta arquitectura, sino una denominada ***Non-Standalone*** (NSA): el UE ancla su conexión de control a una red 4G/EPC ya existente, empleando el 5G NR únicamente como una portadora adicional de datos mediante conectividad dual (*EN-DC, E-UTRA New Radio Dual Connectivity*) [6]. Este enfoque permitía a

los operadores desplegar 5G reutilizando su infraestructura 4G existente, acelerando así el despliegue comercial y reduciendo la inversión inicial, aunque a costa de no poder beneficiarse de mejoras propias de 5G como una menor latencia o el *network slicing* [6]. Con el paso del tiempo, y a medida que los operadores han ido desplegando y consolidando sus propios núcleos 5G nativos, la arquitectura NSA ha ido quedando relegada a un papel de transición, cada vez menos relevante frente a los despliegues SA.

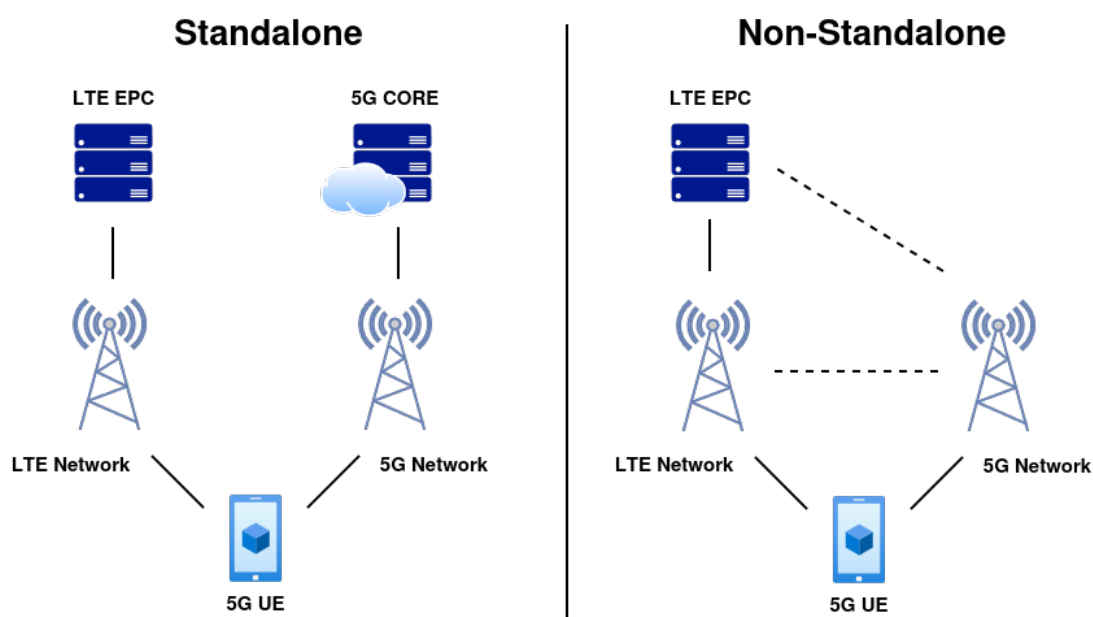


Figura 3.3: Arquitecturas *Standalone* y *Non-Standalone*
Fuente: Propia

Redes 5G privadas de código abierto

Una red 5G privada consiste en un despliegue de red móvil de quinta generación limitado a un entorno concreto, como puede ser un campus, una fábrica o un laboratorio, gestionado por una entidad diferente a un operador de telecomunicaciones. Este tipo de despliegues cobra gran protagonismo en entornos de investigación y docencia, ya que permite experimentar con la arquitectura completa descrita anteriormente, en el apartado sobre la arquitectura de las redes 5G, sin necesidad de licencias de espectro comerciales ni acuerdos con operadoras, siempre que se empleen bandas y potencias adecuadas para uso experimental.



La creciente aparición de implementaciones *software* de código abierto tanto para la RAN como para el núcleo de red [7], ha sido determinante para popularizar este tipo de despliegues, permitiendo así sustituir hardware de red propietario por servidores de propósito general y tarjetas SDR de un coste económico no tan prohibitivo.

Plataformas de acceso radio de código abierto

En el campo de la tecnología RAN de código abierto, existen principalmente dos grandes proyectos que abarcan en gran medida los despliegues: **OpenAirInterface** (OAI) y **srsRAN**. Ambos permiten implementar un gNB en software, ejecutado sobre un servidor de propósito general conectado a una tarjeta SDR.

Estudios recientes que comparan ambas plataformas coinciden en señalar diferencias relevantes para la elección de una frente a la otra. Por un lado, srsRAN se define como una iniciativa enteramente abierta, presentando un modelo de desarrollo enfocado en un esfuerzo comunitario, mientras que OAI, siendo el proyecto más veterano y contando con un equipo de desarrollo mucho más numeroso y centralizado, resulta significativamente más hermético en su proceso de contribución [8]. Por otro lado, en lo que a la compatibilidad de hardware respecta, se conoce que srsRAN ofrece soporte para toda la gama de tarjetas USRP, para bladeRF y para LimeSDR, además de un controlador ZeroMQ que permite simular una radio de forma completamente virtual [8]. Otros trabajos de investigación sobre esta materia enfocan esta comparativa desde un punto de vista más orientado al despliegue, dando así más valor a aspectos como la calidad de la documentación, la facilidad de instalación, la usabilidad o el soporte de bandas de frecuencia de ambas plataformas para la construcción de *benchmarks* 5G *Standalone* [9].

Para este trabajo, al igual que en el trabajo del que se parte, se opta por utilizar la plataforma srsRAN como implementación de la RAN, principalmente porque su compatibilidad declarada con bladeRF (aunque, como se detallará en el Capítulo de Metodología, no se trate de un soporte nativo) la convierte en la opción más viable para el hardware disponible, si bien, como se detalla en el Capítulo de Implementación y Desarrollo, la USRP B210 terminaría siendo la tarjeta empleada como principal a lo largo



de la mayor parte de este trabajo.

Núcleos de red 5G de código abierto

En cuanto al núcleo de red, también existen varias alternativas de código abierto, entre las que destacan **Open5GS**, **OAI-CN5G** y **free5GC**. Las tres implementan las funciones de red descritas anteriormente en el apartado de arquitectura, pero al igual que ocurre con las plataformas RAN, cada una tiene una diferente comunidad de desarrollo y facilidad de despliegue, entre otras características.

Este trabajo hereda la elección de Open5GS que tuvo lugar durante el desarrollo del proyecto predecesor [1], no solo por proporcionar cierta continuidad con dicho trabajo sino porque se trata de una de las implementaciones más extendidas en el ámbito, con una documentación rica y de ampliación activa; y un ritmo de desarrollo bastante alto. Esto último resulta especialmente relevante para este trabajo, y será expandido en la sección de este mismo capítulo dedicada a la monitorización del *Core* 5G.

Implementaciones del gNB

Tanto OAI como srsRAN, tal como se han presentado hasta ahora, implementan el gNB como una única pieza de *software* que ejecuta de forma conjunta las tres capas del protocolo radio (PHY, MAC y RRC/PDCP), lo que se conoce como una implementación **monolítica** del gNB. Es el enfoque más sencillo de desplegar, con un único proceso, en una única máquina, conectado a una tarjeta SDR, y es el que emplea tanto el trabajo predecesor como este mismo trabajo.

Sin embargo, el propio 3GPP contempla desde sus primeras especificaciones la posibilidad de **dividir** el gNB en varias unidades funcionales independientes: una **RU** (*Radio Unit*), encargada únicamente del *front-end* de radiofrecuencia; una **DU** (*Distributed Unit*), que procesa las capas bajas (PHY/MAC); y una **CU** (*Centralized Unit*), que procesa las capas altas (RRC/PDCP), pudiendo esta última dividirse a su vez en un plano de control (CU-CP) y un plano de usuario (CU-UP). Esta división, unida a



la definición de interfaces abiertas y estandarizadas entre cada unidad (como la interfaz F1 entre DU y CU), es el punto de partida de la iniciativa **O-RAN**, impulsada por la O-RAN Alliance, cuyo objetivo es permitir desplegar redes RAN con componentes de distintos fabricantes de forma interoperable, en lugar de depender de una solución cerrada de un único proveedor, incorporando además un controlador de inteligencia de red (*RAN Intelligent Controller*, RIC) para la gestión y optimización dinámica de la red [7].

Esta tendencia hacia la desagregación no es ajena al propio ecosistema de este trabajo: srsRAN Project, cuyo uso se plantea en este documento, ha pasado en 2026 a denominarse **OCUDU**, bajo la gobernanza de la Linux Foundation, centrando explícitamente su desarrollo en los componentes **CU y DU** de una arquitectura O-RAN, y estando pensado para operar junto a otros componentes O-RAN (RU, *core*, RIC) de terceros [10]. No obstante, conviene precisar que, en el despliegue realizado en este trabajo, no se hace uso de una arquitectura O-RAN desagregada: tanto la DU como la CU se ejecutan de forma conjunta en un único proceso monolítico sobre el mismo equipo que aloja la tarjeta SDR, sin separación física de la RU (la propia bladeRF actúa como tal) ni utilización de un RIC. La alineación con O-RAN es, en este caso, una característica del *software* empleado más que un rasgo de la arquitectura desplegada.

Radio definida por software - SDR

Antes de justificar la elección concreta del hardware SDR utilizado en este trabajo, conviene explicar en qué consiste esta tecnología y qué alternativas existen.

Una **radio definida por software** (*Software Defined Radio*, SDR) es un equipo de radiofrecuencia en el que las funciones que de forma tradicional se implementaban mediante componentes analógicos o circuitería digital, son implementadas en su lugar mediante *software* ejecutado sobre un procesador genérico o una FPGA. Los dispositivos SDR típicos cuentan con un *frontend* de radiofrecuencia que se encarga de trasladar la señal entre la banda de radio y una frecuencia intermedia o banda base, un conversor analógico-digital y digital-analógico (ADC/DAC) que convierte la señal entre los dominios analógicos y digitales, y un microcontrolador que se encarga de gestionar el flujo de



muestras hacia y desde un host, donde el resto del procesamiento de la señal se ejecuta mediante *software*. Este comportamiento es el que permite que un mismo dispositivo físico pueda usarse bien para implementar un gNB 5G, una estación base 4G o cualquier otro sistema de comunicaciones de radio, simplemente cambiando el *software* que se ejecuta sobre él.

En el mercado existen varias familias de dispositivos SDR, entre las que cabe destacar:

- **USRP** (*Universal Software Radio Peripheral*), de Ettus Research: es la familia más extendida, con modelos que van desde equipos de baja gama (B200/B210) hasta plataformas de alto rendimiento orientadas a despliegues industriales.
- **bladeRF**, de Nuand: es la alternativa de menor coste respecto a las ofertas de USRP, con soporte de fábrica para operación en 2×2 MIMO en su modelo 2.0 micro.
- **LimeSDR**, de Lime Microsystems: es una plataforma orientada a comunicaciones celulares, que además ofrece de forma íntegra los planos y el diseño de la placa bajo licencia abierta [11].
- **HackRF One**, de Great Scott Gadgets: es un dispositivo de bajo coste orientado principalmente a la exploración del espectro radioeléctrico y la seguridad, pero presenta una limitación clave para un caso de uso como el de este trabajo, y es que se cuenta con un transceptor **semidúplex** (*half-duplex*), es decir, que solo puede transmitir o recibir información en cada instante, pero no ambas cosas de forma simultánea [12], lo que lo hace inadecuado para implementar un gNB, que necesita operar en dúplex.

De todas estas opciones listadas, únicamente USRP, bladeRF y LimeSDR ofrecen la operación en dúplex, que como hemos mencionado antes, necesitamos para la implementación de un gNB. Esto además es respaldado porque son las únicas familias que aparecen citadas como compatibles con srsRAN en el estudio que mencionamos anteriormente[8]. La elección concreta del modelo utilizado en este trabajo, frente al empleado en el proyecto predecesor, se justifica con detalle en el Capítulo de Metodología.



Monitorización del *Core* 5G

Conocer el estado interno de un núcleo 5G en ejecución resulta imprescindible en cualquier despliegue, y de forma particular en el ámbito de las redes privadas, donde no existe un sistema de gestión de red comercial preexistente sobre el que apoyarse: qué funciones de red están activas, cuántos equipos de usuario están conectados y qué sesiones de datos tienen abiertas son preguntas recurrentes en este contexto. En el ámbito de las implementaciones de código abierto, coexisten distintos mecanismos para obtener esta información, cada uno con un grado de madurez y de estructuración distinto.

El mecanismo más directo, y disponible desde las primeras versiones de cualquier implementación, es el **análisis de los ficheros de registro** (*logs*) que cada función de red genera durante su funcionamiento normal. Este enfoque no requiere ninguna funcionalidad adicional por parte del propio *software* del núcleo, más allá de un registro suficientemente detallado, pero presenta una limitación estructural: la información se encuentra en forma de texto libre, pensado para que lo lea una persona, no para ser consumido de forma automática por otro programa. Cualquier herramienta que dependa de este mecanismo debe recurrir a la búsqueda de patrones de texto (habitualmente expresiones regulares) sobre dicho registro, lo que la hace frágil frente a cambios en el formato exacto de los mensajes entre versiones del *software*, y limita la información disponible a lo que, por decisión de quien desarrolló esa parte del código, haya quedado reflejado en algún mensaje de registro concreto.

Como alternativa más estructurada, ha ido consolidándose en el ecosistema de monitorización del *software* el uso de **métricas en formato Prometheus**: un formato de texto simple pero estructurado (pares de nombre de métrica, etiquetas y valor numérico), expuesto por la propia aplicación a través de un *endpoint* HTTP, y pensado para ser recolectado periódicamente por un sistema externo. Prometheus, el propio proyecto que da nombre a este formato, es un proyecto de código abierto de la *Cloud Native Computing Foundation* (la misma fundación que aloja Kubernetes), y su formato de exposición de métricas se ha convertido en un estándar de facto empleado por numerosos proyectos de *software* ajenos al propio Prometheus, sin necesidad de utilizar el resto de su ecosistema



(base de datos de series temporales, sistema de alertas) [13]. Frente al análisis de *logs*, este mecanismo ofrece una fuente de datos ya estructurada y pensada para su consumo automático, aunque limitada, por su propio diseño, a valores numéricos agregados (contadores, medidores), sin capacidad de representar de forma natural información más rica sobre entidades individuales (como los datos concretos de un equipo de usuario conectado).

Por último, algunos proyectos incorporan además **APIs propias en formato JSON**, expuestas también mediante HTTP, específicamente pensadas para complementar las métricas agregadas con el detalle de entidades individuales que estas no pueden representar. Este mecanismo, al ser específico de cada proyecto y no seguir un estándar compartido entre distintas implementaciones de núcleo 5G, resulta el menos generalizable de los tres, pero es también el que puede ofrecer una mayor riqueza de información cuando existe.

La disponibilidad concreta de cada uno de estos tres mecanismos en Open5GS, la comparativa entre ellos para el caso concreto de este trabajo, y la elección final realizada, se desarrollan con detalle en el Capítulo de Metodología (Sección 4.1.3).

Antecedentes y punto de partida del trabajo

Como se ha mencionado durante este capítulo, este trabajo no parte de cero, sino que continúa el desarrollado por un antiguo alumno, Antonio Hidalgo Fernández [1], quien implementó un panel de monitorización web para una red privada srsRAN/Open5GS sobre una tarjeta USRP B200. Dicho panel, construido con Flask y Socket.IO, obtenía la práctica totalidad de su información mediante el análisis de los registros del núcleo, según el mecanismo descrito en la sección de este capítulo dedicada a la monitorización del *Core* 5G, limitado en la práctica al registro del AMF, y únicamente en tiempo real: el panel mostraba las líneas nuevas a medida que se generaban, sin ofrecer ninguna vía para consultar el contenido ya existente del fichero ni para buscar información dentro de él. Para el tráfico histórico de la red, el trabajo predecesor recurre a un conjunto de herramientas de terceros (*ntopng*, *InfluxDB* y *Grafana*), y para la inspección de tráfico en vivo, a una



captura de paquetes mediante `tcpdump` retransmitida al navegador por *WebSocket*. El alta de nuevos suscriptores, por su parte, no se realizaba desde el propio panel, sino a través de la interfaz web oficial de Open5GS, ajena a la aplicación desarrollada. Ese trabajo incluye, además, a modo de anexo, una prueba preliminar de uso de una tarjeta bladeRF con srsRAN mediante una capa de compatibilidad SoapySDR, sin llegar a integrarla como vía principal del proyecto ni a consolidarla como una solución robusta.

Este Trabajo de Fin de Grado recoge dicho trabajo como punto de partida, y amplía su alcance en dos direcciones principales. En primer lugar, de la **monitorización de solo lectura** a la **gestión activa** de la infraestructura desde el propio panel (arranque, parada y reconfiguración del núcleo y de la estación base, y administración de suscriptores sin salir de la aplicación), sustentando además dicha monitorización sobre los mecanismos estructurados de Open5GS (Sección 4.1.3) en lugar de sobre el análisis de *logs* empleado originalmente. En segundo lugar, retomando la prueba preliminar con bladeRF ya mencionada, se ha explorado su consolidación como segunda vía de radio junto a la USRP B210 empleada como principal, documentándose las incidencias técnicas reales encontradas durante dicha exploración, descritas con detalle en el Capítulo de Implementación y Desarrollo, sin llegar a completarse con éxito dentro del tiempo disponible para este trabajo.

Capítulo 4

Metodología

En este capítulo se describe el enfoque de trabajo seguido durante el desarrollo del proyecto, así como los criterios empleados para tomar las decisiones técnicas que condicionan el resto del documento. A diferencia del Capítulo de Estado del Arte, donde se presentaban las alternativas disponibles de forma general, aquí se justifica la elección concreta realizada en este trabajo para cada una de ellas.

La Figura 4.1 resume la arquitectura propuesta y desarrollada en este trabajo, mostrando de forma conjunta el plano de radio y datos (equipo de usuario, estación base y núcleo de red) y el plano de gestión (panel de monitorización y gestión, con su *backend* y *frontend*), cuyas decisiones de diseño se justifican en las siguientes secciones.

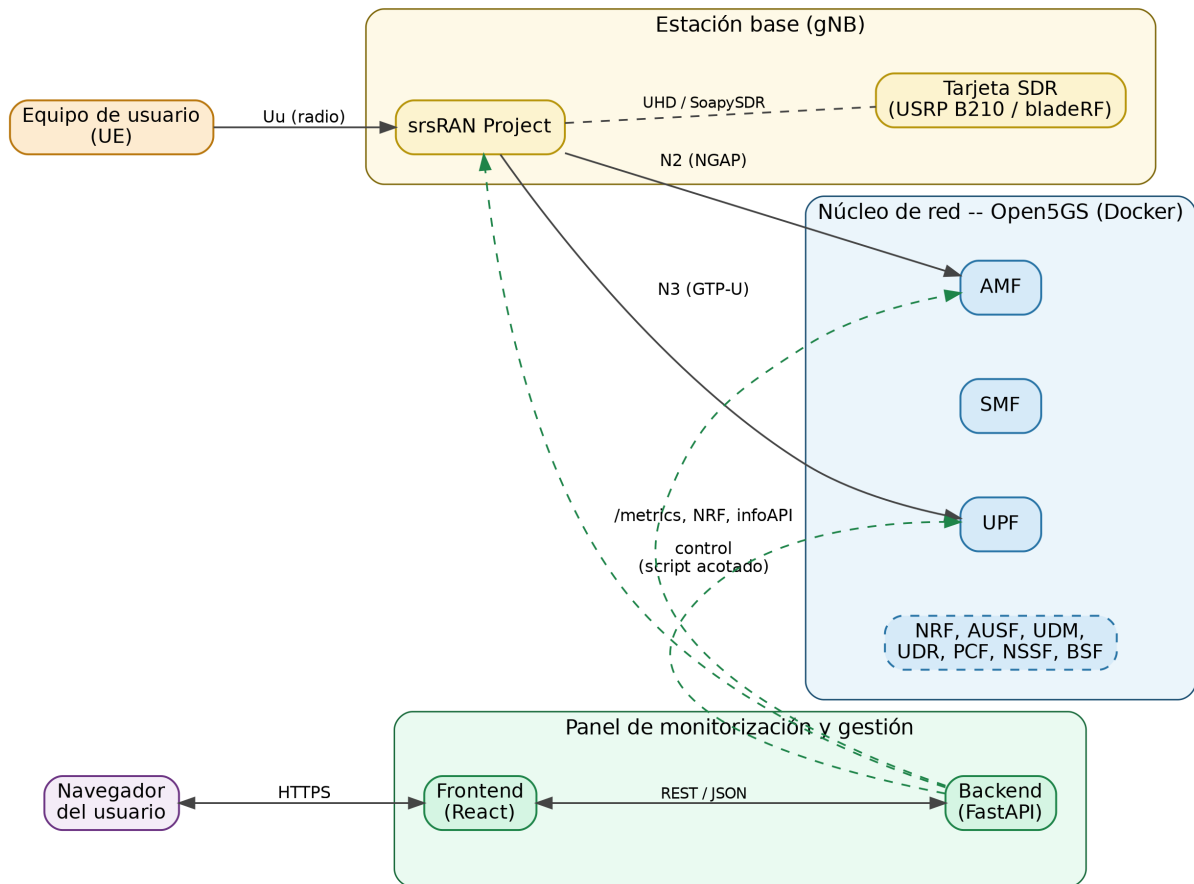


Figura 4.1: Arquitectura del proyecto, planos de radio y de gestión
Fuente: Propia

4.1. Selección de tecnologías

4.1.1. Elección del núcleo de red: Open5GS

Como se ha mencionado en el Capítulo de Estado del Arte, el proyecto predecesor emplea **Open5GS** como núcleo de red [1], decisión que este trabajo mantiene por continuidad directa con dicho proyecto. No obstante, y aunque la elección venga condicionada por el trabajo previo, resulta pertinente justificar por qué dicha continuidad no supone una limitación frente a las alternativas de código abierto disponibles.

Entre los núcleos 5G de código abierto más empleados en el ámbito académico se encuentran, además de Open5GS, **free5GC** (desarrollado en Go, convertido en proyecto



de la Linux Foundation en 2024) y **OpenAirInterface CN5G**. Un estudio comparativo reciente entre los tres, realizado sobre un banco de pruebas 5G *Standalone* con hardware comercial, concluye que Open5GS ofrece las mejores latencias en los procedimientos del plano de control, frente a las mayores tasas de datos de OpenAirInterface y el menor consumo de recursos de free5GC [14]. Otros trabajos de referencia sitúan igualmente a Open5GS entre las opciones más eficientes en consumo de CPU de las tres [15]. Cabe señalar que free5GC implementa un mayor número de funciones de red (10 frente a las 8 de Open5GS) y ofrece un control de acceso basado en OAuth 2.0 conforme a la especificación 3GPP Release 17, no disponible en Open5GS [15], si bien esta característica no resulta determinante para el alcance de este trabajo.

Dado que las latencias del plano de control resultan especialmente relevantes para un panel que consulta periódicamente el estado del núcleo (Sección 4.3.3), y que la eficiencia de recursos condiciona la posibilidad de ejecutar el núcleo completo en equipos de prestaciones modestas, una circunstancia real de este trabajo, validada en la Sección 5.14, se considera que Open5GS continúa siendo una elección técnicamente sólida para este trabajo, más allá de la propia continuidad con el proyecto predecesor.

4.1.2. Elección del hardware SDR: bladeRF frente a USRP B210

Como se ha mencionado en el Capítulo de Estado del Arte, el proyecto predecesor, del cual se toma el relevo, empleó una tarjeta **USRP B210**. Esta ofrece una cobertura continua de 70 MHz a 6 GHz y hasta 56 MHz de ancho de banda en tiempo real mediante un único canal, apoyándose en el controlador UHD (*USRP Hardware Driver*) [16], y cuenta con soporte nativo en prácticamente todo el software SDR de código abierto, incluido srsRAN.

Este trabajo emplea en su lugar una tarjeta **bladeRF 2.0 micro**, que presenta un rango de frecuencias equivalente (47 MHz a 6 GHz) y una capacidad de funcionar en 2×2 MIMO, algo que la B200 no es capaz de hacer de base (su variante superior, la B210, sí que es capaz de esto) [17]. Otro aspecto relevante para los objetivos que se intentan conseguir



en este trabajo es el grado de apertura que presenta el propio hardware. Mientras que la documentación de Nuand (empresa responsable de bladeRF) indica que las librerías, *firmware* y el diseño de la propia tarjeta se publican bajo licencias de código abierto, la documentación que ofrece Ettus (tarjetas B200/B210), a pesar de describir su FPGA como “abierta y reprogramable”, sitúa el acceso a los esquemas y a otra información de diseño de las tarjetas tras la redacción de una solicitud a través de su sistema de soporte [18]. Esta diferencia ayuda a equilibrar la balanza hacia el lado de bladeRF, al menos para un proyecto en el que uno de sus objetivos es construir una aplicación utilizando tecnología abierta, y no únicamente *software* de código abierto ejecutado sobre hardware cerrado.

La contraparte de esta elección es que, a diferencia de la USRP B210, la tarjeta bladeRF no cuenta con soporte nativo en srsRAN, lo que lleva a necesitar una capa de compatibilidad adicional, algo que se desarrolla en detalle en el Capítulo de Implementación y Desarrollo.

Cabe adelantar, para evitar confusión al lector, que esta fue la elección **inicial** de hardware, y la que se justifica en esta sección; sin embargo, una incidencia de conexión sin resolver con la bladeRF llevó a introducir la USRP B210 (la misma tarjeta empleada por el trabajo predecesor) como prueba de control durante el desarrollo. Dicha tarjeta terminaría por convertirse en la vía de radio **principal** empleada a lo largo de la mayor parte de este trabajo, incluyendo la totalidad del desarrollo del panel de monitorización y gestión y la validación final con un equipo de usuario real, mientras que la bladeRF quedó como una segunda vía explorada, sin llegar a completarse con éxito. El proceso completo que lleva a este cambio se documenta con detalle en el Capítulo de Implementación y Desarrollo y en el Anexo de solución de problemas.

4.1.3. Elección del mecanismo de monitorización del *Core*

Una de las formas más directas y comunes de conocer el estado interno de un núcleo Open5GS es mediante el análisis de sus ficheros de registro (*logs*), en los que cada NF vuelca mensajes de texto sobre eventos de red (altas y bajas de sesión, registro de gNBs, etcétera). Concretamente, esta fue la aproximación seguida por el trabajo en el que se



basa este, cuyo *dashboard* obtenía toda la información mostrada (número de gNBs y UEs conectados, IP y APN de cada sesión) mediante la búsqueda de patrones de texto concretos (usando expresiones regulares) en los registros del AMF y del SMF [1].

Este enfoque, aunque es algo completamente funcional y válido, resulta frágil. Esto se debe a que depende del formato textual de los mensajes de *log* y de que este no cambie entre versiones del *software*, y además no ofrece visibilidad sobre el estado individual de cada NF más allá de lo que quede reflejado en esos mensajes concretos.

Como alternativa a esta debilidad, Open5GS incorpora desde hace varios años un mecanismo de métricas en formato Prometheus, expuesto por cada NF en un puerto HTTP propio. Se debe mencionar que esta funcionalidad no es reciente, pues su incorporación fue propuesta y discutida públicamente por el propio equipo de desarrollo ya en 2022 [19], y para el año 2023, usos y despliegues estables de este mecanismo fueron documentados. Entre las ventajas que ofrece está permitir conocer, entre otros datos, el número de UEs activos por NF, lo que resulta suficiente para determinar si una función de red concreta está operativa (si responde a una petición HTTP, lo está), pero también es cierto que, por sí sola, esta vía no permite obtener detalles por cada UE (información como el IMSI, sesiones, IP asignada, etc.), algo que sí proporcionaba el análisis de logs del trabajo predecesor.

Ese nivel de detalle sí es posible alcanzarlo mediante una funcionalidad bastante más reciente: un conjunto de *endpoints* JSON (*/ue-info*, */pdu-info*, */gnb-info*) añadidos sobre ese mismo servidor HTTP, implementados por la comunidad de Open5GS como una vía dedicada para sustituir el análisis de logs por una fuente de datos estructurada. La propia discusión de implementación de la comunidad del proyecto señala expresamente que, hasta que se incorporó esta funcionalidad, Open5GS no disponía de ninguna otra forma de obtener la información que se busca, salvo analizando los logs [20], lo cual permite entender mejor la limitación con la que se encontró el anterior alumno. Al tratarse de una funcionalidad reciente, no se encuentra incluida todavía en los paquetes distribuidos de Open5GS en el momento de redactar este texto, lo que obliga a compilar el propio Open5GS desde su rama de desarrollo principal.



En el ámbito comercial, productos como OAIBOX ofrecen *dashboards* más completos, con indicadores de estado por cada NF del núcleo, métricas de rendimiento en tiempo real y herramientas de inspección de tráfico integradas. Aunque dicho producto utiliza una arquitectura distinta a la de este trabajo (OpenAirInterface en lugar de srsRAN/Open5GS) y no existe relación técnica entre ambos proyectos, su catálogo de funcionalidades resulta útil como referencia del tipo de información que un panel de este tipo puede ofrecer.

Para este trabajo se decide evolucionar el mecanismo empleado por el proyecto predecesor, sustituyendo el análisis de *logs* por los *endpoints* JSON descritos anteriormente, al ofrecer una fuente de datos estructurada y no depender del formato textual de los mensajes de registro.

4.1.4. Elección entre srsRAN Project y OCUDU

Como se ha mencionado en el Capítulo de Estado del Arte, durante el desarrollo de este trabajo srsRAN Project pasó a denominarse **OCUDU**, bajo la gobernanza de la Linux Foundation [10]. Esto plantea una decisión adicional no prevista al inicio del proyecto: continuar con la última versión congelada de srsRAN Project, o adoptar directamente OCUDU como plataforma activa.

Para este trabajo se opta por emplear la última versión congelada de **srsRAN Project** (rama `main` del repositorio archivado), principalmente para mantener la comparación con el trabajo predecesor [1] en condiciones equivalentes, empleando la misma base de software que dicho trabajo. Queda como posible ampliación, si el desarrollo del resto del proyecto lo permite, evaluar el funcionamiento del sistema desarrollado sobre OCUDU, comprobando si la migración es directa dado que ambos proyectos comparten arquitectura.

4.2. Entorno y recursos empleados

Además de las decisiones de arquitectura ya justificadas, este trabajo se ha desarrollado sobre un conjunto concreto de equipos y dispositivos físicos, cuyo empleo se documenta a



continuación.

4.2.1. Equipos de desarrollo

El desarrollo del panel de monitorización y gestión, así como la ejecución del propio gNB y del núcleo Open5GS, se han llevado a cabo sobre dos equipos distintos, cuyas especificaciones se recogen en la Tabla 4.1. El empleo de dos equipos, en lugar de uno solo, responde a la necesidad práctica de continuar el desarrollo en distintas ubicaciones a lo largo del proyecto, lo que a su vez motivó validar la portabilidad de la solución desarrollada como parte de los objetivos de este trabajo (Sección 5.14).

	Sobremesa	Portátil
Fabricante / modelo	MSI, MS-7E27	HP Laptop 15s-eq2xxx
CPU	AMD Ryzen 5 7600X (6C/12H)	AMD Ryzen 5 5500U (6C/12H)
RAM	32 GB	8 GB
Almacenamiento	741 GB	53 GB
Sistema operativo	Debian 13 (trixie)	Debian 13 (trixie)

Tabla 4.1: Especificaciones de los dos equipos empleados durante el desarrollo.

4.2.2. Hardware de radio

La Figura 4.2 muestra la tarjeta USRP B210 empleada como SDR principal de este trabajo (Sección 4.1.2), conectada al equipo de desarrollo mediante un cable USB-A a USB-B (Figura 4.3) al que se ha conectado un adaptador USB-A a USB-C puesto que el portátil no cuenta con puertos USB 3.0, necesarios para soportar la tasa de datos de la tarjeta.



Figura 4.2: Tarjeta USRP B210 empleada en este trabajo
Fuente: Propia



Figura 4.3: Cable USB-A a USB-B con adaptador USB-A a USB-C
Fuente: Propia

4.2.3. Equipo de usuario y programación de la SIM

Como equipo de usuario para la validación de este trabajo (Sección 5.7) se ha empleado un teléfono móvil **Google Pixel 8** (Figura 4.4), con soporte confirmado de 5G *Standalone* en la banda n78 empleada en este trabajo.

La programación de la tarjeta SIM de pruebas se ha realizado con un lector de tarjetas SIM de propósito general (Figura 4.5) y la aplicación **GRSIMWrite** (Figura 4.7), ejecutada sobre un ordenador portátil Lenovo disponible por parte del laboratorio de investigación GITACA (Figura 4.6).



Figura 4.4: Google Pixel 8, UE empleado
Fuente: Propia



Figura 4.5: Lector de tarjetas SIM empleado
Fuente: Propia



Figura 4.6: Portátil del laboratorio empleado con GRSIMWrite
Fuente: Propia

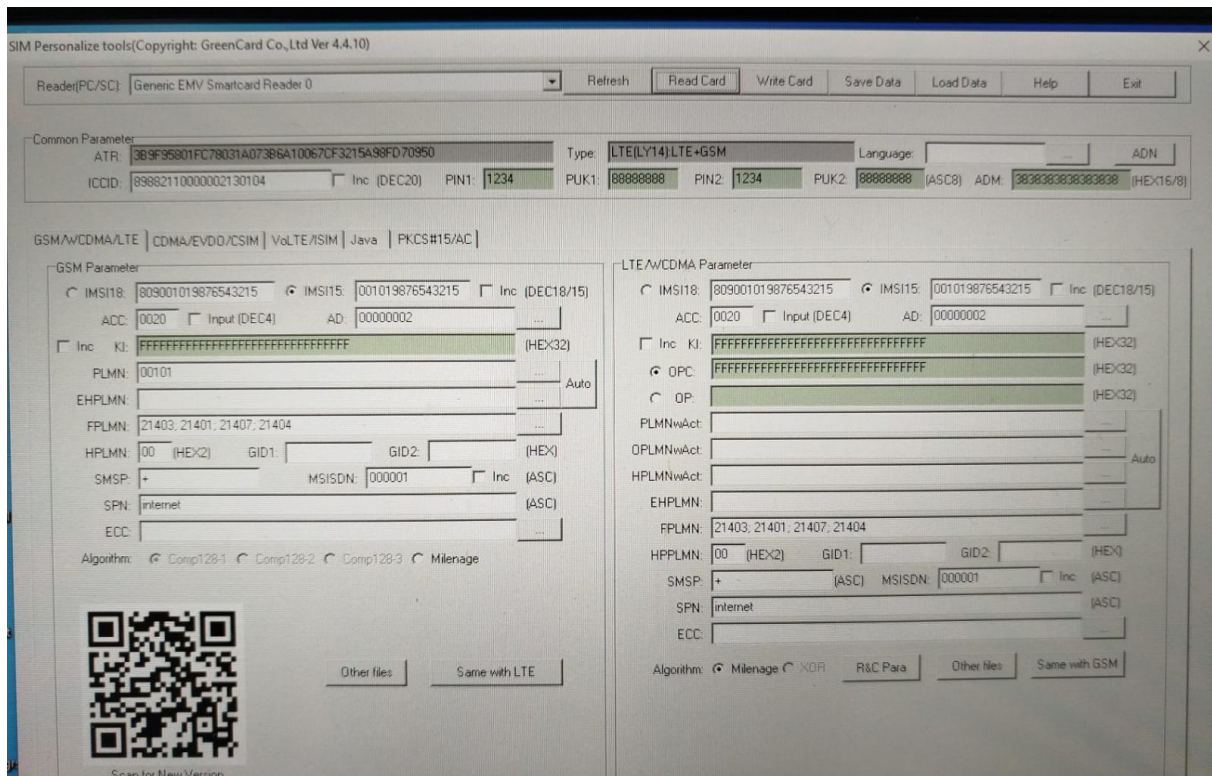


Figura 4.7: GRSIMWrite, empleado para programar la SIM
Fuente: Propia

4.3. Diseño del panel de monitorización

Antes de comenzar la implementación del panel de monitorización (desarrollada en el Capítulo de Implementación y Desarrollo), se establecieron una serie de decisiones de diseño que condicionan su arquitectura. Estas decisiones se documentan aquí, siguiendo el mismo criterio que el resto de este capítulo: no como una descripción de lo que existe, sino como la justificación razonada de la alternativa elegida frente a las demás consideradas.

4.3.1. Alcance: de la monitorización a la gestión

El trabajo predecesor [1] limitaba su panel a la monitorización de solo lectura: mostrar el estado de la red, sin capacidad de actuar sobre ella. Para este trabajo se decide ampliar el alcance hacia la **gestión** de determinados parámetros de la red desde el propio panel, concretamente: la banda/ARFCN, el ancho de banda de canal y las ganancias de



transmisión/recepción del gNB; el arranque y parada individual de las funciones de red del *core*; y el alta y baja de suscriptores sin necesidad de emplear el WebUI de Open5GS. Esta decisión constituye una de las aportaciones diferenciales de este trabajo frente al predecesor, si bien introduce un cambio cualitativo en los requisitos de la aplicación: un panel de solo lectura puede, en el peor caso, mostrar información incorrecta; un panel con capacidad de actuación puede, en el peor caso, interrumpir el servicio de la propia red que administra. Esta distinción motiva las decisiones de seguridad descritas en la Subsección 4.3.4.

4.3.2. Arquitectura del *backend*

El trabajo predecesor empleaba Flask junto con Socket.IO para la comunicación en tiempo real con el cliente. Para este trabajo se opta por modernizar esta capa, empleando en su lugar **FastAPI**. Esta elección se apoya en tres motivos, cada uno vinculado a otras decisiones de este mismo capítulo:

- **Validación y documentación automática de datos:** FastAPI emplea modelos de datos tipados (mediante la biblioteca Pydantic) tanto para las peticiones como para las respuestas de cada *endpoint*, generando a partir de ellos documentación interactiva de la API sin trabajo adicional. Esto resuelve de forma directa la necesidad de fijar un contrato de datos estable entre el *backend* y el cliente antes de implementar ambos por separado.
- **Operación asíncrona nativa:** dado que el *backend* debe consultar periódicamente varios *endpoints* de distintas funciones de red (Subsección 4.3.3), un modelo de ejecución asíncrono resulta más adecuado que el modelo síncrono por defecto de Flask.
- **Soporte nativo de *WebSocket*,** por si resultara conveniente complementar en el futuro la actualización por sondeo con una vía de comunicación más reactiva hacia el cliente.



4.3.3. Mecanismo de actualización de datos

Se valoraron dos vías para mantener actualizada la información mostrada por el panel. La primera, y la finalmente elegida, es el **sondeo periódico** (*polling*) de los *endpoints* de Open5GS descritos en la Subsección 4.1.3 (`/metrics`, `/ue-info`, `/pdu-info`), realizado cada **5 segundos** desde todas las vistas del panel que muestran datos en vivo (Resumen, Núcleo, Topología, gNB y Suscriptores) y con la misma cadencia por parte del muestreador de tráfico en segundo plano del *backend* (Sección 5.21). La segunda, descartada para este trabajo, es el uso del servicio estándar 3GPP de exposición de eventos del AMF (`Namf_EventExposure`), al que es posible suscribirse mediante una petición a `/namf-evts/v1/subscriptions` indicando una URI de notificación propia, de forma que sea el propio núcleo quien notifique activamente los cambios de estado en lugar de tener que preguntarlos [21]. Se descarta esta segunda vía para el alcance de este trabajo por su mayor complejidad de integración, ya que requiere implementar un receptor de notificaciones propio y gestionar la semántica de suscripción de eventos definida por 3GPP, y por tratarse de una funcionalidad con menor documentación disponible que el infoAPI ya validado, quedando anotada como posible ampliación futura.

4.3.4. Seguridad: privilegio mínimo y autenticación configurable

Como se ha señalado en la Subsección 4.3.1, la capacidad del panel para actuar sobre la infraestructura real (deteniendo funciones de red, reiniciando el gNB con otra configuración de radio) introduce un requisito de seguridad que no existía en el trabajo predecesor. Se han tomado dos decisiones de diseño al respecto.

Privilegio mínimo en la ejecución de acciones sobre el sistema. El *backend* no ejecuta ninguna operación sobre la infraestructura con privilegios de superusuario de forma directa. En su lugar, las acciones que requieren dichos privilegios (parada y reinicio del proceso del gNB, entre otras) se delegan en un script de control acotado, análogo al ya empleado para la orquestación del *core* nativo, al que se concede una regla específica de `sudoers` limitada a la ejecución de ese script concreto sin solicitud de contraseña, en



lugar de conceder al proceso del *backend* acceso genérico a `sudo`. De este modo, un fallo de seguridad en el propio *backend* queda acotado a las operaciones que ese script permite realizar, en vez de comprometer el sistema completo.

Autenticación activada por defecto. El acceso a los *endpoints* que modifican el estado del sistema (creación/eliminación de suscriptores, cambios de configuración del gNB, arranque/parada de funciones de red) requiere autenticación mediante token (JWT), obtenido tras un inicio de sesión. Dado que el panel puede desplegarse en contextos muy distintos, desde el uso individual del propio autor hasta la posible reutilización por parte de otro estudiante o la evaluación por parte de un tribunal, se ha optado por que este mecanismo esté **activado de forma predeterminada**, pudiendo desactivarse explícitamente mediante una variable de entorno para contextos de uso individual donde no se considere necesario. Esta decisión sigue el principio de diseño *fail-safe defaults* (“ausencia de acceso por defecto”), formulado por Saltzer y Schroeder como uno de los principios clásicos de diseño de sistemas seguros [22]: ante la duda de si el mecanismo de protección se necesita o no en un contexto concreto, el sistema debe partir del estado más restrictivo, siendo una decisión explícita del operador relajar esa restricción, y no al revés. Como medida adicional, cuando la autenticación se encuentra desactivada, el *backend* emite un aviso visible en su registro de arranque, de forma que dicha circunstancia quede documentada y no pase inadvertida.

Capítulo 5

Implementación y desarrollo

Este capítulo describe el trabajo realizado durante todas las fases del proyecto: la construcción de la infraestructura de radio y núcleo, y el desarrollo del panel de monitorización y gestión sobre ella. Se ha escrito como una guía de lo construido y de cómo ponerlo en funcionamiento, justificando en cada caso el porqué de las decisiones tomadas. Las incidencias reales encontradas durante el desarrollo, con su proceso completo de diagnóstico, se documentan por separado en el Anexo D, al que se referencia puntualmente desde cada sección afectada.

5.1. Preparación del entorno base

A diferencia del proceso seguido por el trabajo predecesor en Ubuntu, que requería compilar `libbladeRF` desde código fuente [1], se comprobó que Debian Trixie empaqueta de forma nativa todas las herramientas necesarias para la bladeRF: `bladerf` (herramientas de línea de comandos, incluyendo `bladeRF-cli`), `libbladerf2/libbladerf-dev`, y paquetes de FPGA independientes por modelo concreto de tarjeta.

5.1.1. Instalación de los paquetes base

El paquete de *firmware* del controlador USB (`bladerf-firmware-fx3`) se distribuye dentro del componente `contrib` del repositorio de Debian, no habilitado por defecto



junto a `main` y `non-free-firmware`. Antes de instalar los paquetes, es necesario añadirlo al fichero `/etc/apt/sources.list`:

```
1 deb http://deb.debian.org/debian/ trixie main contrib non-free-firmware
2 deb-src http://deb.debian.org/debian/ trixie main contrib
```

Con los índices de paquetes actualizados, la instalación se completa sin más incidencias (ver Anexo D.1 para el diagnóstico completo de este requisito):

```
1 sudo apt update
2 sudo apt install bladerf libbladerf2 libbladerf-dev bladerf-firmware-fx3
```

5.1.2. Detección e identificación del modelo

Con la tarjeta conectada por USB 3.0, se verificó su detección a nivel de sistema:

```
1 lsusb | grep -i nuand
2 bladeRF-cli -p
```

El resultado confirmó una **bladeRF 2.0 micro**, con número de serie `6fa70090da274161b386fed5b3fe6b3c`. Este dato resultó especialmente relevante: coincide exactamente con el número de serie que aparece en el registro incluido en el Anexo B del trabajo predecesor [1], confirmando que se trata de la misma unidad física empleada en aquel trabajo, y no de una tarjeta distinta del mismo modelo.

El modelo exacto (necesario para instalar la imagen de FPGA correcta, ya que Debian distribuye una por variante) se determinó mediante:

```
1 bladeRF-cli -e info
```

El campo `FPGA size: 49 KLE` identificó la tarjeta como una variante **xA4** (la FPGA Cyclone V 5CEA4 monta 49 KLE, frente a los 301 KLE del modelo 5CEA9 de la variante xA9), lo que permitió instalar el paquete de FPGA correspondiente:



```
1 sudo apt install bladerf-fpga-hostedxa4
```

5.1.3. Versiones finales del entorno

La Tabla 5.1 recoge las versiones finales verificadas mediante `bladeRF-cli -e version`, útiles para la reproducibilidad del entorno.

Componente	Versión
Modelo	bladeRF 2.0 micro xA4
bladeRF-cli	1.9.0-0.2024.05-1
libbladeRF	2.5.0-0.2024.05-1
Firmware	2.4.0-git-a3d5c55f
FPGA	0.15.0
Origen de los paquetes	Repositorios oficiales de Debian Trixie (<code>main+contrib</code>)

Tabla 5.1: Versiones del entorno bladeRF verificadas en la Fase 1.

Con la tarjeta detectada, identificada y con la FPGA cargada correctamente (`FPGA loaded: yes`), se dio por cerrada la Fase 1: la tarjeta funciona de forma aislada, sin intervención todavía de srsRAN ni de Open5GS.

5.2. Instalación de SoapySDR

Como se explica en el Capítulo de Estado del Arte, srsRAN no ofrece soporte nativo para bladeRF, por lo que es necesario intercalar una capa de compatibilidad (SoapySDR, SoapyBladeRF y SoapyUHD) que presente la tarjeta como si fuera un dispositivo UHD. A diferencia del Anexo B del trabajo predecesor, que compilaba estos tres componentes desde código fuente, se comprobó que Debian Trixie los empaqueta también de forma nativa:

```
1 sudo apt install soapysdr-tools libsoapysdr-dev soapysdr-module-bladerf \
2 uhd-soapysdr libuhd-dev uhd-host
```



La cadena completa de detección se verificó en dos niveles independientes. Primero, a nivel de SoapySDR directamente:

```
1 SoapySDRUtil --info
2 SoapySDRUtil --find
```

Ambos comandos confirmaron que los módulos `libbladeRFsupport.so` y `libuhdSupport.so` cargaban correctamente y que la tarjeta era detectada (`driver = bladerf`) con su número de serie. Segundo, y de forma más relevante, a nivel de UHD, la vía que realmente empleará srsRAN:

```
1 uhd_find_devices
```

Su salida mostró el dispositivo con `type: soapy` y `driver: bladerf`, confirmando que la cadena completa `UHD → SoapyUHD → SoapyBladeRF → libbladeRF` funcionaba de extremo a extremo sin necesidad de compilar ningún componente en esta primera verificación.

5.3. Obtención y compilación de srsRAN Project

Siguiendo la decisión justificada en la Sección 4.1.4 de emplear la última versión congelada de srsRAN Project, se clona explícitamente la rama `main` del repositorio archivado, necesario dado que su rama por defecto ya no contiene el código fuente, sino únicamente un aviso de la migración a OCUDU (ver Anexo D.2 para el diagnóstico completo), y se compila de la forma habitual:

```
1 git clone -b main https://github.com/srsran/srsRAN_Project.git
2 cd srsRAN_Project
3 mkdir build && cd build
4 cmake ../
5 make -j$(nproc)
```



Esto genera, entre otros binarios, el ejecutable `gnb`, con el que se despliega la estación base. Se obtiene la versión **25.10.0** (*commit* 4bf1543936), verificable mediante `./gnb --version`.

5.4. Despliegue del núcleo Open5GS

5.4.1. Compilación e instalación

Al no existir un paquete de Open5GS con compatibilidad confirmada para Debian 13/Trixie en el repositorio de terceros habitual (OBS), se optó por compilar Open5GS desde código fuente mediante Meson, la vía oficialmente soportada:

```
1 sudo apt install python3-pip python3-setuptools python3-wheel ninja-build \
2   build-essential flex bison git cmake libsctp-dev libgnutls28-dev \
3   libgcrypt-dev libssl-dev libmongoc-dev libbson-dev libyaml-dev \
4   libnghttp2-dev libmicrohttpd-dev libcurl4-gnutls-dev libtins-dev \
5   libtalloc-dev libidn-dev meson
6
7 git clone https://github.com/open5gs/open5gs
8 cd open5gs
9 meson build --prefix=`pwd`/install
10 ninja -C build
11 cd build
12 ninja install
```

Para MongoDB (necesario para las funciones UDR/PCF), al no ofrecer el fabricante repositorio oficial para Debian en todas sus versiones, se empleó el repositorio de Ubuntu 22.04 (*jammy*), binariamente compatible:

```
1 curl -fsSL https://pgp.mongodb.com/server-8.0.asc | sudo gpg -o
   ↪ /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor
```

```
2 echo "deb [ arch=amd64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ]  
  ↪ https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/8.0 multiverse" |  
  ↪ sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list  
3 sudo apt update && sudo apt install -y mongodb-org  
4 sudo systemctl enable --now mongod
```

La instalación quedó ubicada en un directorio propio (`open5gs/install/`) en lugar de en las rutas estándar del sistema (`/etc/open5gs/`), al haberse especificado un `--prefix` local durante la compilación, una diferencia relevante a tener en cuenta frente a una instalación por paquete, ya que no se generan unidades `systemd` automáticas para cada función de red.

5.4.2. Configuración del dispositivo TUN

El plano de usuario (UPF) requiere un dispositivo de red virtual `ogstun` para encaminar el tráfico de datos. Se configuró mediante `systemd-networkd`:

```
1 # /etc/systemd/network/99-open5gs.netdev  
2 [NetDev]  
3 Name=ogstun  
4 Kind=tun  
5  
6 # /etc/systemd/network/99-open5gs.network  
7 [Match]  
8 Name=ogstun  
9 [Network]  
10 Address=10.45.0.1/16  
11 Address=2001:db8:cafe::1/48
```

Cabe señalar, como aclaración práctica útil durante esta fase, que la interfaz `ogstun` permanece en estado `no-carrier` (`configuring`) y sin la dirección IP aplicada hasta



que un proceso en espacio de usuario (en este caso, el propio UPF) abre el dispositivo `/dev/net/tun` y lo mantiene abierto, comportamiento normal y documentado del propio Open5GS, no un fallo de configuración.

5.4.3. Configuración del PLMN de pruebas

Se configuró inicialmente el mismo PLMN de pruebas empleado por el trabajo predecesor (MCC 901 / MNC 70, TAC 7) en los ficheros `nrf.yaml` y `amf.yaml` (bloques `serving`, `guami`, `tai` y `plmn.support`), por continuidad y para facilitar la comparación entre ambos trabajos. Este valor se sustituyó posteriormente por el PLMN reservado de pruebas 001/01, según se documenta en la Sección 5.24.

5.4.4. Automatización del arranque del núcleo

Dado que el despliegue consta de 11 funciones de red independientes (NRF, SCP, AMF, AUSF, UDM, UDR, PCF, NSSF, BSF, SMF y UPF; se excluyen las funciones exclusivas de 4G/NSA `mmed`, `hssd`, `sgwcd`, `sgwud` y `pcrfd`, no necesarias en un despliegue 5G SA), cada una de las cuales debe arrancarse en un orden concreto (el NRF en primer lugar, ya que el resto se registra contra él; el UPF en último lugar), se desarrolló un script de orquestación (`open5gs_ctl.sh`) que automatiza el arranque, la parada y la comprobación de estado de las 11 funciones con un único comando, registrando el PID y el log de cada una por separado. Este script se incluye como Anexo del presente documento.

5.5. Configuración del gNB con bladeRF

5.5.1. Fichero de configuración de partida

Como base de configuración se empleó el ejemplo `gnb_rf_b200_tdd_n78_20mhz.yml`, distribuido en el propio repositorio de srsRAN Project (banda n78, TDD, 20 MHz de ancho de banda), adaptándolo al entorno del presente trabajo según recoge la Tabla 5.2.

El cambio de `tx_gain` no fue arbitrario: la escala de ganancia que expone la bladeRF



Campo	Original (B200)	Adaptado
<code>amf.addr</code>	127.0.1.100	127.0.0.5 (AMF real de Open5GS)
<code>device_driver</code>	uhd (con <code>type=b200</code>)	uhd (sin forzar tipo)
<code>tx_gain</code>	80 dB	−20 dB
<code>plmn</code>	"00101"	"90170" (901/70)

Tabla 5.2: Adaptación del fichero de configuración de ejemplo del gNB.

a través de Soapy no coincide con la de una USRP, y el propio registro del Anexo B del trabajo predecesor advertía ya de un error de rango (*out-of-range*) al emplear valores de ganancia pensados para una B200 [1].

A partir de este fichero de partida, se llegó a la configuración final tras resolver tres incidencias encadenadas, cada una en una capa distinta de la pila (parseo de configuración, comportamiento interno de UHD, y temporización de los búferes de radio), documentadas con su proceso completo de diagnóstico en el Anexo D.3. La configuración final requiere, en concreto:

- `device_driver`: uhd de forma explícita (no vacío), dejando `device_args` sin el parámetro `type` para que UHD autodetecte el dispositivo disponible.
- Una versión de SoapyBladeRF compilada con una implementación de compatibilidad para `listTimeSources/getTimeSource/setTimeSource`, reportando ‘‘internal’’ como única fuente de tiempo válida, necesaria porque SoapyBladeRF no sobrescribe estos métodos de la clase base de SoapySDR, y UHD exige una fuente de tiempo soportada antes de abrir la radio.
- Los valores `TRANSMIT_TIMEOUT_S/RECEIVE_TIMEOUT_S` del propio código fuente de srsRAN fijados a cero, frente a sus valores por defecto (pensados para una USRP nativa), insuficientes para el flujo de muestras real de la bladeRF a través de SoapyUHD.

Este último punto tiene una implicación metodológica relevante para el resto del trabajo: los valores de *timeout* de transmisión y recepción de srsRAN no son universales, sino específicos de cada SDR, por lo que las ramas de compilación de la



bladeRF y de la USRP B210 (Sección 5.8) deben mantenerse en directorios de compilación separados, cada uno con el parche correspondiente aplicado sobre su propio árbol de código fuente.

5.6. Verificación: primer arranque estable

Tras aplicar la configuración anterior, el gNB arrancó y se mantuvo estable, sin errores de *streaming* en bucle, con la siguiente traza relevante:

```
1 Cell pci=1, bw=20 MHz, 1T1R, dl_arfcn=632628 (n78), dl_freq=3489.42 MHz,  
2 dl_ssb_arfcn=632256, ul_freq=3489.42 MHz  
3  
4 N2: Connection to AMF on 127.0.0.5:38412 completed  
5 ==== gNB started ===
```

Este resultado marca el cierre de la Fase 2: la bladeRF, integrada mediante la capa de compatibilidad SoapySDR y los parches descritos, transmite de forma estable con srsRAN Project y el gNB completa su registro contra un núcleo Open5GS real, sin que fuese necesario replicar íntegramente ninguno de los parches del trabajo predecesor tal cual estaban escritos, ya que cada uno de ellos requirió, en mayor o menor medida, ser reinterpretado y adaptado a las versiones de *software* empleadas en este trabajo.

5.7. Validación con un equipo de usuario real

Con el gNB transmitiendo de forma estable, se procedió a intentar la conexión de un teléfono móvil real, siguiendo el procedimiento descrito en el Anexo B del trabajo predecesor [1]: programación de una tarjeta SIM con las credenciales del suscriptor y configuración manual del punto de acceso (APN) en el terminal.

5.7.1. Programación de la SIM y alta del suscriptor

A diferencia del trabajo predecesor, que empleaba `pySim` sobre Linux, la tarjeta programable se grabó con **GRSIMWrite** (Windows), disponible en el equipo del laboratorio asignado para esta tarea (véase la Figura 4.7, en la Sección de Metodología). Se programaron los mismos valores en ambas secciones de parámetros que ofrece la herramienta (5G y LTE/WCDMA), dado que la clave `Ki` y el `OPc` son compartidos por ambas generaciones a nivel de aplicación USIM, y solo divergen las claves derivadas específicas de cada una:

```

1 IMSI: 901700000000001
2 Ki: 465B5CE8B199B49FAA5F0A2EE238A6BC
3 OPc: E8ED289DEBA952E4283B54E88E6183CA

```

Los valores de `Ki/OPc` corresponden al par de prueba ampliamente documentado y reutilizado en la comunidad de `srsRAN`, `Open5GS` y `free5GC`, en lugar de un valor arbitrario como el empleado en el trabajo predecesor, por resultar más fácilmente verificable por terceros. Se dio de alta el mismo suscriptor en el WebUI de `Open5GS`, con idénticos valores de `Ki/OPc/AMF`, `DNN internet` en modo IPv4 y `sst: 1`, coincidente con el *slice* configurado en el núcleo.

En un primer intento, el terminal no detectó en ningún momento la celda emitida por el gNB, pese a confirmarse mediante prueba directa que la tarjeta emitía señal real en la frecuencia correcta; el proceso completo de descarte sistemático de causas seguido en ese momento se documenta en el Anexo D.4. La resolución definitiva de esta incidencia, alcanzada en una fase muy posterior del trabajo, se documenta en la Sección 5.24.

5.8. Migración y configuración con la USRP B210

Ante la incidencia anterior, y dado que no se disponía de instrumental de medida de RF independiente para confirmar de forma concluyente el correcto funcionamiento de la cadena de radio completa, se decidió realizar una prueba de control: sustituir



temporalmente la bladeRF por la misma USRP B210 empleada por el trabajo predecesor, dejando el resto del sistema (núcleo, configuración de celda, PLMN) sin modificar, con el objetivo de aislar si la incidencia de conexión se debía al hardware bladeRF o a algo anterior en la cadena. Como se documenta en la Sección 5.24, esta tarjeta terminaría por convertirse en la vía de radio principal de este trabajo.

5.8.1. Configuración con UHD nativo

Al tratarse de una USRP real, srsRAN emplea UHD de forma nativa, sin la capa de compatibilidad SoapySDR requerida por la bladeRF. El fichero de configuración de ejemplo se adapta con `device_driver: uhd`, `device_args: type=b200`, y las ganancias de transmisión y recepción dentro del rango propio de UHD (Sección 5.23). Es necesario además descargar las imágenes de *firmware*/FPGA propias de UHD:

```
1 uhd_images_downloader
```

Este paso no hace falta con la bladeRF al venir esta integrada en los paquetes de Debian. Se fija además la variable de entorno `UHD_IMAGES_DIR` de forma explícita, ya que la ruta por defecto que reporta el propio mensaje de error de UHD no coincide con la que emplea el paquete de Debian Trixie.

Alcanzar una configuración estable con esta tarjeta exigió, sin embargo, resolver una incidencia de arquitectura no evidente a primera vista (los mismos valores de *timeout* de transmisión/recepción fijados a cero para la bladeRF, Sección 5.5, resultan excesivamente agresivos para una USRP nativa), cuyo proceso completo de diagnóstico, incluyendo el descarte sistemático de más de media docena de causas alternativas antes de aislar la real, se documenta en el Anexo D.5. La lección de arquitectura ya recogida al final de la Sección 5.5 (mantener una compilación de srsRAN independiente por cada SDR, cada una con su propio valor de *timeout*) es consecuencia directa de esta incidencia.



5.9. Despliegue del núcleo Open5GS en Docker

Como se indica en la Metodología, el despliegue nativo del núcleo Open5GS (Sección 5.4) se adoptó como decisión deliberada frente al uso de Docker previsto inicialmente en la hoja de ruta, para reducir el número de variables durante la fase de depuración de la integración de radio. Una vez desacoplado el desarrollo del panel de monitorización de la resolución de las incidencias de radio descritas en las secciones anteriores, se retomó el despliegue en contenedores, aprovechando además que el núcleo nativo ya validado sirve de referencia de comportamiento correcto contra la que contrastar el despliegue en Docker.

5.9.1. Selección del repositorio base

Se optó por el repositorio comunitario `Borjis131/docker-open5gs` [23], que ofrece imágenes independientes por función de red (no un único contenedor monolítico) y un perfil de despliegue (*basic*) preparado explícitamente para operar con un gNB externo al propio Docker, publicando los puertos N2 (SCTP 38412, en el AMF) y N3 (UDP 2152, en el UPF) hacia el host.

5.9.2. Instalación de Docker en Debian Trixie

Se instaló Docker Engine desde el repositorio oficial del propio Docker, ya que el paquete `docker.io` de los repositorios de Debian suele ir sensiblemente más atrasado en versión:

```
1 curl -fsSL https://download.docker.com/linux/debian/gpg | \  
2     sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
3 echo "deb [arch=$(dpkg --print-architecture) \  
4     signed-by=/etc/apt/keyrings/docker.gpg] \  
5     https://download.docker.com/linux/debian trixie stable" | \  
6     sudo tee /etc/apt/sources.list.d/docker.list  
7 sudo apt install docker-ce docker-ce-cli containerd.io \  
8
```



```
8  docker-buildx-plugin docker-compose-plugin
```

5.9.3. Adaptación de la configuración

Se construyeron las 11 imágenes (`make all`) y se adaptó la configuración por defecto del perfil *basic* al PLMN de pruebas del proyecto (901/70, TAC 7, Sección 5.4.3), modificando `configs/basic/amf.yaml` y `configs/basic/nrf.yaml`, y se añadió a AMF y SMF el siguiente bloque, necesario para exponer `/metrics`, `/ue-info` y `/pdu-info`:

```
1  metrics:
2    addr: 0.0.0.0
3    port: 9090    # 9091 en el caso del SMF
```

Además, se publican ambos puertos (9090, 9091) hacia el host en el propio `docker-compose.yaml`, ya que no vienen expuestos por defecto en el perfil empleado.

Al tratarse de un despliegue en contenedores con red en modo *bridge*, el AMF deja de ser accesible en 127.0.0.5 como en la instalación nativa; el fichero de configuración del gNB debe apuntar en su lugar a la dirección IP real del equipo anfitrión, publicada a través de la variable `DOCKER_HOST_IP` del propio despliegue.

Es importante emplear la versión **v2.7.7** de Open5GS (fijada mediante la variable `OPEN5GS_VERSION` del propio despliegue) y no la versión por defecto del repositorio (v2.7.6): esta última carece todavía del infoAPI (`/ue-info`, `/pdu-info`), incorporado según confirma una respuesta oficial del propio equipo de desarrollo de Open5GS únicamente a partir de la versión 2.7.7 [20, 24].

Con esto, la Fase 5 queda cerrada: núcleo Open5GS desplegado en contenedores Docker, con el mismo PLMN validado en la instalación nativa, y las dos fuentes de datos estructuradas (`/metrics` y el infoAPI) verificadas y listas para ser consumidas por el panel de monitorización.



5.10. Desarrollo del *backend*

Siguiendo el diseño establecido en la Metodología (Sección 4.3), se implementó un primer incremento del *backend* con FastAPI, cubriendo la estructura del proyecto, la autenticación y los tres *endpoints* de monitorización de solo lectura (GET `/status/core`, `/status/gnb`, `/status/ues`), dejando los *endpoints* de gestión (configuración del gNB, arranque/parada de NFs, alta y baja de suscriptores) para un incremento posterior.

5.10.1. Estructura del proyecto

El *backend* se organiza como un proyecto Python independiente (`dashboard-backend/`), separado del *frontend*, cada uno con su propio ciclo de vida, dependencias y proceso de despliegue, de forma que, por ejemplo, un cambio en el *frontend* no exija reiniciar el proceso del *backend*, y viceversa. Dentro de él, el código se organiza siguiendo la propia separación de capas de FastAPI: modelos de datos (`models/`, esquemas Pydantic empleados tanto para validar peticiones como para documentar automáticamente la API, Sección 5.10.2), *routers* (`routers/`, un fichero por área funcional: autenticación, monitorización, gNB, *core*, suscriptores) y servicios (`services/`, la lógica real de cada operación, sin ninguna dependencia de FastAPI, lo que permite probarla de forma aislada mediante `pytest`).



```
dashboard-backend/
├── app/
│   ├── main.py    Punto de entrada, ciclo de vida de la app
│   ├── config.py  Configuración centralizada (pydantic-settings)
│   ├── auth.py    JWT, dependencia require_auth
│   ├── models/    Esquemas Pydantic (validación de datos)
│   ├── routers/   Endpoints agrupados por área
│   └── services/  Lógica de negocio, sin FastAPI de por medio
├── scripts/
│   ├── gnb_ctl.sh  Con privilegio mínimo (Anexo B)
│   ├── docker_ctl.sh  Con privilegio mínimo (Anexo C)
│   └── deployment.conf.example
├── tests/    Pruebas automatizadas (pytest)
├── data/     Configuraciones guardadas del gNB
└── .env.example
```

5.10.2. Documentación automática de la API

Una ventaja directa de emplear FastAPI junto con modelos Pydantic para validar las peticiones y respuestas de cada *endpoint* es que dichos modelos se reutilizan automáticamente para generar documentación interactiva de la API completa (especificación OpenAPI), accesible en `/docs` sin necesidad de mantenerla por separado ni de que quede desactualizada respecto al propio código, a diferencia de una documentación de API escrita a mano, con el riesgo real de divergir del comportamiento efectivo del *backend* según este evoluciona.



5.10.3. Autenticación y degradación controlada

Se implementó la dependencia `require_auth` descrita en la Metodología, que valida un token JWT salvo que `AUTH_ENABLED=false`, en cuyo caso se omite la comprobación y se emite el aviso correspondiente en el registro de arranque del servicio:

```
1 async def require_auth(authorization: str = Header(default="")) -> str:
2     if not settings.auth_enabled:
3         return "anonimo"
4     token = authorization.removeprefix("Bearer ").strip()
5     return decode_token(token) # lanza HTTPException 401 si no es válido
```

De forma independiente a la autenticación, se decidió que los *endpoints* de monitorización **degraden de forma controlada** cuando el núcleo no esté disponible, devolviendo un estado HTTP 200 con los campos correspondientes marcados como no alcanzables, en lugar de propagar una excepción, ya que un panel de monitorización cuyo propio *backend* deja de responder precisamente cuando la red que vigila tiene problemas resultaría contraproducente:

```
1 if nrf_registry is None:
2     # El propio NRF no respondió, no se afirma "caída", sino
3     # "no verificable en este instante".
4     monitored, reachable = False, None
```

5.10.4. Verificación automatizada con datos reales

Se desarrolló una batería de pruebas automatizadas (`pytest`) que no dependen de tener el núcleo Open5GS en ejecución, verificando explícitamente que el sistema degrada según lo descrito en el párrafo anterior. Para el análisis de las dos fuentes de datos (formato Prometheus e infoAPI), las pruebas emplean fragmentos capturados realmente durante las pruebas manuales de las Secciones 5.9 y siguientes, en lugar de datos de ejemplo inventados, de forma que una prueba superada constituya una garantía más sólida de



que el analizador funciona frente al formato real devuelto por Open5GS, y no únicamente frente a una estructura idealizada.

5.10.5. *Scripts* de control con privilegio mínimo

Para el segundo incremento del *backend* (*endpoints* de gestión: configuración del gNB, arranque/parada de NFs, alta y baja de suscriptores), se implementó el mecanismo de privilegio mínimo justificado en la Metodología (Sección 4.3.4): el *backend* nunca ejecuta directamente ninguna acción sobre el gNB o sobre Docker, sino que delega en dos *scripts* de *shell* acotados, cada uno con su propia regla *sudoers* específica.

El primero, `gnb_ctl.sh` (código completo en el Anexo B), gestiona el ciclo de vida del proceso del gNB mediante su PID, guardado en un fichero temporal, y redirige su salida al registro consultado por el propio panel (Sección 5.17.2). La comprobación de estado, por ejemplo, se reduce a verificar si el proceso con ese PID sigue vivo:

```
1 status_gnb() {
2     if [ -f "$PID_FILE" ] && kill -0 "$(cat "$PID_FILE")" 2>/dev/null; then
3         echo "activo (PID $(cat "$PID_FILE"))"
4     else
5         echo "parado"
6     fi
7 }
```

Nótese que las rutas específicas de cada máquina (la del binario del gNB, las de una posible UHD compilada a mano) no están escritas dentro del propio *script*, sino que se leen de `deployment.conf`, el único fichero que es necesario editar al migrar el proyecto a otro equipo, tal como se valida en la Sección 5.14.

El segundo, `docker_ctl.sh` (código completo en el Anexo C), equivalente para las funciones de red del *core* en Docker, se describe con mayor detalle en la Sección 5.12.1, donde se justifica su necesidad frente a otras alternativas consideradas.



5.11. Monitorización de cada NF

Al extender la configuración de `metrics`: (Sección 5.9) a las diez funciones de red desplegadas, únicamente cuatro de ellas (AMF, SMF, PCF y UPF) resultaron alcanzables a través de su puerto de métricas; las seis restantes (NRF, AUSF, UDM, UDR, NSSF y BSF) no exponen ningún servidor HTTP en dicho puerto, sin que esto suponga ningún error de configuración por su parte. La documentación oficial de Open5GS confirma esta limitación de forma explícita:

“AMF, MME and SMF support exporting metrics so far, though other may hopefully follow soon” [25]

si bien se comprobó empíricamente que, en la versión 2.7.7 empleada en este trabajo, PCF y UPF también exponen esta funcionalidad, sin que la documentación oficial reflejase aún esta ampliación en el momento de redactar este texto.

Dado que reportar estas seis NFs como “inactivas” sería activamente engañoso, ya que se encuentran en funcionamiento correcto, solo que no son consultables por esta vía, se introdujo una distinción explícita en el modelo de datos del *backend* (campo `monitored`, independiente de `reachable`), de forma que el panel pueda mostrar estas seis funciones como “no monitorizable por esta vía” en lugar de como una caída del servicio. La cobertura completa de las diez funciones de red se alcanza finalmente combinando esta vía con el registro del NRF, según se describe en la Sección 5.16.

5.12. Verificación de los *endpoints*

Con el segundo incremento del *backend* implementado (*endpoints* de configuración del gNB, arranque/parada de NFs y gestión de suscriptores), se procedió a verificar cada uno de ellos contra el entorno real del proyecto, no únicamente mediante las pruebas automatizadas descritas en la Sección 5.10.



5.12.1. Control de Docker con privilegio mínimo

El proceso del *backend*, ejecutado con un usuario normal, no pertenece al grupo `docker` ni invoca `sudo`, por lo que no tiene acceso directo al socket de la API de Docker (`/var/run/docker.sock`). Se valoraron dos soluciones. La primera, añadir el usuario del *backend* al grupo `docker`, se descartó explícitamente: pertenecer a dicho grupo equivale en la práctica a privilegios de superusuario sobre todo el sistema, ya que un contenedor con montajes de volúmenes arbitrarios (por ejemplo, `docker run -v /:/host --privileged ...`) permite acceder al sistema de ficheros completo del host sin restricción alguna. Se optó en su lugar por extender el mismo mecanismo de privilegio mínimo ya empleado para el gNB (Sección 4.3 de la Metodología): un segundo script acotado, `docker_ctl.sh`, con las rutas del despliegue fijadas dentro del propio script y una lista blanca interna de funciones de red gestionables (como segunda barrera de validación, independiente de la que ya realiza el *backend* en Python), sobre el cual se concede una regla `sudoers` específica:

```
1 VALID_SERVICES="amf smf pcf upf nrf ausf udm udr nssf bsf"
2 # ...
3 valid=false
4 for s in $VALID_SERVICES; do
5     if [ "$s" = "$nf" ]; then
6         valid=true
7         break
8     fi
9 done
10 if [ "$valid" = false ]; then
11     echo "ERROR: '$nf' no es una función de red gestionada por este script"
12     exit 1
13 fi
```

Esta doble validación (en Python y en el propio *script*) implica que, aunque un fallo en el *backend* permitiera invocar el *script* con un nombre de función de red arbitrario, este seguiría rechazando cualquier valor fuera de la lista blanca antes de ejecutar ninguna



acción sobre Docker.

5.12.2. Verificación del control de NFs y de suscriptores

Con `docker_ctl.sh` desplegado, se verificó el ciclo completo de parada y arranque de una NF desde el propio *endpoint* (deteniendo y volviendo a arrancar el AMF, confirmando en ambos casos el cambio de estado mediante `docker compose ps`), así como el CRUD de suscriptores (alta, listado, e intento de alta duplicada devolviendo el código HTTP 409 esperado), verificando en los tres casos que el resultado observado en la infraestructura real coincidía con el reportado por el *endpoint*.

Al extender esta verificación al reinicio real del gNB con `PUT /gnb/config` contra el AMF ya desplegado en Docker (combinación no probada hasta este punto), la conexión N2 falló de forma persistente. El diagnóstico completo de esta incidencia, una de las más profundas de todo el trabajo, con dos causas independientes encadenadas, se documenta en el Anexo D.7. La configuración final que la resuelve fija el campo `bind_addr` del gNB a la dirección del equipo anfitrión dentro de la propia red de Docker (10.33.33.1, la puerta de enlace del *bridge*) en lugar de 127.0.0.1, y deshabilita el proxy de aplicación de Docker para el reenvío de puertos (`“userland-proxy”: false` en `/etc/docker/daemon.json`).

5.13. Estabilidad y portabilidad del despliegue

Las direcciones IP internas de las funciones de red, dentro de la red *bridge* de Docker, se asignan por defecto de forma dinámica según el orden de arranque, en lugar de permanecer fijas. Esto genera dos problemas distintos: por un lado, una fragilidad operativa (una IP que cambia entre reconstrucciones de la red obliga a reconfigurar manualmente cualquier componente externo que dependa de ella, como el propio gNB, y fue en efecto una de las dos causas concurrentes de la incidencia de red documentada en el Anexo D.6); por otro, un obstáculo para la portabilidad del despliegue a otra máquina, al no poder documentarse de antemano una dirección estable a la que apuntar.



Se resolvió fijando explícitamente la dirección de cada una de las diez funciones de red en el propio `docker-compose.yaml`, mediante el campo `ipv4_address` dentro de la definición de red de cada servicio, posible al tratarse de una subred ya definida de forma estática (10.33.33.0/24) en la configuración de red del despliegue:

```
1 networks:
2   open5gs:
3     ipv4_address: 10.33.33.6
4     aliases:
5       - amf.open5gs.org
```

Tras aplicar este cambio a las diez funciones de red y reconstruir el despliegue, se verificó que la dirección asignada a cada una coincidía exactamente con el valor fijado, y que las diez permanecían estables desde el primer arranque. Si en algún momento la resolución de nombres interna de Docker dejara de funcionar correctamente (síntoma de la incidencia documentada en el Anexo D.6), la solución consiste en reconstruir por completo la red del despliegue:

```
1 docker compose -f compose-files/basic/docker-compose.yaml --env-file=.env
   ↪ down
2 docker network prune -f
3 docker compose -f compose-files/basic/docker-compose.yaml --env-file=.env up
   ↪ -d
```

Como medida preventiva frente a la causa concurrente de dicha incidencia, se recomienda además desactivar el arranque automático del servicio nativo de MongoDB (`sudo systemctl disable mongod`) mientras se trabaje principalmente con el despliegue en contenedores, evitando el conflicto de puerto entre ambas instalaciones (nativa y en Docker) de MongoDB.

5.14. Portabilidad del *backend* y *scripts* de control

Como consecuencia directa de las incidencias de red anteriores, y anticipando la necesidad de ejecutar el sistema completo en el laboratorio del tutor del trabajo o durante la defensa del mismo, se revisó la portabilidad de todo el sistema de gestión a una máquina distinta de aquella en la que se ha desarrollado. Se identificaron dos categorías de valores dependientes de la máquina concreta: por un lado, los ya centralizados en el fichero `.env` del propio *backend* (Sección 4.3); por otro, rutas de sistema (al binario del gNB, a la instalación de UHD compilada a mano, al despliegue de Docker) que, hasta este punto, se encontraban escritas directamente dentro del código de los *scripts* de control `gnb_ctl.sh` y `docker_ctl.sh`, en contradicción con el objetivo de portabilidad.

Se extrajeron dichas rutas a un segundo fichero de configuración, `deployment.conf`, ubicado junto a los propios *scripts* (fuera del árbol del proyecto versionado, igual que estos, por las mismas razones de seguridad expuestas en la Metodología) y cargado por ambos mediante `source` al inicio de su ejecución. De este modo, mover el sistema completo a otra máquina se reduce a: copiar el proyecto, generar `.env` y `deployment.conf` a partir de sus plantillas correspondientes con los valores de la nueva máquina, y repetir la configuración de `sudoers` descrita en la Metodología, sin necesidad de modificar ningún fichero de código. Este mecanismo se validó de forma práctica migrando el proyecto completo a un segundo equipo (un portátil, frente al sobremesa empleado hasta ese punto), confirmando su funcionamiento sin más cambios que los dos ficheros de configuración mencionados.

Queda como limitación conocida, y no resuelta por este mecanismo, la variable que fija la dirección de red local del equipo anfitrión (`DOCKER_HOST_IP`) empleada por el despliegue Docker para que un equipo de usuario físico, conectado por radio, pueda alcanzar el plano de usuario (UPF), al ser esta, por su propia naturaleza, una característica de la red local de cada máquina concreta y no del propio proyecto.

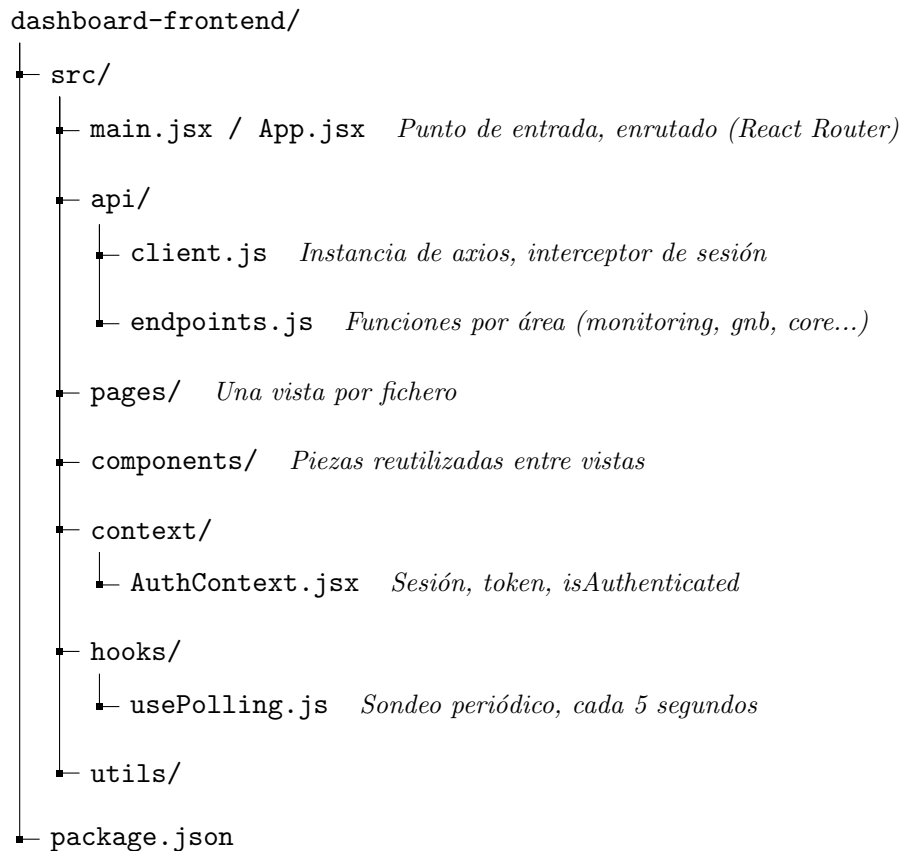


5.15. Desarrollo del *frontend*

Con el *backend* verificado de extremo a extremo, se desarrolló el *frontend* del panel con React, Vite y Tailwind CSS, siguiendo el diseño acordado en la Metodología.

5.15.1. Estructura del proyecto

El *frontend* se organiza igualmente como un proyecto independiente (`dashboard-frontend/`), por el mismo motivo de independencia de ciclo de vida ya expuesto para el *backend* (Sección 5.10.1): en particular, permite servir la aplicación ya compilada (`dist/`) sin depender de que el proceso de desarrollo (`npm run dev`) esté activo, relevante para la estabilidad del sistema durante su uso prolongado. La navegación entre las distintas vistas del panel se gestiona con **React Router**, y la comunicación con el *backend* mediante **axios**, centralizada en un único cliente HTTP (`api/client.js`) que añade el token de autenticación a cada petición y gestiona de forma consistente los errores de sesión expirada.



5.15.2. Política de mismo origen del navegador (CORS)

Al ejecutarse *frontend* y *backend* en puertos distintos del mismo equipo (5173 y 8000 respectivamente), constituyen orígenes distintos a efectos de la política de **CORS** (*Cross-Origin Resource Sharing*) que aplican los navegadores: sin autorización explícita, el navegador bloquea la lectura de la respuesta por parte del código JavaScript, aunque el servidor haya procesado la petición correctamente. Se resuelve añadiendo el *middleware* `CORSMiddleware` de FastAPI, autorizando explícitamente el origen del *frontend*:

```

1 app.add_middleware(
2     CORSMiddleware,
3     allow_origins=settings.cors_allowed_origins,
4     allow_credentials=True,
5     allow_methods=["*"],
6     allow_headers=["*"],
  
```



7

)

Se verificó mediante una petición de prueba con la cabecera `Origin` propia del navegador, confirmando que la respuesta incluye la cabecera `Access-Control-Allow-Origin` esperada.

5.15.3. Recuento de gNBs mediante la métrica nativa del AMF

El número de gNBs registrados se obtiene directamente de la métrica `gnb` (tipo *gauge*) que el propio AMF expone en su `/metrics`, en lugar de aproximararlo contando identificadores distintos dentro del infoAPI de equipos de usuario (`/ue-info`), una vía alternativa que se descartó por depender de que exista al menos un UE conectado para poder inferir nada, mostrando sistemáticamente cero gNBs mientras no sea el caso, con independencia de si el gNB está realmente registrado.

5.16. Monitorización de NFs mediante NRF

La limitación documentada en la Sección 5.11 (solo AMF, SMF, PCF y UPF exponen `/metrics`) deja sin verificar el estado real de seis de las diez funciones de red desplegadas por esta única vía. Se investigó una alternativa: dado que toda función de red se registra contra el NRF al arrancar, que es, de hecho, la función que da nombre al propio NRF, *Network Repository Function*, dicho registro puede servir como fuente de estado independiente de si cada NF implementa `/metrics` o no.

5.16.1. El registro del NRF como fuente de estado

Se confirmó, contra el propio código fuente de Open5GS y un hilo de su repositorio en el que se documenta su uso [26], que el NRF expone el servicio estándar 3GPP `Nnrf-NFManagement` en su interfaz SBI, con un *endpoint* que lista todas las instancias de NF registradas:



```
1 GET /nnrf-nfm/v1/nf-instances
```

Este *endpoint* devuelve una lista de enlaces a cada instancia, cuyo detalle (incluyendo los campos `nfType` y `nfStatus`) se obtiene con una segunda consulta por instancia. Se verificó el mecanismo completo contra el despliegue real de este trabajo, confirmando además que la interfaz SBI de Open5GS emplea HTTP/2 en texto plano sin negociación previa (*prior knowledge*), a diferencia del resto de *endpoints* consultados hasta este punto (HTTP/1.1 estándar), particularidad que obligó a emplear una configuración de cliente HTTP distinta (`httpx` con `http1=False`, `http2=True`) específicamente para esta fuente de datos.

A diferencia de `/metrics`, disponible únicamente en un subconjunto de NFs por decisión de implementación de Open5GS, el registro del NRF es un mecanismo estándar 3GPP al que toda función de red del plano de control se adhiere por diseño de la propia arquitectura de servicios (SBA), por lo que cubre de forma natural las diez funciones de red desplegadas, sin necesidad de ampliar los privilegios del *backend* sobre Docker, alternativa que se descartó explícitamente por motivos de coherencia con el principio de privilegio mínimo ya establecido en la Metodología.

5.16.2. El caso particular del UPF

Al aplicar el mecanismo anterior, nueve de las diez funciones de red pasaron a reportar su estado correctamente; el UPF, sin embargo, no aparece nunca en dicho registro, pese a estar activo. No se trata de un fallo, sino de una característica arquitectónica real del UPF, confirmada por una respuesta directa de un colaborador del propio proyecto Open5GS: “*el UPF no tiene interfaz SBI, por lo que no puede registrarse en el NRF*” [27]. El UPF, como elemento exclusivo de plano de usuario, se comunica con el SMF mediante PFCP (*Packet Forwarding Control Protocol*, sobre UDP), no mediante la interfaz de servicios HTTP/SBI que emplea el resto de funciones de plano de control.

Se trata este caso de forma diferenciada: el estado del UPF se determina a partir



de si responde a su propio `/metrics` (como antes de introducir el mecanismo del NRF), mientras que las nueve funciones de red restantes emplean el registro del NRF. El resultado final combina ambas fuentes según corresponda a cada NF, verificado con las diez funciones de red mostrando su estado real en el panel.

5.17. Ampliaciones de gestión del gNB

A partir del uso real del panel ya con las cuatro vistas de gestión operativas, surgieron dos necesidades no contempladas en el diseño original de la Sección 4.3: poder arrancar el gNB directamente desde el panel sin pasar por el botón de "guardar y reiniciar" (pensado para aplicar cambios de configuración, no como acción de arranque en sí), y poder confirmar qué tarjeta SDR concreta detectó realmente el sistema, distinguiéndolo de lo que simplemente figura en el fichero de configuración.

5.17.1. Arranque independiente de la modificación de configuración

Se añadió el *endpoint* `POST /gnb/start`, que invoca la acción `start` de `gnb_ctl.sh` sin reescribir el fichero `YAML`, a diferencia de `PUT /gnb/config`, que siempre escribe los valores del formulario antes de reiniciar. El *frontend* muestra este botón únicamente cuando el gNB no está en marcha, evitando una llamada sin efecto (aunque el propio `gnb_ctl.sh` ya la gestiona sin error, ver Sección 4.3).

5.17.2. Confirmación del hardware SDR realmente detectado

Dado que el panel no selecciona ni cambia de tarjeta SDR por sí mismo, sino que controla un único binario del gNB, el que apunte `GNB_BIN` en `deployment.conf` (Sección 5.14), se decidió mostrar de forma explícita qué dispositivo detectó realmente UHD en el último arranque, en lugar de limitarse a mostrar la configuración solicitada, que podría no coincidir con lo detectado si algo fallara. Se aprovechó para ello una línea de registro ya observada repetidamente a lo largo de este trabajo (Sección 5.8):



```
1 [INFO] [B200] Detected Device: B210
```

Se implementó `gnb_control.detected_hardware()`, que localiza la ocurrencia más reciente de este patrón en el registro de arranque capturado por `gnb_ctl.sh`, y se expone mediante `GET /gnb/hardware` junto con el estado de ejecución actual (`gnb_ctl.sh status`). Se documenta explícitamente, tanto en el código como en esta memoria, que esta línea es específica del controlador UHD empleado por el binario gestionado actualmente por el panel (la rama B210, Sección 5.8) y no un mecanismo genérico válido para cualquier SDR, en particular, no se ha verificado su equivalente para la rama bladeRF, que emplea una cadena de compatibilidad SoapySDR distinta.

5.18. Ampliación del modo sin autenticación

Al emplear por primera vez el panel con `AUTH_ENABLED=false` (uso individual, ver Metodología), se detectó una limitación de experiencia no contemplada en el diseño original: aunque ningún *endpoint* de gestión exigiera realmente autenticación en ese modo, el *frontend* seguía mostrando la pantalla de inicio de sesión en todos los casos, aceptando cualquier credencial introducida, un comportamiento funcionalmente correcto pero confuso, que no comunicaba al usuario que la protección estaba desactivada.

Se añadió un *endpoint* público adicional, deliberadamente sin exigir token (a diferencia de cualquier otro *endpoint* de gestión), que informa únicamente del valor actual de `AUTH_ENABLED`:

```
1 @router.get("/status")
2 async def auth_status():
3     return {"auth_enabled": settings.auth_enabled}
```

El *frontend* lo consulta al arrancar la aplicación: si la autenticación está desactivada y no existe ya una sesión guardada, obtiene un token en silencio (el propio *backend* lo emite sin comprobar credenciales en ese modo, ver Sección 4.3) y omite por completo la pantalla de



inicio de sesión. Se mantiene deliberadamente la ausencia de cualquier mecanismo para desactivar la autenticación desde la propia interfaz web, coherente con el principio de *fail-safe defaults* ya justificado en la Metodología: la decisión de relajar la protección debe tomarse de forma explícita a nivel de servidor (editando `.env` y reiniciando el proceso), no mediante un control accesible desde el navegador.

5.19. Diagrama de topología de red en vivo

Como ampliación del panel, no contemplada en el diseño original pero solicitada durante el uso del sistema ya construido, se incorporó una vista adicional de topología, sin sustituir ninguna de las vistas basadas en listas ya existentes, que representa gráficamente las interfaces reales de la arquitectura 5G SA entre los elementos monitorizados: **Uu** (interfaz radio entre el equipo de usuario y el gNB), **N2** (señalización NGAP entre el gNB y el AMF), **N3** (plano de usuario GTP-U entre el gNB y el UPF), **N4** (PFCP entre el SMF y el UPF, representada con trazo discontinuo por emplear un protocolo distinto al resto) y **N6** (salida del UPF hacia la red de datos externa).

A diferencia de un diagrama estático, cada nodo y enlace del diagrama toma su color del mismo conjunto de datos en vivo empleado por el resto del panel (sondeado cada 5 segundos): las nueve funciones de red con interfaz SBI se agrupan visualmente dentro de un mismo clúster, reflejando la propia arquitectura de servicios de 3GPP, mientras que el UPF se representa de forma independiente, coherente con la Sección 5.16.2, ya que su color depende de `/metrics`, no de su pertenencia al clúster SBI, y así se indica explícitamente en la propia vista para evitar una lectura errónea por parte de quien la consulte sin conocer dicho matiz.

5.20. Revisión y nuevas funcionalidades

Con las cinco vistas del panel ya operativas, se realizó una revisión deliberada frente a dos referencias: el propio trabajo predecesor [1], para identificar funcionalidades presentes en él y ausentes en este trabajo, y OAIBOX [28], la plataforma comercial empleada como



referencia de funcionalidad esperada desde la Metodología del trabajo. De esta revisión surgieron las cuatro ampliaciones descritas en las secciones siguientes.

5.20.1. Comparativa con el trabajo predecesor

El trabajo predecesor incorporaba dos capacidades ausentes en este trabajo hasta este punto: una vista de captura de tráfico en vivo, basada en `tcpdump` sobre la interfaz `ogstun` con cada paquete analizado y enviado al cliente mediante `WebSocket`; y una segunda vista de estadísticas históricas de tráfico, construida sobre un conjunto de herramientas de terceros (`ntopng` para la obtención de datos, `InfluxDB` como base de datos de series temporales, y `Grafana` para su visualización, embebido como panel dentro de la propia interfaz web). De ambas, se decidió abordar la segunda, el histórico de tráfico, por las razones expuestas en la Sección 5.21.1; la primera (captura e inspección de paquetes en vivo) queda fuera del alcance abordado en esta revisión.

5.20.2. Funcionalidades inspiradas en OAIBOX

De la revisión del manual de OAIBOX [28] se identificaron, entre otras, tres capacidades del panel de referencia ausentes en este trabajo: un botón de “*Logs*” por cada módulo gestionable, que permite consultar su registro de actividad sin salir del propio panel; una acción de “*Restart*” independiente de las de arranque y parada; y un resumen agregado de tráfico en tiempo real. Las dos primeras se abordan en la Sección 5.22; la tercera queda cubierta, en la medida permitida por los datos disponibles en este despliegue, por el histórico de tráfico N3 de la Sección 5.21.



5.21. Histórico de tráfico N3

5.21.1. Decisión de arquitectura: réplica del *stack* del predecesor frente a solución propia

Antes de implementar el histórico de tráfico, se valoraron dos vías. La primera, replicar el *stack* completo del trabajo predecesor (`ntopng` + `InfluxDB` + `Grafana`), requeriría capturar tráfico en bruto con privilegios de superusuario sobre una interfaz de red y desplegar tres servicios adicionales. La segunda, construir una solución propia dentro de la arquitectura ya establecida en este trabajo, dependía de que Open5GS expusiera contadores de tráfico reales por alguna vía ya empleada (`/metrics`), sin necesidad de captura de paquetes.

Se confirmó, contra un caso reportado en el propio repositorio de Open5GS [29] y verificado empíricamente contra el despliegue de este trabajo, que el UPF expone contadores acumulativos de paquetes GTP en la interfaz N3 (`fivegs_ep_n3_gtp_in/outdatapktn3upf`), siempre presentes en su `/metrics` (a diferencia de las métricas de volumen en bytes por nivel de QoS, que solo generan muestras una vez existe un flujo QoS activo). Esto permitió optar por la segunda vía, evitando tanto el privilegio elevado sobre captura de paquetes como la incorporación de tres piezas de infraestructura de terceros, coherente con el principio de privilegio mínimo ya aplicado al resto del panel, y con una arquitectura íntegramente propia en lugar de una integración de herramientas ajenas.

5.21.2. Muestreo en segundo plano y cálculo de tasas

Se implementó un muestreador (`throughput_sampler.py`) que se ejecuta como tarea de fondo de la propia aplicación, arrancada mediante el gestor de ciclo de vida de FastAPI, en lugar de activarse solo cuando el *frontend* consulta el panel, de forma que el histórico continúe acumulándose aunque nadie tenga el panel abierto en un momento dado, como cabría esperar de un sistema de monitorización real. Dado que los contadores del UPF son de tipo *counter* de Prometheus (acumulativos, nunca decrecientes salvo reinicio del



proceso), la tasa de tráfico de cada intervalo se calcula como la diferencia entre dos lecturas consecutivas dividida por el tiempo transcurrido, aislada en la siguiente función pura (`compute_rate`), verificada mediante pruebas unitarias independientes de cualquier E/S de red:

```
1 def compute_rate(prev_pkts, curr_pkts, elapsed_s):
2     if elapsed_s <= 0:
3         return 0.0
4     delta = curr_pkts - prev_pkts
5     if delta < 0:
6         return 0.0 # el contador se reinició, sin dato fiable, no negativo
7     return delta / elapsed_s
```

Esta función cubre también el caso de un contador que retrocede (reinicio del UPF), tratado como ausencia de dato fiable en ese intervalo en lugar de una tasa negativa sin sentido.

5.22. Gestión ampliada de las funciones de red

Siguiendo el patrón identificado en OAIBOX (Sección 5.20), se extendió `docker_ctl.sh` con dos acciones nuevas, manteniendo el mismo modelo de seguridad ya establecido (script acotado por una única regla `sudoers`, con lista blanca de funciones de red gestionables validada de forma independiente dentro del propio script): `restart`, que invoca `docker compose restart` como operación atómica en lugar de encadenar una parada y un arranque por separado desde el *backend*; y `logs`, que devuelve las últimas líneas del registro del contenedor mediante `docker compose logs --tail`, con el número de líneas acotado entre 1 y 1000, validado tanto en el propio *script* como en el *backend*, como segunda barrera independiente, siguiendo el mismo criterio ya aplicado a la lista blanca de NFs.

En el *frontend*, cada función de red de la vista de gestión del *core* incorpora ahora un botón de registro que despliega un panel con las últimas 100 líneas en fuente



monoespaciada, actualizable bajo demanda en lugar de mediante sondeo automático, evitando invocar `docker compose logs` de forma innecesaria cuando el panel de registro no está en uso.

5.23. Configuración del gNB mediante *presets*

La revisión del formulario de configuración del gNB reveló una omisión con implicaciones de seguridad funcional: el campo `srate` (tasa de muestreo), pese a ser interdependiente con la banda, el ancho de banda de canal y el *subcarrier spacing*, interdependencia que ya había causado una incidencia real durante este mismo trabajo (Sección 5.17.2, error de formato PRACH por una combinación de `srate` incompatible), no formaba parte del modelo `GnbConfig` ni se escribía al aplicar cambios desde el panel. Esto significaba que, desde su primera versión, el formulario permitía generar, sin ningún aviso, exactamente el tipo de configuración inválida ya sufrida, si alguien cambiaba el ancho de banda de canal sin ajustar manualmente `srate` en el propio fichero. Se corrigió incorporando el campo a la lectura y escritura de configuración.

5.23.1. Límites reales de ganancia de la USRP B210

Se consultó la documentación oficial de Ettus [30] para fijar los límites válidos de las ganancias de transmisión y recepción, en lugar de dejarlas como campos numéricos sin restricción: la etapa de transmisión de la B210 dispone de 89,8 dB de ganancia disponible, y la de recepción de 76 dB, ambas partiendo de 0, nunca negativas en esta tarjeta, a diferencia del rango observado en la bladeRF (Sección 5.5). Se incorporaron estos límites como restricciones del propio modelo de datos (`Field(ge=0, le=89.8)` y equivalente para recepción), de forma que una petición fuera de rango se rechace por validación antes de llegar a tocar el fichero de configuración.



5.23.2. Diseño: *presets* validados frente a campos libres

Para el resto de parámetros de radio (banda, ARFCN, ancho de banda de canal, *subcarrier spacing* y *srate*), se descartó mantenerlos como campos numéricos independientes, dado que codificar de forma exhaustiva las reglas de validez 3GPP para toda combinación posible excede el alcance de este trabajo, y la alternativa de dejarlos completamente libres ya había demostrado su riesgo real. Se optó por un nuevo *endpoint*, GET `/gnb/presets`, que expone únicamente combinaciones verificadas de forma efectiva a lo largo de este trabajo, no una lista exhaustiva de lo teóricamente válido, sino las dos configuraciones que se han empleado y confirmado funcionando con la USRP B210: la de 20 MHz con la que se logró la primera conexión N2 estable, y la de 10 MHz empleada durante la investigación de los *timeouts* de transmisión (Anexo D.5).

El *frontend* presenta estos *presets* como opción por defecto, detectando automáticamente si la configuración actual del gNB coincide con alguno de ellos para preseleccionarlo, y ofrece un modo “*Personalizado*” explícitamente marcado como avanzado, con un aviso que referencia la propia incidencia de este trabajo que motivó la restricción, en lugar de una advertencia genérica sin contexto.

5.24. Resolución de la conexión del UE

La incidencia documentada en la Sección 5.7, abierta desde las primeras fases de este trabajo, quedó resuelta al retomar las pruebas con un segundo equipo de usuario y dos cambios aplicados de forma conjunta: el empleo del PLMN 001/01 en lugar del PLMN de pruebas 901/70 empleado hasta este punto, y la reprogramación de la tarjeta SIM con un IMSI y credenciales (K/OPc) coherentes con dicho PLMN, dados de alta como suscriptor mediante el propio panel de gestión desarrollado en este trabajo (Sección 5.12) en lugar de mediante el WebUI oficial de Open5GS.

Cabe señalar, por rigor, que ambos cambios se aplicaron a la vez, por lo que no puede aislarse con total certeza cuál de los dos fue determinante, ya que es posible que el problema real residiera en la programación de la tarjeta SIM en algún intento



anterior (Sección 5.7 ya apuntaba a esta posibilidad como principal sospechosa) y que el cambio de PLMN fuera incidental. No obstante, resulta relevante señalar que 001/01 es el PLMN reservado explícitamente por la Recomendación UIT-T E.212 para pruebas y validación [31], a diferencia de 901/70: el MCC 901 está asignado a operadores de redes internacionales/por satélite reales, por lo que no puede descartarse que el equipo de usuario aplicara algún tratamiento distinto a un MCC perteneciente a ese rango frente al MCC de pruebas reservado.

5.24.1. Evidencia del registro completo

El registro del AMF confirma, por primera vez en este trabajo, el procedimiento NAS de registro 5G completo, no solo la conexión N2 del propio gNB:

```
1 [gmm] INFO: Registration request (../src/amf/gmm-sm.c:1670)
2 [gmm] INFO: [imsi-...] Registration complete (../src/amf/gmm-sm.c:3146)
```

Esto queda confirmado además por el propio terminal, que mostró cobertura, el nombre de red **Open5GS** y conexión de datos activa bajo tecnología 5G. Los mensajes **Ue requested DNN ‘ims’ Not Supported OR Not Subscribed in the Slice**, presentes de forma recurrente en el registro tras el éxito del procedimiento, corresponden a un intento del propio terminal, en segundo plano, de establecer adicionalmente una sesión para servicios de voz sobre datos (VoLTE/VoNR), un DNN no dado de alta ni necesario para el alcance de este trabajo; el rechazo por parte del AMF es el comportamiento esperado y no constituye ninguna incidencia.

5.24.2. Salida real a la red de datos

La sesión PDU establecida confirma el registro y la asignación de una dirección IP al equipo de usuario dentro de la red gestionada por Open5GS. Dicho tráfico llega, además, a redes externas reales: como se muestra en la Sección de Resultados, se verificó con éxito tanto la navegación web como la reproducción de vídeo y una prueba de velocidad desde



el propio equipo de usuario, reflejándose dicho tráfico en el histórico de subida N3 descrito en la Sección 5.21.

Esta salida a Internet no requirió ninguna configuración manual de traducción de direcciones (NAT) por parte de este trabajo. Al inspeccionar las reglas `iptables` del equipo anfitrión, se confirmó que Docker las genera de forma automática al crear la red `bridge` del despliegue (`br-ogs`): una primera regla `MASQUERADE`, ya conocida (Sección 5.21.1), traduce el tráfico de la subred de equipos de usuario (`10.45.0.0/16`) con la dirección del propio contenedor UPF al salir de este hacia la red `bridge`; una segunda regla, añadida por Docker sobre el propio equipo anfitrión, traduce a su vez el tráfico de dicha red `bridge` (`10.33.33.0/24`) con la dirección real del equipo al salir por la interfaz con acceso a Internet. La cadena `DOCKER-FORWARD` completa este comportamiento, aceptando explícitamente el tráfico entrante desde `br-ogs` hacia el resto de interfaces. El resultado es un encadenamiento de NAT de doble salto, resuelto íntegramente por el propio Docker sin intervención adicional.

5.25. Ampliaciones de gestión adicionales

Tras una sesión de revisión del estado del panel con el tutor del trabajo, se identificaron tres ampliaciones de la capa de gestión, descritas en esta sección: la posibilidad de guardar y recuperar configuraciones del gNB elegidas por el propio usuario, la consulta directa de los registros de actividad (tanto del gNB como de las funciones de red del *core*) sin salir del panel, y la capacidad de arrancar o detener el *core* completo con una única acción.

5.25.1. Configuraciones del gNB guardadas por el usuario

Además de los *presets* fijos y verificados descritos en la Sección 5.23, se incorporó la posibilidad de que el propio usuario guarde, con un nombre elegido por él, cualquier configuración personalizada que haya introducido en modo avanzado, incluidas las ganancias, a diferencia de los *presets* fijos, que no las fijan de antemano.

Se optó por un fichero JSON simple en disco, en lugar de MongoDB: son datos de



configuración propios del panel, no suscriptores de Open5GS, y mezclar ambos conceptos en la misma base de datos no tendría justificación conceptual, siendo además un fichero de texto más que suficiente para una lista de unas pocas configuraciones guardadas.

Ubicación de almacenamiento configurable desde el propio panel

Se planteó la posibilidad de que el usuario eligiera dónde guardar estas configuraciones, al estilo de un diálogo de “*Guardar como*” de una aplicación de escritorio. Esto no es directamente posible en una aplicación web: un navegador no puede abrir un diálogo nativo que navegue por el sistema de ficheros del **servidor**, ya que los navegadores lo impiden deliberadamente por seguridad, y de lo contrario cualquier página web podría curiosear o escribir en el disco de quien la visitara. En su lugar, se ofrece el equivalente practicable: un campo donde escribir la ruta absoluta deseada, validada por el propio *backend* (debe ser absoluta y su directorio contenedor debe ser escribible) antes de aceptarla, con migración automática de las configuraciones ya guardadas a la nueva ubicación si esta todavía no existe. Se incluye también detección de si la ruta indicada es en realidad una carpeta, en lugar de un fichero:

```
1 looked_like_a_folder = new_path.endswith("/") or new_path_obj.is_dir()
2 if looked_like_a_folder:
3     new_path_obj = new_path_obj / "gnb_custom_presets.json"
```

Se emplea en ese caso un nombre de fichero por defecto dentro de ella, evitando así generar un fichero mal nombrado sin ningún aviso, una confusión real detectada durante las pruebas de esta funcionalidad.

La elección de ruta se recuerda entre reinicios del *backend* mediante un pequeño fichero “puntero” interno, en lugar de reescribir el propio `.env` desde la aplicación en tiempo de ejecución, ya que una modificación de ese tipo sobre su propio fichero de configuración se consideró una práctica de peor calidad, al mezclar configuración de despliegue (gestionada por quien instala el sistema) con preferencias de uso (gestionadas por quien lo usa).



5.25.2. Consulta, búsqueda y descarga de registros de actividad

Se amplió la consulta de registros, tanto del gNB como de las funciones de red del *core* (esta última ya introducida en la Sección 5.22), en tres direcciones.

Inicio y final del registro

La consulta original solo permitía ver las últimas líneas del fichero. Se añadió la posibilidad de elegir entre las primeras líneas (*“inicio”*, útil para ver la información de arranque, detección de hardware, carga de la FPGA...) y las últimas (*“final”*, el comportamiento por defecto), evitando así que dicha información quedara fuera de la ventana visible cuando el proceso lleva ya un tiempo en marcha.

Búsqueda en el fichero completo

Ante la dificultad real de no saber de antemano en qué punto de un registro puede aparecer la información buscada, se incorporó una búsqueda de texto que recorre el fichero **completo** (no solo la ventana de líneas mostrada), devolviendo únicamente las líneas coincidentes. Para el gNB, al ser una simple lectura de fichero en Python, no presenta ningún riesgo adicional. Para las funciones de red del *core*, la búsqueda se resuelve dentro del propio `docker_ctl.sh` con la siguiente línea, que incorpora dos medidas de seguridad activamente verificadas durante el desarrollo (no solo argumentadas):

```
1 docker compose logs --no-color "$nf" | grep -F -i -- "$search_term"
```

El término de búsqueda (`$search_term`) se pasa como argumento separado del proceso, nunca concatenado dentro de una cadena de intérprete de comandos, de forma que los metacaracteres que pudiera contener no se interpreten como código; y se emplea `grep -F` (cadena literal, no expresión regular) junto con `--` antes del término, para evitar tanto ataques de expresión regular catastrófica como que un término que comience por guion se interprete como una opción del propio `grep`. Se comprobó expresamente, introduciendo un término de búsqueda con una sustitución de comandos de intérprete (`$(...)`), que dicho intento no llegaba a ejecutarse.



Descarga del registro completo

Mostrar un fichero de registro completo, sin límite, directamente en el navegador comparte el mismo riesgo ya identificado en la Sección 5.22 (sobrecarga de memoria del *frontend* con un bloque de texto potencialmente muy extenso). En su lugar, se añadió una descarga como fichero: para el gNB, servida directamente desde disco sin cargarla en memoria del *backend*; para las NFs del *core*, obtenida vía `docker compose logs` sin límite de líneas, expresamente pensada para guardarse y consultarse con herramientas del propio sistema operativo (editor de texto, `grep`, `less`), no para renderizarse en pantalla.

5.25.3. Arranque y parada del *core* completo

Hasta este punto, el panel permitía arrancar y detener cada función de red de forma individual, pero no el *core* en su conjunto, una limitación señalada durante la revisión con el tutor, ya que en el uso real es habitual querer levantar o detener el conjunto completo de una sola vez, sin recurrir a la terminal. Se añadieron dos acciones nuevas al mismo script acotado ya existente (`docker_ctl.sh`), equivalentes a `docker compose up -d` y `docker compose down` sobre las diez funciones de red, manteniendo el mismo modelo de seguridad ya establecido en el resto del trabajo (privilegio mínimo mediante una regla `sudoers` acotada a este script concreto).

Todas las llamadas bloqueantes a `subprocess.run` de los servicios de control del *backend* (incluidas estas dos acciones nuevas) se delegan en un hilo aparte mediante `asyncio.to_thread`, evitando que el bucle de eventos asíncrono del servidor quede bloqueado mientras dura una operación de varios segundos, necesario en particular para estas dos acciones, al afectar a las diez funciones de red a la vez.

Como medida adicional de robustez ante fallos de red genuinamente pasajeros, se incorporó en el *frontend* un reintento automático, en silencio, para estas acciones concretas: si la petición falla sin que el servidor llegue a responder en absoluto, se reintenta una vez antes de mostrar cualquier mensaje al usuario, justificado porque arrancar o detener el *core* son operaciones seguras de repetir (si ya se completaron correctamente la primera vez, un segundo intento simplemente confirma que no hay nada más que hacer). Se

comprobó explícitamente que este mecanismo no reintenta ante un error real devuelto por el propio servidor (por ejemplo, una autenticación inválida), reservándose exclusivamente para fallos a nivel de red.

Capítulo 6

Resultados

En este capítulo se ilustrarán las pruebas llevadas a cabo sobre el trabajo desarrollado, sirviendo así no sólo como muestra de los objetivos alcanzados, sino además como guía de uso.

6.1. Funcionamiento de la aplicación construida

En primer lugar, se navegará por la aplicación en su totalidad para validar que el funcionamiento es el esperado. Para poder hacer esto, el primer paso es arrancar el *backend* y el *frontend*.

Los comandos para levantar el *backend* son los siguientes:

```
1 source venv/bin/activate
2
3 uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Por otro lado, en cualquier otra terminal, se levanta el *frontend* de la siguiente forma:

```
1 serve -s dist -p 5173
```

Merece la pena indicar que cada grupo de comandos de arranque debe ejecutarse en la ruta concreta en la que se encuentren los ficheros correspondientes. En el caso del entorno



de desarrollo de este proyecto, los comandos para el arranque de forma completa serían los siguientes:

```
1 cd /home/jbg/Documents/TFG/Code/dashboard-backend
2
3 source venv/bin/activate
4
5 uvicorn app.main:app --host 0.0.0.0 --port 8000
```

```
1 cd /home/jbg/Documents/TFG/Code/dashboard-frontend
2
3 serve -s dist -p 5173
```

Una vez ejecutados todos los comandos, tendremos nuestra aplicación corriendo. El siguiente paso será acceder al panel mediante un navegador cualquiera. Basta con introducir la dirección `localhost:5173/`.

Tras acceder a esa dirección, lo primero que veremos será la pantalla de autenticación. Las credenciales son las siguientes:

- Usuario: admin
- Contraseña: admin

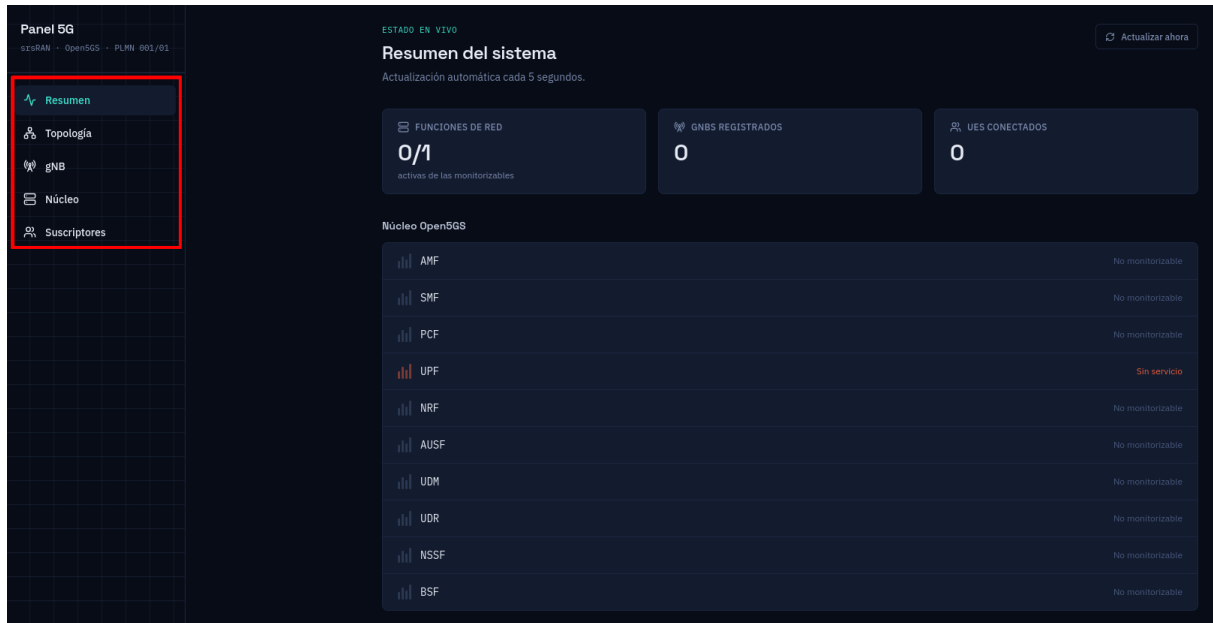
Figura 6.1: Formulario de *login*.
Fuente: Propia

Lo primero que veremos nada más entrar al *dashboard* es el resumen. Al no tener en funcionamiento ninguna parte de la arquitectura aparte de la propia aplicación, los contadores se encuentran a cero, el estado del core indica que no está corriendo, y no hay ningún UE conectado.

Por otro lado, en la parte izquierda del panel tenemos una lista con el resto de apartados con los que cuenta el panel de monitorización. Cada uno de los apartados hace las veces de botón y es lo que nos permite navegar por el resto de la aplicación. Tenemos los siguientes:

- **Resumen**, que muestra la pantalla que hemos mencionado.
- **Topología**, que muestra de forma gráfica los elementos de la topología y su estado actual.
- **gNB**, que muestra la información sobre el gNB así como las diferentes acciones que se pueden tomar.

- **Núcleo**, que muestra las diferentes funciones de red que conforman el *core* y cuenta con las herramientas para trabajar sobre estas *network functions*.
- **Suscriptores**, que muestra qué suscriptores están dados de alta mediante Open5GS y permite dar de alta a nuevos suscriptores, sin tener que salirse de la aplicación.

Figura 6.2: Panel resumen del *dashboard*.

Fuente: Propia

Además, todos los distintos paneles son refrescados cada 5 segundos, actualizando la información en caso de que haya cambios.

El siguiente panel que se muestra es Topología, que como se ha mencionado antes, muestra las partes que componen la arquitectura y su estado. Ahora mismo al no tener nada arrancado, el diagrama tiene este aspecto:

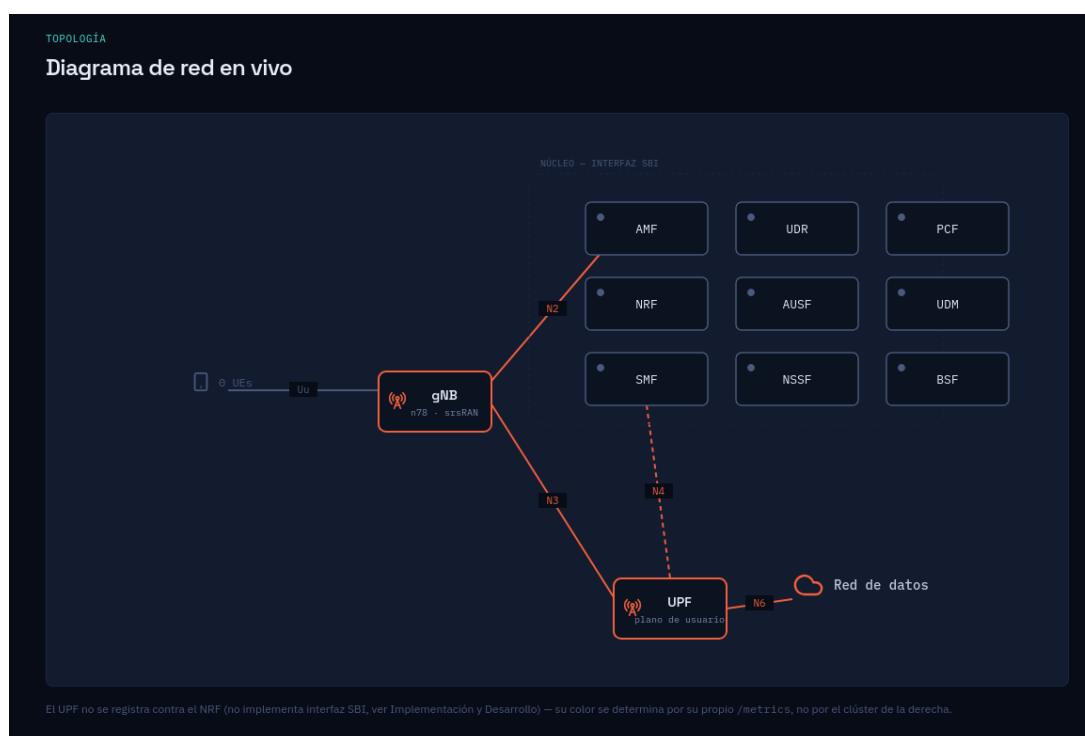


Figura 6.3: Panel Topología del *dashboard*, todo inactivo.

Fuente: Propia

El siguiente paso es levantar el *core*. Para ello, es necesario dirigirse al panel Núcleo.

En este panel podremos actuar sobre el núcleo en su totalidad, bien arrancando todas las NF al mismo tiempo o parándolas. En el caso de que se necesite actuar sobre una de las funciones de red del núcleo en concreto, también se cuenta con las acciones de arranque, reinicio y parada sobre cada una de ellas.

Además de esto, también se pueden recuperar y observar los registros de cada una de estas funciones.

Como indicador del estado de cada una de las NF tenemos el símbolo a la parte izquierda de cada una de ellas. Cuando se encuentren paradas, ese símbolo aparecerá de color gris (estado no verificable), a excepción de UPF, que como ya se ha mencionado en la memoria, no cuenta con SBI y por ello se comprueba de forma distinta al resto. Esto provoca que su símbolo aparezca de color rojo (se intenta verificar pero falla, en lugar de resultar no verificable), pero sabemos que esto se debe a que está parado. Como veremos más adelante, si están en funcionamiento el icono será verde y estará en movimiento.

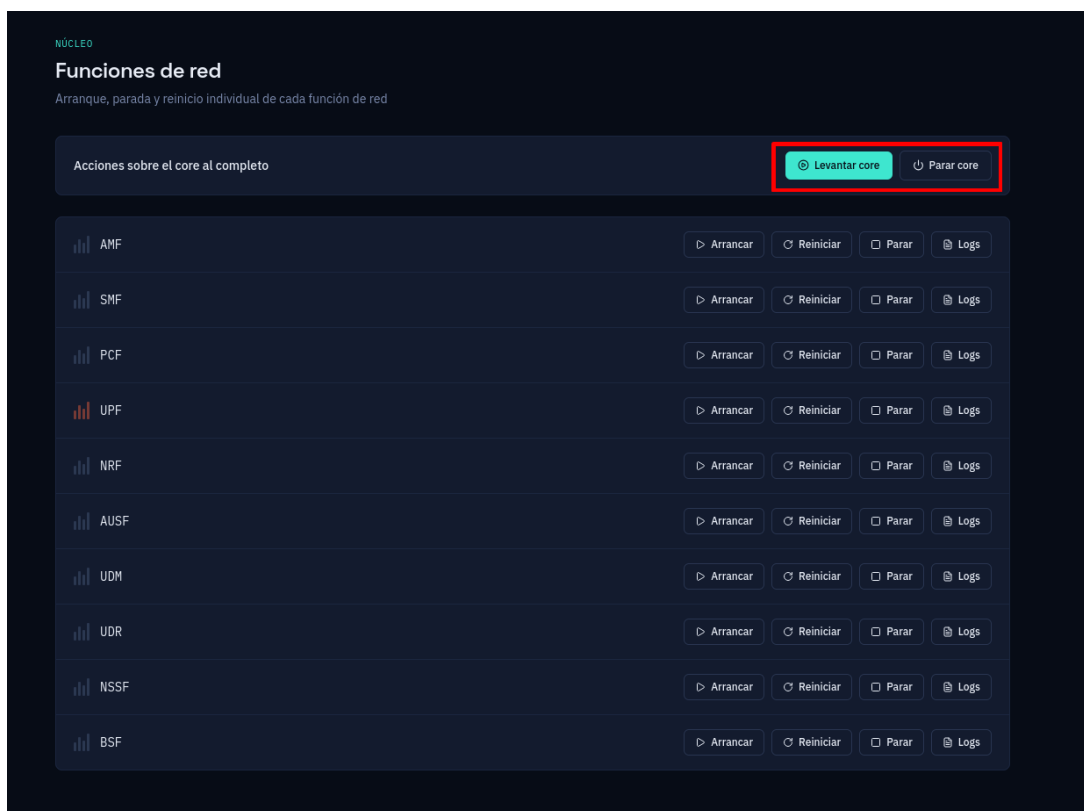


Figura 6.4: Panel Núcleo del *dashboard*, todo inactivo.
Fuente: Propia

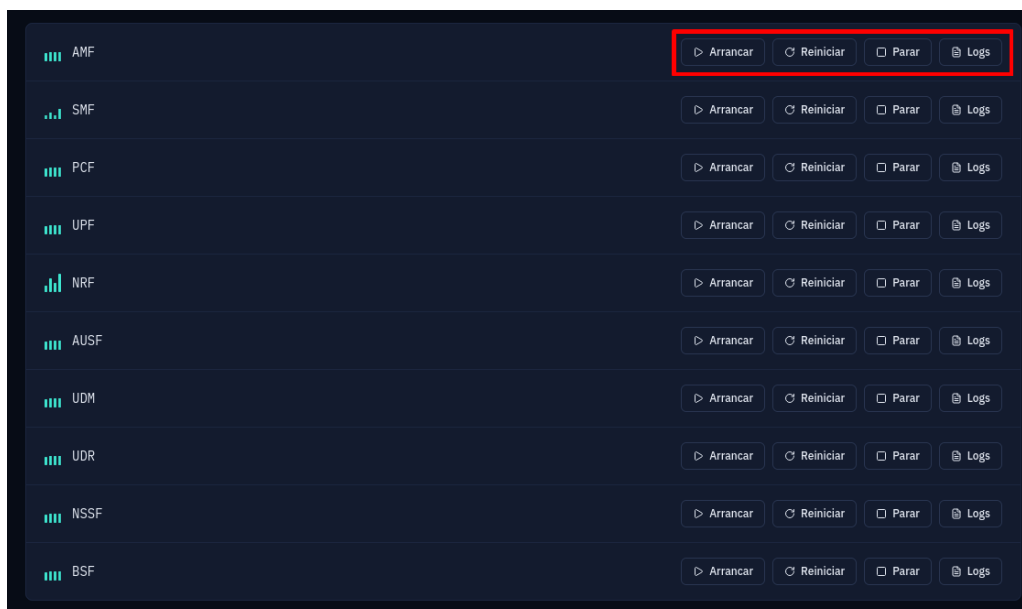


Figura 6.5: Acciones sobre los *Network Functions*.
Fuente: Propia

Al pulsar en el botón *Logs* de un NF cualquiera, se desplegará una ventana nueva en la que se mostrará lo siguiente:

- Las últimas 100 líneas del registro.
- El botón **Actualizar**, que fuerza el refresco de las últimas 100 líneas que se muestran.
- El botón de descarga, que permite la descarga del fichero.
- Una barra de búsqueda, que tiene como alcance todo el fichero entero no solo las últimas 100 líneas.
- Un botón para cerrar esta nueva ventana que se ha desplegado.

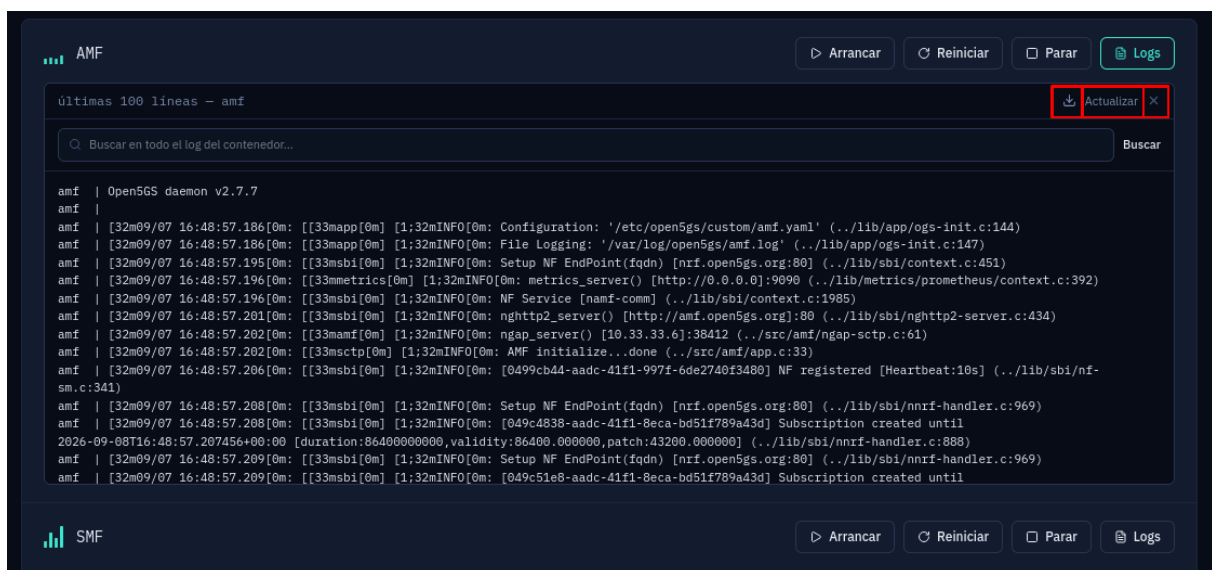
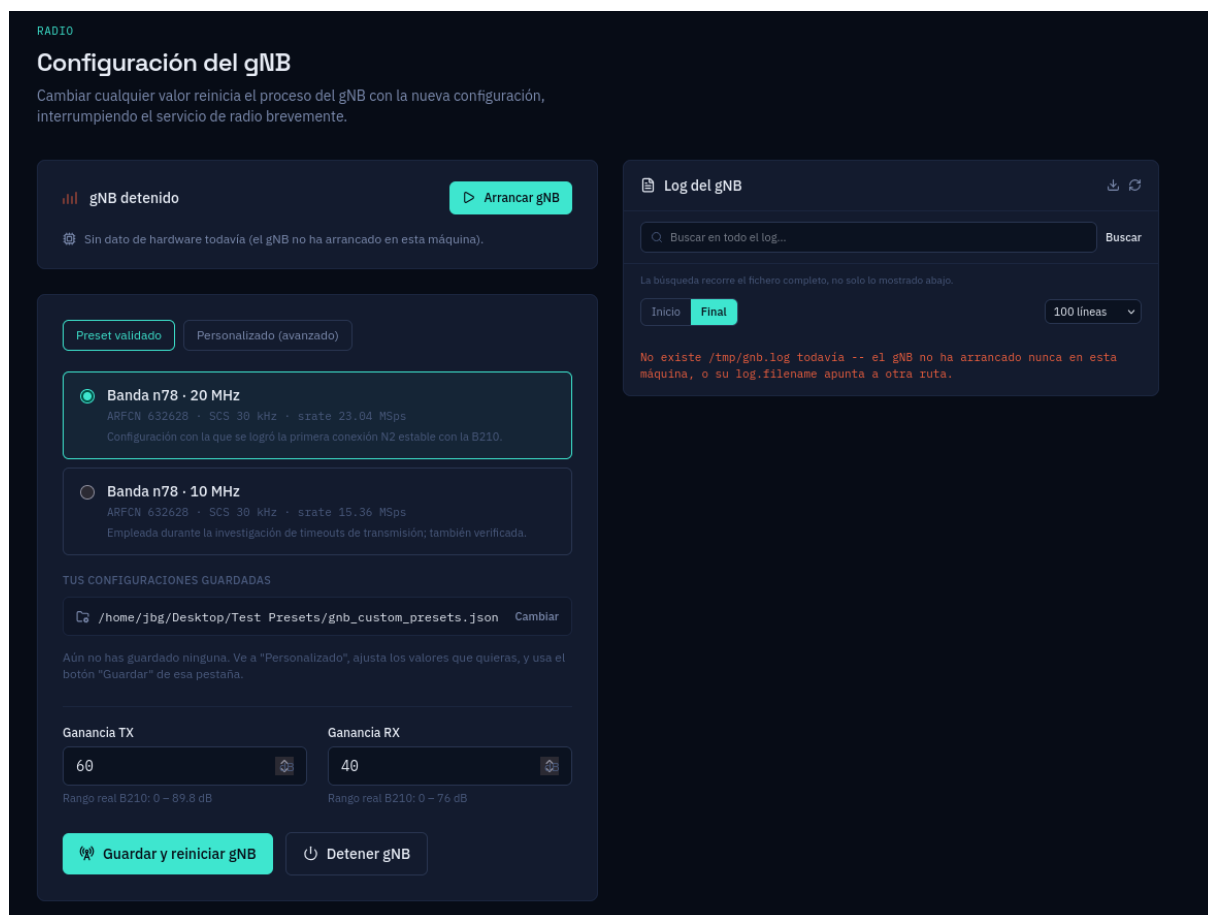


Figura 6.6: Acciones sobre los registros del *Core*.

Fuente: Propia

El siguiente panel es **gNB**, que permite el acceso a una serie de funciones sobre la estación base.

Figura 6.7: Panel gNB del *dashboard*, aún inactivo

Fuente: Propia

De forma similar al panel del núcleo, el panel del gNB que se muestra está inactivo. Cuenta con las siguientes funciones:

- Un botón para arrancar la estación base, junto al nombre de la tarjeta usada como hardware y un icono para indicar su estado (idéntico al que tienen las NF del *core*).
- Selector de *presets* validados o personalizados. Los validados ya se han probado y no dan problemas, mientras que los personalizados dan mayor libertad para modificar los parámetros. Véase la figura 6.8.
- Un campo para modificar la ruta donde se guardan las configuraciones personalizadas.

- Si se selecciona utilizar un preset personalizado, se muestra un botón para guardar esa configuración en la ruta establecida.
- Dos campos numéricos para modificar la ganancia TX y RX.
- Botones para guardar los cambios (si se han hecho) y reiniciar el gNB, y para detener el gNB.
- Un visor del registro del gNB. De forma muy parecida a cómo se comportan los *logs* de las funciones de red, se permite buscar en todo el registro, descargarlo, actualizarlo, elegir si ver el inicio o el final y cambiar el número de líneas del *log* que se muestran (100, 300 o 1000).

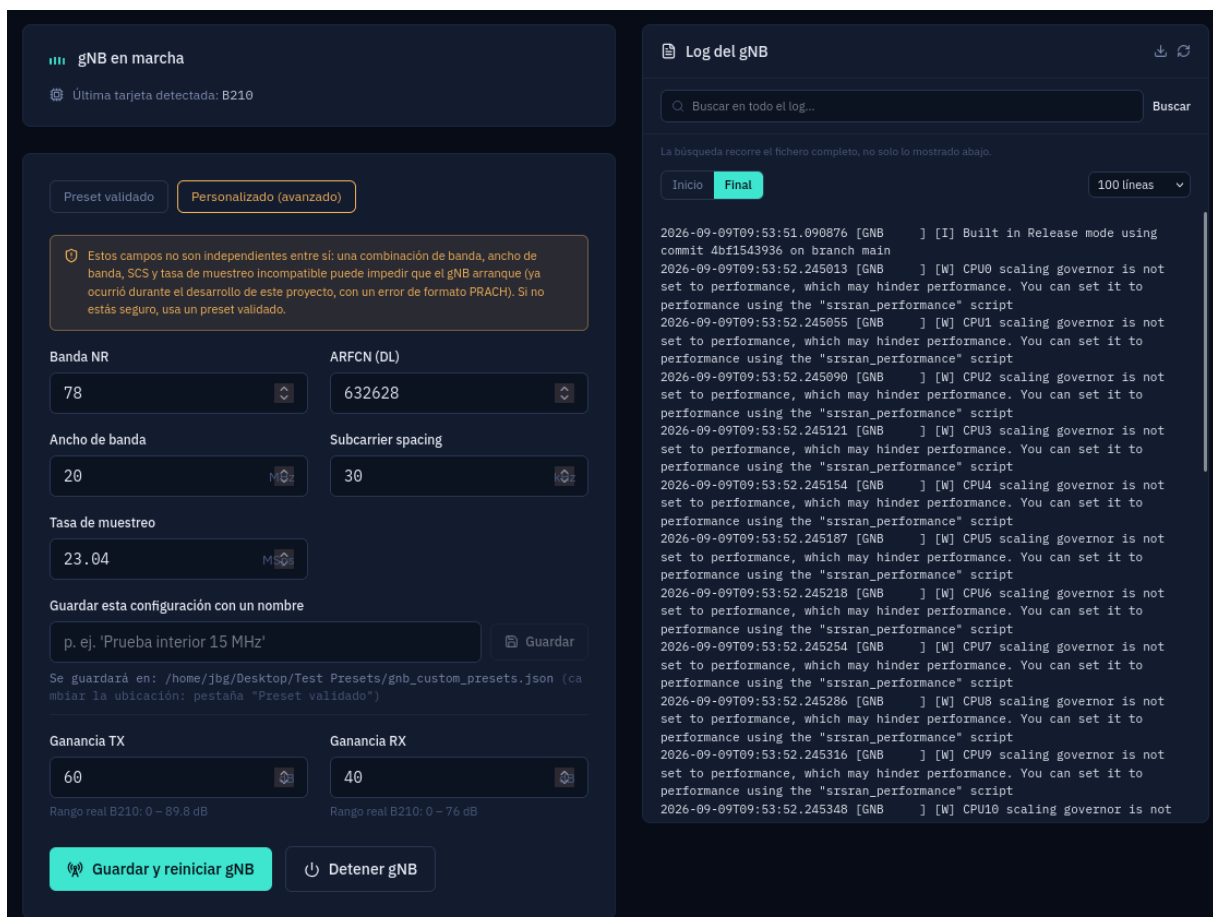


Figura 6.8: Panel gNB del *dashboard*, mostrando un *preset* personalizado
Fuente: Propia

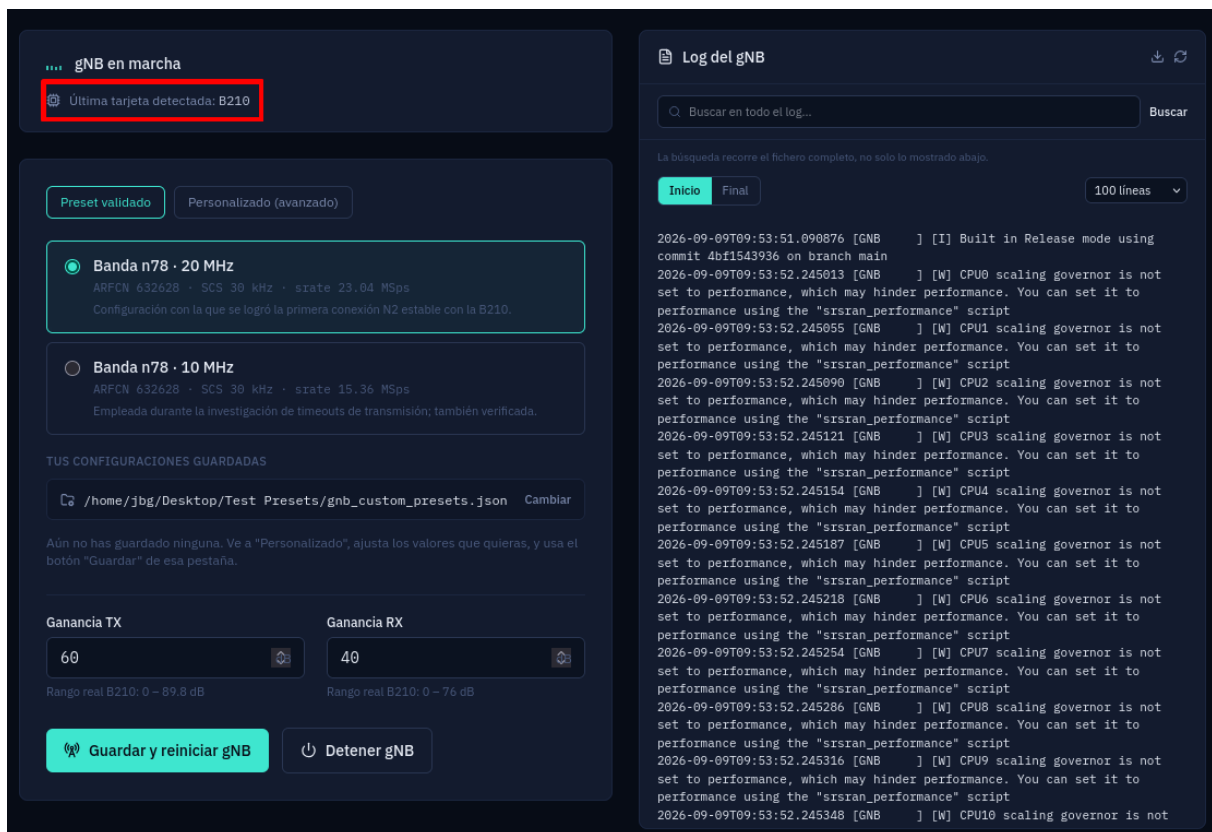


Figura 6.9: Panel gNB del *dashboard*, ya activo, mostrando el *log*
Fuente: Propia

Una vez la estación base está en funcionamiento se podrá ver cómo se muestra el nombre del hardware utilizado, y el *log* pasa a estar disponible.

El último panel que queda es Suscriptores. Las funciones disponibles en este panel son:

- Observar los UEs dados de alta junto a su IMSI, DNN y SST.
- Dar de baja a cualquier suscriptor mediante el botón con un icono de papelera.
- Dar de alta a un nuevo suscriptor mediante el botón Nuevo suscriptor.

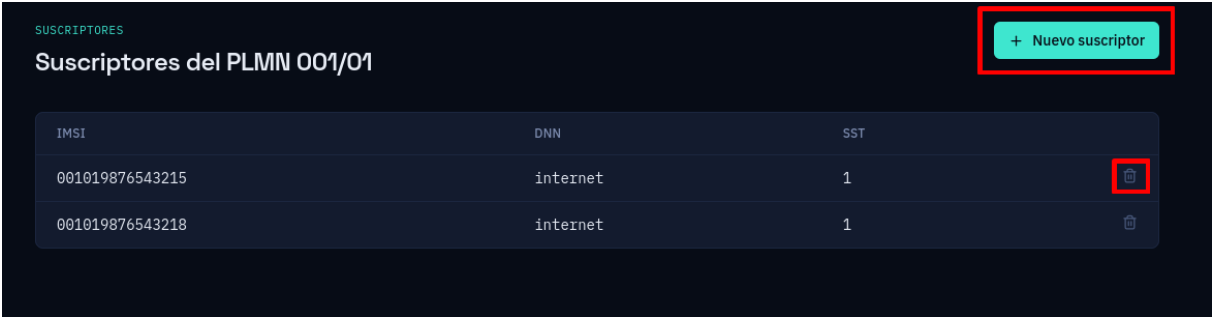


Figura 6.10: Panel Suscriptores del *dashboard*
Fuente: Propia

Los dos suscriptores que se han dado de alta para hacer las pruebas durante el desarrollo y para validar el proyecto una vez terminado cuentan con las siguientes características:

	Suscriptor 1	Suscriptor 2
IMSI	001019876543215	001019876543218
K	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
OPc	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
DNN	internet	internet
SST	1	1

Tabla 6.1: Suscriptores dados de alta para la validación de este trabajo.

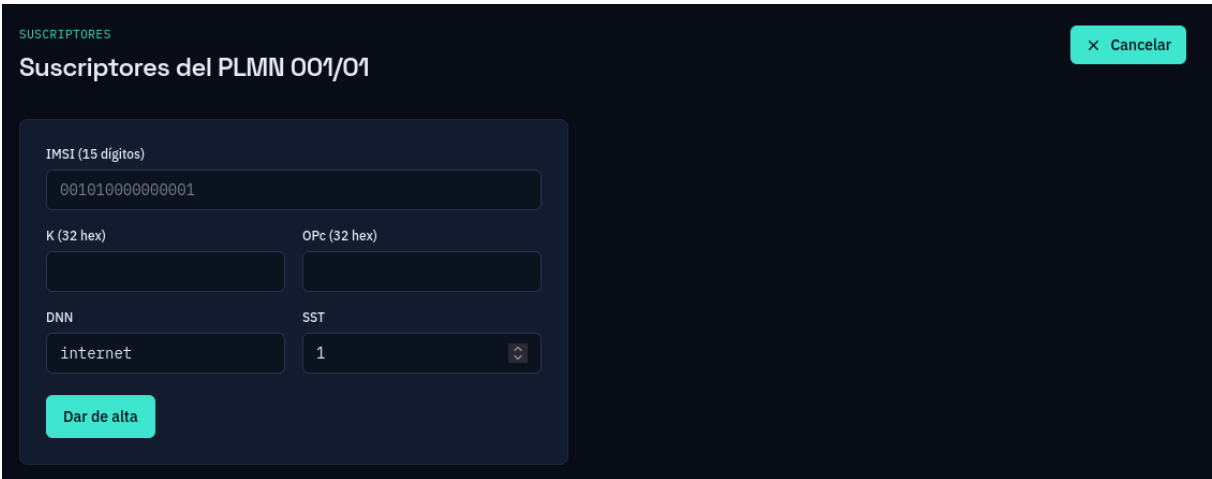


Figura 6.11: Formulario para dar de alta un nuevo UE en el *dashboard*
Fuente: Propia

Una vez puesto en funcionamiento el núcleo y la estación base, se puede comprobar la conexión entre ambos a través del registro del AMF. Haciendo uso de la función de búsqueda implementada, podemos ver que la conexión ha sido exitosa. Véase la figura 6.12.

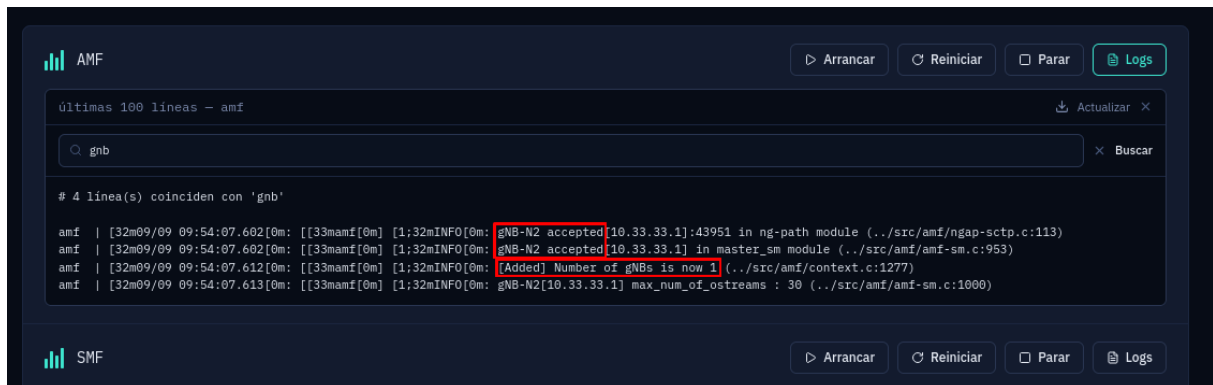


Figura 6.12: Registro del AMF, conexión entre el *core* y el gNB
Fuente: Propia

El siguiente paso a tomar es el arranque del UE y su conexión a la red 5G que se ha puesto en funcionamiento. Al tener el APN configurado como indican apartados anteriores, el equipo se conecta a la red sin necesidad de pasos adicionales.

Para comprobar que esta conexión es exitosa, se vuelve a filtrar el *log* del AMF buscando gNB-UEs. Si el equipo se conecta correctamente, así se indicará en el registro. Véase la figura 6.13.

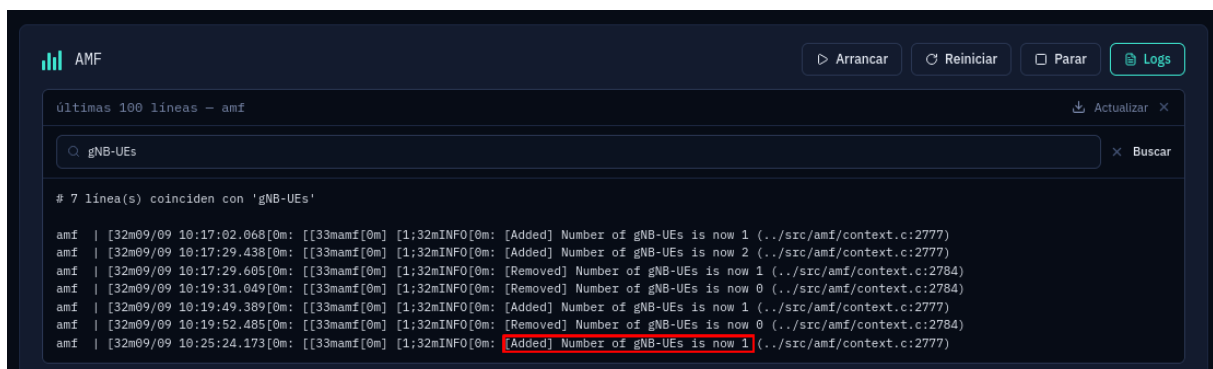


Figura 6.13: Registro del AMF, conexión entre el UE y el gNB
Fuente: Propia

Además de comprobar los registros, el propio UE aparece como conectado a la red privada.

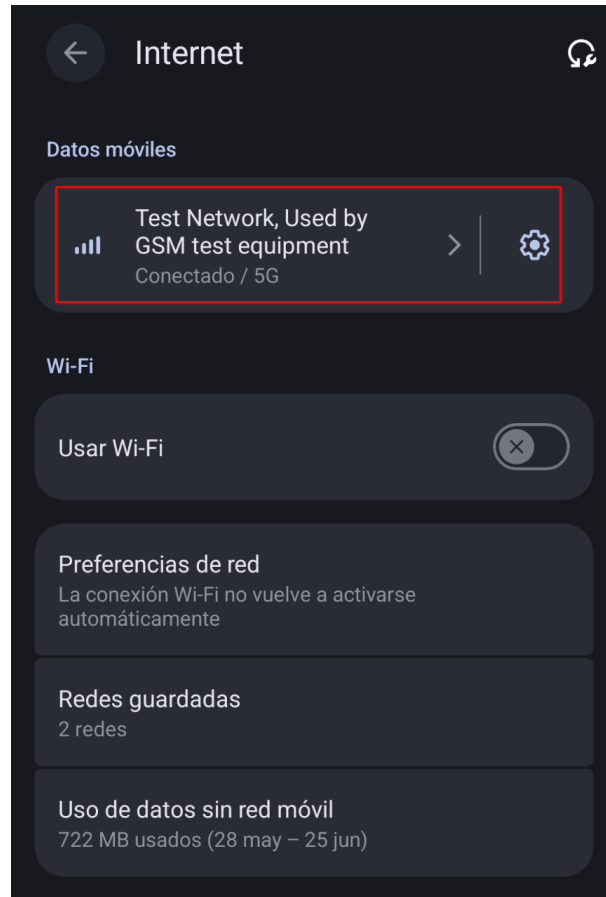


Figura 6.14: UE conectado a la red privada 5G
Fuente: Propia

Tras haber puesto en funcionamiento toda la arquitectura, el panel de resumen ahora indica que está conectado y funcionando: *core*, gNB y UE.

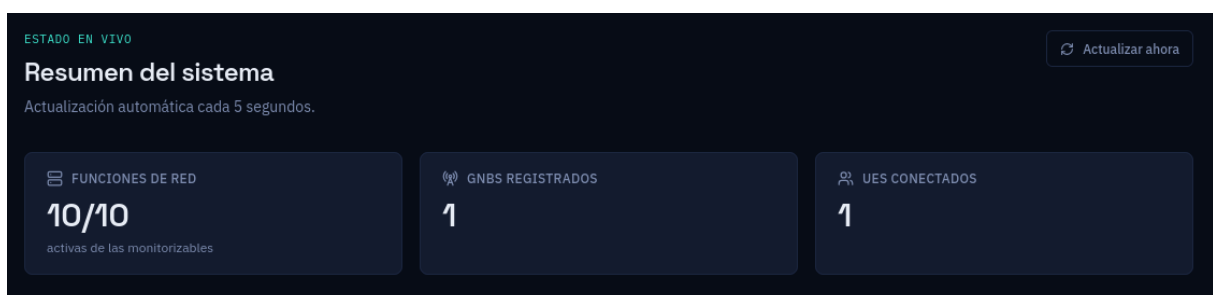


Figura 6.15: Panel Resumen mostrando todos los elementos conectados
Fuente: Propia

De forma similar al cambio que se produce en el panel de resumen, ahora el panel de topología muestra todos los elementos activos y conectados entre sí.

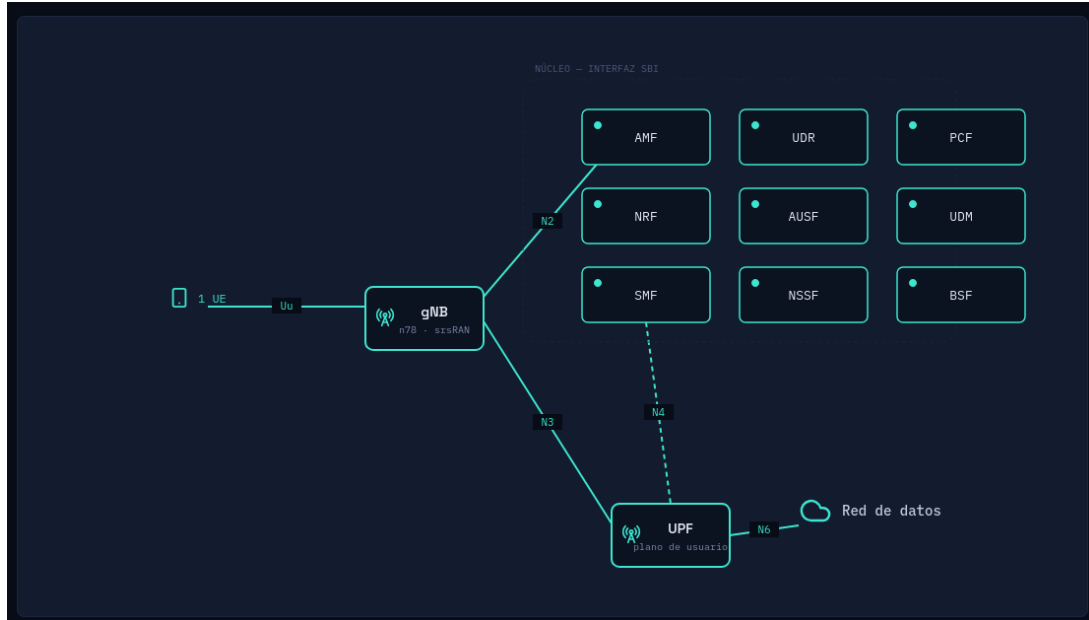


Figura 6.16: Panel Topología del *dashboard*, mostrando todo activo
Fuente: Propia

Por último, el panel de resumen, además de mostrar los contadores, también muestra información sobre el UE conectado a la estación base junto a una gráfica del tráfico de subida de este. Véase la figura 6.17

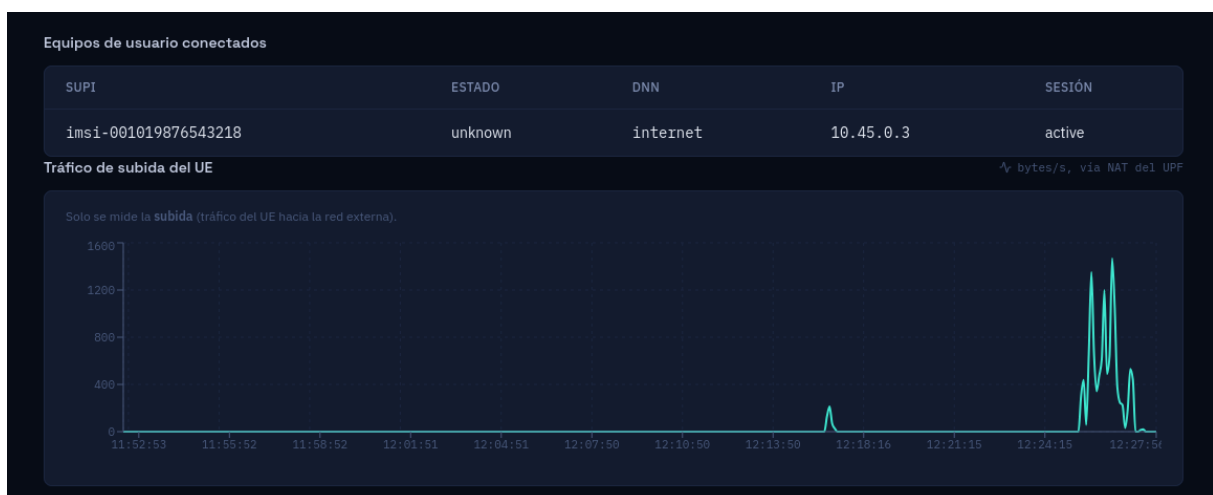


Figura 6.17: Panel Resumen del *dashboard*, mostrando tráfico del UE
Fuente: Propia

La primera instancia de tráfico registrada se corresponde con la conexión del UE a la estación base, mientras que el resto del tráfico detectado se corresponde a una búsqueda de navegador realizada desde el UE.

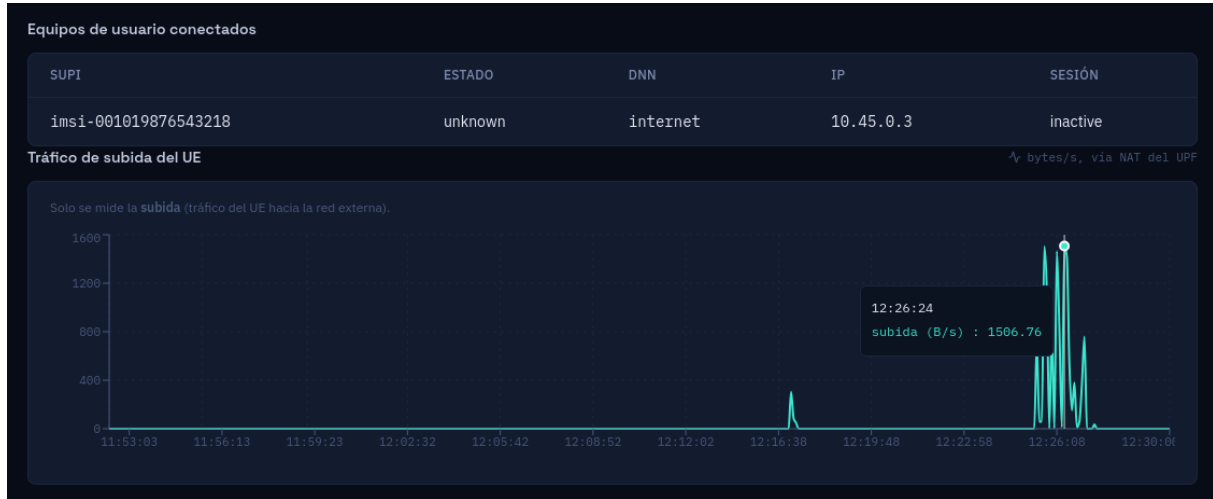


Figura 6.18: Panel Resumen del *dashboard*, detalles del tráfico del UE
Fuente: Propia

Con esto concluye el apartado de resultados. Tras la navegación por toda la aplicación construida se concluye que la aplicación logra los objetivos propuestos y cumple con el objetivo principal: facilitar las tareas de monitorización y gestión de redes privadas 5G.

Capítulo 7

Conclusiones y trabajos futuros

7.1. Conclusiones

Como ya se mencionó al principio de esta memoria, este proyecto se planteó con el objetivo principal de desarrollar una herramienta de monitorización y gestión para una red 5G privada construida enteramente con tecnologías de código abierto, validando su funcionamiento con un despliegue real. A la vista de los resultados presentados en el capítulo anterior, dicho objetivo se considera como alcanzado.

Cada uno de los cinco objetivos específicos planteados en el Capítulo 2 también se ha cumplido: se ha analizado el estado del arte de las tecnologías empleadas y de las herramientas de referencia (el trabajo predecesor y OAIBOX); se ha desplegado una infraestructura de red 5G privada completa y funcional, junto al registro real de un equipo de usuario; se ha diseñado e implementado un panel web de monitorización en tiempo real del núcleo, la estación base y los equipos de usuario conectados; dicho panel se ha ampliado con capacidades de gestión activa; y se ha validado la portabilidad y robustez de la solución, tanto mediante su migración a un segundo equipo como mediante la resolución de las incidencias reales detectadas durante su desarrollo y uso.

El resultado más significativo de todo el trabajo es, sin duda, el registro exitoso de un equipo de usuario real en la red desplegada (Sección 5.24). Este problema no tuvo solución durante gran parte del proyecto, y su resolución exigió, entre otras cosas, recurrir



al PLMN de pruebas reservado por la Recomendación UIT-T E.212 en lugar del empleado inicialmente. A pesar de tener una solución relativamente sencilla, este resultado tiene mucha importancia para la aplicación y el cumplimiento del resto de objetivos, tanto específicos como principal.

Respecto al panel en sí, este trabajo demuestra que es posible ampliar sustancialmente el alcance de una herramienta de monitorización de solo lectura hacia una de gestión activa, sin renunciar por ello a principios de seguridad como el privilegio mínimo o la autenticación de usuarios. La comparación deliberada con el trabajo predecesor y con OAIBOX como referencia comercial (Sección 5.20) ha resultado en una hoja de ruta concreta de ampliaciones que ha probado ser muy útil para el desarrollo.

También se debe mencionar que no todos los aspectos explorados en este trabajo han resultado exitosos, y se ha optado por documentarlos con el mismo nivel de detalle que el resto del trabajo, en lugar de omitirlos. La integración de una tarjeta bladeRF 2.0 como segunda vía de radio, en particular, no llegó a completarse dentro del tiempo disponible, pese a resolverse una cadena considerable de incidencias reales durante su investigación (Anexo D). Se considera que este proceso de investigación, aun sin conseguir resultados tangibles, constituye en sí mismo una aportación válida del trabajo.

Por último, cabe destacar que buena parte del valor de este trabajo reside no solo en el resultado final, sino en el trayecto realizado: la resolución de incidencias reales de integración entre *software* de código abierto de distintas procedencias (srsRAN Project, Open5GS, SoapySDR, Docker), muchas de ellas no documentadas previamente en ningún otro sitio para la combinación exacta de versiones empleada en este trabajo, y que ahora quedan recogidas con su proceso completo de diagnóstico para quien se enfrente a una situación similar en el futuro.

7.2. Trabajos futuros

A lo largo del desarrollo de este trabajo se han identificado diversas líneas de ampliación que, por motivos de alcance o de tiempo disponible, no se han abordado. Se recogen a continuación, ordenadas de mayor a menor cercanía con el trabajo ya realizado.



- **Integración de la tarjeta bladeRF 2.0.** Retomar la investigación descrita en el Anexo D, previsiblemente con acceso a instrumental de medida de radiofrecuencia (un analizador de espectro) que permita confirmar de forma concluyente el comportamiento de la cadena de transmisión, más allá de las verificaciones indirectas realizadas en este trabajo.
- **Contador de tráfico de bajada.** El histórico de tráfico desarrollado (Sección 5.21) mide únicamente la subida del equipo de usuario, al ser esta la dirección que permite aislar mediante la regla **MASQUERADE** del UPF. Una vía no explorada en este trabajo consistiría en emplear **conntrack**, el sistema de seguimiento de conexiones del propio núcleo Linux que ya emplea Docker para el NAT, que registra el tráfico en ambas direcciones de cada conexión activa.
- **Persistencia histórica del tráfico.** El histórico de tráfico desarrollado se mantiene únicamente en memoria, con una ventana limitada a una hora, y se pierde por completo al reiniciar el *backend*. Incorporar una base de datos de series temporales permitiría consultar el comportamiento de la red más allá de dicha ventana.
- **Desglose de tráfico por equipo de usuario individual.** El contador de tráfico actual es un agregado de toda la subred de equipos de usuario, sin distinguir entre ellos. Con varios equipos conectados simultáneamente, esta limitación impediría saber cuál de ellos genera un volumen de tráfico concreto.
- **Validación con la banda n77.** Señalada como ampliación de interés durante el seguimiento con el tutor de este trabajo, no llegó a explorarse por haberse priorizado la investigación de la tarjeta bladeRF durante ese mismo período. Al tratarse de un cambio acotado a la configuración de banda y ARFCN del gNB, sería una ampliación de coste reducido.
- **Consulta de mensajes entre funciones de red.** Como ampliación más ambiciosa, se valoró incorporar al panel una vista de los mensajes intercambiados entre funciones de red, al estilo de un analizador de protocolos como Wireshark.



Descodificar los protocolos NGAP y PFCP empleados en las interfaces N2 y N4 excede claramente el alcance de este trabajo, al tratarse de protocolos binarios codificados en ASN.1; sin embargo, una versión acotada a las llamadas de la interfaz de servicios (SBI, en HTTP/JSON) entre funciones del plano de control resultaría mucho más viable, al no requerir la implementación de ningún descodificador de protocolo binario.

- **Otras métricas del equipo de usuario.** Ampliar la información mostrada sobre cada equipo de usuario conectado con indicadores de calidad de señal (como RSRP o SINR), en la medida en que las funciones de red del núcleo o el propio gNB los expongan.

Anexos

Anexo A

Script de orquestación del núcleo Open5GS

Como se menciona en la Sección 5.4, se desarrolló el siguiente script (`open5gs_ctl.sh`) para automatizar el arranque, parada y comprobación de estado de las 11 funciones de red del núcleo Open5GS empleado en este trabajo, evitando así tener que gestionar manualmente una terminal por cada función.

```
1  #!/bin/bash
2  #
3  # open5gs_ctl.sh - Arranca/para el core de Open5GS (despliegue 5G SA) en el
4  # orden correcto, sin necesidad de abrir varias terminales a mano.
5  #
6  # Uso:
7  #   ./open5gs_ctl.sh start    # arranca todo el core
8  #   ./open5gs_ctl.sh stop    # para todo el core
9  #   ./open5gs_ctl.sh status  # muestra qué funciones siguen vivas
10 #   ./open5gs_ctl.sh logs amf # muestra el log en vivo de una función
    ↪ concreta
11
12 set -u
13
```

Anexo A. Script de orquestación del núcleo Open5GS

```
14 # -----
15 # AJUSTA ESTA RUTA si tu instalación de Open5GS está en otro sitio
16 # -----
17 INSTALL_DIR="/home/jbg/Documents/UEx/TFG/open5gs/open5gs/install"
18
19 BIN_DIR="$INSTALL_DIR/bin"
20 CFG_DIR="$INSTALL_DIR/etc/open5gs"
21 LOG_DIR="$INSTALL_DIR/var/log/open5gs-ctl"
22 PID_DIR="$INSTALL_DIR/var/run/open5gs-ctl"
23
24 # Orden de arranque: NRF primero (todo se registra contra él), SCP y AMF
25 # como intermediarios, el resto de funciones de control después, y el UPF
26 # el último (abre la interfaz TUN, conviene que el resto ya esté listo).
27 # Excluidos a propósito: mmed, hssd, sgwcd, sgwud, pcrfd (son de 4G/NSA).
28 NF_ORDER="nrf scp amf ausf udm udr pcf nssf bsf smf upf"
29
30 mkdir -p "$LOG_DIR" "$PID_DIR"
31
32 # -----
33 # Comprobación inicial: que la instalación exista donde esperamos
34 # -----
35 check_install() {
36     if [ ! -x "$BIN_DIR/open5gs-nrfd" ]; then
37         echo "ERROR: no encuentro $BIN_DIR/open5gs-nrfd"
38         echo "Revisa la variable INSTALL_DIR al principio de este script."
39         exit 1
40     fi
41 }
42
43 # -----
44 # start: arranca cada NF en segundo plano, guardando su PID y su log
```

```
45 # -----
46 start_all() {
47     check_install
48     echo "Arrancando core de Open5GS..."
49
50     for nf in $NF_ORDER; do
51         pidfile="$PID_DIR/${nf}.pid"
52
53         if [ -f "$pidfile" ] && kill -0 "$(cat "$pidfile")" 2>/dev/null; then
54             echo " [skip] $nf ya está en marcha (PID $(cat "$pidfile"))"
55             continue
56         fi
57
58         echo " [start] open5gs-${nf}d"
59         "$BIN_DIR/open5gs-${nf}d" -c "$CFG_DIR/${nf}.yaml" \
60             > "$LOG_DIR/${nf}.log" 2>&1 &
61         echo $! > "$pidfile"
62
63         # Pequeña espera tras el NRF y el AMF, que actúan de intermediarios
64         # para el resto de funciones que se registran contra ellos.
65         if [ "$nf" = "nrf" ] || [ "$nf" = "amf" ]; then
66             sleep 2
67         else
68             sleep 0.3
69         fi
70     done
71
72     echo ""
73     echo "Core arrancado. Comprobando estado en 3 segundos..."
74     sleep 3
75     status_all
```

```
76 }
77
78 # -----
79 # stop: para todas las NF, en orden inverso (UPF primero, NRF al final)
80 # -----
81 stop_all() {
82     echo "Parando core de Open5GS..."
83     for nf in $(echo "$NF_ORDER" | tr ' ' '\n' | tac); do
84         pidfile="$PID_DIR/$nf.pid"
85         if [ -f "$pidfile" ]; then
86             pid="$(cat "$pidfile")"
87             if kill -0 "$pid" 2>/dev/null; then
88                 echo " [stop] open5gs-${nf}d (PID $pid)"
89                 kill "$pid" 2>/dev/null
90             fi
91             rm -f "$pidfile"
92         fi
93     done
94     echo "Hecho."
95 }
96
97 # -----
98 # status: qué NF siguen vivas según su PID guardado
99 # -----
100 status_all() {
101     printf "%-6s %-8s %s\n" "NF" "PID" "ESTADO"
102     for nf in $NF_ORDER; do
103         pidfile="$PID_DIR/$nf.pid"
104         if [ -f "$pidfile" ] && kill -0 "$(cat "$pidfile")" 2>/dev/null; then
105             printf "%-6s %-8s %s\n" "$nf" "$(cat "$pidfile")" "activo"
106         else
```

```
107     printf "%-6s %-8s %s\n" "$nf" "-" "parado"
108     fi
109 done
110 echo ""
111 echo "Interfaz ogstun:"
112 ip -brief addr show ogstun 2>/dev/null || echo " (no existe todavía / UPF
    ↪ no arrancado)"
113 }
114
115 # -----
116 # logs <nf>: sigue en vivo el log de una función concreta
117 # -----
118 tail_log() {
119     nf="${1:-}"
120     if [ -z "$nf" ]; then
121         echo "Uso: $0 logs <nrf|scp|amf|ausf|udm|udr|pcf|nssf|bsf|smf|upf|all>"
122         exit 1
123     fi
124
125     if [ "$nf" = "all" ]; then
126         # Muestra todos los logs entrelazados, cada línea precedida por el
127         # nombre de fichero de origen
128         tail -f "$LOG_DIR"/*.log
129         return
130     fi
131
132     logfile="$LOG_DIR/${nf}.log"
133     if [ ! -f "$logfile" ]; then
134         echo "No existe $logfile (¿está arrancado $nf?)"
135         exit 1
136     fi
```

```
137     tail -f "$logfile"
138 }
139
140 # -----
141 case "${1:-}" in
142     start)  start_all ;;
143     stop)   stop_all ;;
144     status) status_all ;;
145     logs)   tail_log "${2:-}" ;;
146     *)
147         echo "Uso: $0 {start|stop|status|logs <nf>}"
148         exit 1
149     ;;
150 esac
```

Anexo B

Script de control del gNB

Como se menciona en la Sección 5.10.5, se desarrolló el siguiente *script* (`gnb_ctl.sh`) para gestionar el ciclo de vida del proceso del gNB (arranque, parada, reinicio y consulta de estado) con privilegio mínimo, invocado por el *backend* del panel a través de una regla *sudoers* acotada específicamente a este *script*.

```
1  #!/bin/bash
2  #
3  # gnb_ctl.sh - Arranca/para/reinicia el proceso del gNB de forma controlada.
4  #
5  #
6  # Ejemplo de línea a añadir con 'sudo visudo -f /etc/sudoers.d/dashboard':
7  #   <usuario> ALL=(ALL) NOPASSWD: /ruta/a/gnb_ctl.sh
8  #
9  # Uso:
10 #   ./gnb_ctl.sh start <fichero.yml>
11 #   ./gnb_ctl.sh stop
12 #   ./gnb_ctl.sh restart <fichero.yml>
13 #   ./gnb_ctl.sh status
14 #
15 # Configuración específica de esta máquina: ver deployment.conf (no las
16 # rutas dentro de este script)
```

```
17
18 set -u
19
20 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
21 CONFIG_FILE_PATH="$SCRIPT_DIR/deployment.conf"
22
23 if [ ! -f "$CONFIG_FILE_PATH" ]; then
24     echo "ERROR: no se encuentra $CONFIG_FILE_PATH"
25     echo "Copia deployment.conf.example como deployment.conf en esta misma
    ↪ carpeta y ajusta sus valores."
26     exit 1
27 fi
28 # shellcheck source=deployment.conf
29 source "$CONFIG_FILE_PATH"
30
31 if [ -z "${GNB_BIN:-}" ]; then
32     echo "ERROR: GNB_BIN no está definido en $CONFIG_FILE_PATH"
33     exit 1
34 fi
35
36 GNB_DIR="$(dirname "$GNB_BIN")"
37 LOG_FILE="/tmp/gnb_ctl.log"
38 PID_FILE="/tmp/gnb_ctl.pid"
39
40 # Variables necesarias para que el binario enlace con una UHD compilada a
41 # mano en vez de la del sistema. 'sudo' arranca con un entorno
42 # mínimo, así que hay que fijarlas explícitamente aquí. Si UHD_LIB_DIR
43 # está vacío en deployment.conf (se usa la UHD del sistema), no se tocan
44 if [ -n "${UHD_LIB_DIR:-}" ]; then
45     export LD_LIBRARY_PATH="${UHD_LIB_DIR}:${LD_LIBRARY_PATH:-}"
46 fi
```

```
47 if [ -n "${UHD_IMAGES_DIR:-}" ]; then
48     export UHD_IMAGES_DIR="${UHD_IMAGES_DIR}"
49 fi
50
51 start_gnb() {
52     config_file="${1:-}"
53     if [ -z "$config_file" ]; then
54         echo "ERROR: falta la ruta al fichero de configuración del gNB"
55         exit 1
56     fi
57     if [ ! -f "$config_file" ]; then
58         echo "ERROR: no existe el fichero $config_file"
59         exit 1
60     fi
61     if [ -f "$PID_FILE" ] && kill -0 "$(cat "$PID_FILE")" 2>/dev/null; then
62         echo "El gNB ya está en marcha (PID $(cat "$PID_FILE"))"
63         return 0
64     fi
65
66     cd "$GNB_DIR" || exit 1
67     nohup "$GNB_BIN" -c "$config_file" > "$LOG_FILE" 2>&1 &
68     echo $! > "$PID_FILE"
69
70     # Espera para detectar fallos de arranque inmediatos
71     sleep 2
72     if kill -0 "$(cat "$PID_FILE")" 2>/dev/null; then
73         echo "gNB arrancado correctamente (PID $(cat "$PID_FILE"))"
74     else
75         echo "ERROR: el gNB terminó inmediatamente tras arrancar. Log:"
76         tail -20 "$LOG_FILE"
77         rm -f "$PID_FILE"
```

```
78     exit 1
79 fi
80 }
81
82 stop_gnb() {
83     if [ ! -f "$PID_FILE" ]; then
84         echo "El gNB no parece estar en marcha (no hay PID registrado)"
85         return 0
86     fi
87     pid="$(cat "$PID_FILE")"
88     if kill -0 "$pid" 2>/dev/null; then
89         kill "$pid"
90         sleep 1
91         echo "gNB detenido (PID $pid)"
92     else
93         echo "El proceso (PID $pid) ya no existía"
94     fi
95     rm -f "$PID_FILE"
96 }
97
98 status_gnb() {
99     if [ -f "$PID_FILE" ] && kill -0 "$(cat "$PID_FILE")" 2>/dev/null; then
100         echo "activo (PID $(cat "$PID_FILE"))"
101     else
102         echo "parado"
103     fi
104 }
105
106 case "${1:-}" in
107     start)
108         start_gnb "${2:-}"
```

```
109     ;;
110 stop)
111     stop_gnb
112     ;;
113 restart)
114     stop_gnb
115     sleep 1
116     start_gnb "${2:-}"
117     ;;
118 status)
119     status_gnb
120     ;;
121 *)
122     echo "Uso: $0 {start <config.yml>|stop|restart <config.yml>|status}"
123     exit 1
124     ;;
125 esac
```

Anexo C

Script de control de las funciones de red en Docker

Como se menciona en las Secciones 5.10.5 y 5.12.1, se desarrolló el siguiente *script* (`docker_ctl.sh`) para gestionar, con el mismo modelo de privilegio mínimo empleado para el gNB (Anexo B), el arranque, parada y reinicio de las funciones de red del *core* (de forma individual o conjunta), la consulta de sus registros de actividad, y la lectura del contador real de tráfico de subida del UE.

```
1  #!/bin/bash
2  #
3  # docker_ctl.sh - Arranca/para/reinicia una función de red concreta del
4  # core en Docker, consulta su log reciente, o lee el contador de tráfico
5  # de subida real del UE (regla MASQUERADE del UPF), de forma controlada.
6  #
7  # Ejemplo de línea a añadir con 'sudo visudo -f /etc/sudoers.d/dashboard':
8  #   <usuario> ALL=(ALL) NOPASSWD: /ruta/a/docker_ctl.sh
9  #
10 # Uso:
11 #   ./docker_ctl.sh start <nf_name>
12 #   ./docker_ctl.sh stop <nf_name>
13 #   ./docker_ctl.sh restart <nf_name>
```

```
14 # ./docker_ctl.sh logs <nf_name> [num_lineas]
15 # ./docker_ctl.sh nat-stats
16 # ./docker_ctl.sh core-up
17 # ./docker_ctl.sh core-down
18 #
19 # Configuración específica de esta máquina: ver deployment.conf.
20
21 set -u
22
23 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
24 CONFIG_FILE_PATH="$SCRIPT_DIR/deployment.conf"
25
26 if [ ! -f "$CONFIG_FILE_PATH" ]; then
27     echo "ERROR: no se encuentra $CONFIG_FILE_PATH"
28     echo "Copia deployment.conf.example como deployment.conf en esta misma
29     ↪ carpeta y ajusta sus valores."
30     exit 1
31 fi
32 # shellcheck source=deployment.conf
33 source "$CONFIG_FILE_PATH"
34
35 if [ -z "${DOCKER_COMPOSE_FILE:-}" ] || [ -z "${DOCKER_COMPOSE_ENV_FILE:-}"
36 ↪ ]; then
37     echo "ERROR: DOCKER_COMPOSE_FILE/DOCKER_COMPOSE_ENV_FILE no están definidos
38     ↪ en $CONFIG_FILE_PATH"
39     exit 1
40 fi
41
42 # Whitelist de NFs gestionables
43 VALID_SERVICES="amf smf pcf upf nrf ausf udm udr nssf bsf"
```

```
42 USAGE="Uso: $0 {start|stop|restart} <nf_name> | $0 logs <nf_name>
↪ [num_lineas] | $0 logs-full <nf_name> | $0 logs-search <nf_name>
↪ <texto> | $0 nat-stats | $0 core-up | $0 core-down"
43
44 action="${1:-}"
45
46 # 'nat-stats', 'core-up' y 'core-down' no reciben NF, son comandos fijos,
47 # sin ningún parámetro que venga del backend, por el mismo motivo de
48 # seguridad que 'nat-stats' (ver más abajo).
49 if [ "$action" = "nat-stats" ]; then
50     docker exec upf iptables -t nat -L POSTROUTING -n -v -x
51     exit $?
52 fi
53
54 if [ "$action" = "core-up" ]; then
55     docker compose -f "$DOCKER_COMPOSE_FILE" --env-file
↪ "$DOCKER_COMPOSE_ENV_FILE" up -d
56     exit $?
57 fi
58
59 if [ "$action" = "core-down" ]; then
60     docker compose -f "$DOCKER_COMPOSE_FILE" --env-file
↪ "$DOCKER_COMPOSE_ENV_FILE" down
61     exit $?
62 fi
63
64 nf="${2:-}"
65
66 if [ -z "$action" ] || [ -z "$nf" ]; then
67     echo "$USAGE"
68     exit 1
```

```
69 fi
70
71 valid=false
72 for s in $VALID_SERVICES; do
73     if [ "$s" = "$nf" ]; then
74         valid=true
75         break
76     fi
77 done
78 if [ "$valid" = false ]; then
79     echo "ERROR: '$nf' no es una función de red gestionada por este script"
80     exit 1
81 fi
82
83 case "$action" in
84     start|stop|restart)
85         docker compose -f "$DOCKER_COMPOSE_FILE" --env-file
86             ↪ "$DOCKER_COMPOSE_ENV_FILE" "$action" "$nf"
87         ;;
88     logs)
89         tail_lines="${3:-100}"
90         if ! [[ "$tail_lines" =~ ^[0-9]+$ ]] || [ "$tail_lines" -lt 1 ] || [
91             ↪ "$tail_lines" -gt 1000 ]; then
92             echo "ERROR: el número de líneas debe ser un entero entre 1 y 1000"
93             exit 1
94         fi
95         docker compose -f "$DOCKER_COMPOSE_FILE" --env-file
96             ↪ "$DOCKER_COMPOSE_ENV_FILE" \
97             logs --no-color --tail "$tail_lines" "$nf"
98         ;;
99     logs-full)
```

```
97     # Sin límite de líneas -- pensado para descarga, no para mostrar en
98     # pantalla directamente (puede ser grande en contenedores de larga
99     ↪ duración).
100     docker compose -f "$DOCKER_COMPOSE_FILE" --env-file
101     ↪ "$DOCKER_COMPOSE_ENV_FILE" \
102     logs --no-color "$nf"
103     ;;
104     logs-search)
105     search_term="${3:-}"
106     if [ -z "$search_term" ]; then
107         echo "ERROR: falta el término de búsqueda"
108         exit 1
109     fi
110     docker compose -f "$DOCKER_COMPOSE_FILE" --env-file
111     ↪ "$DOCKER_COMPOSE_ENV_FILE" \
112     logs --no-color "$nf" | grep -F -i -- "$search_term"
113     ;;
114     *)
115     echo "$USAGE"
116     exit 1
117     ;;
118 esac
```

Anexo D

Incidencias, diagnóstico y resolución

Este anexo recoge las incidencias encontradas durante el desarrollo de este trabajo a las que se hace referencia desde el Capítulo de Implementación y Desarrollo, junto al diagnóstico de estas y en algunos casos, la resolución. Se ha optado por separar este bloque del cuerpo principal de la memoria para que dicho capítulo funcione como una guía directa de lo construido, mientras que este anexo documenta el proceso de investigación seguido en cada caso.

D.1. Paquete de *firmware* de la bladeRF en el componente `contrib`

El primer intento de instalación de las herramientas base de la bladeRF (Sección 5.1), se hizo con este comando:

```
1 sudo apt install bladerf libbladerf2 libbladerf-dev bladerf-firmware-fx3
```

Esto falló con el error `Unable to locate package bladerf-firmware-fx3`. La causa se localizó en que dicho paquete se distribuye dentro del componente `contrib` del repositorio de Debian (al tratarse de un instalador de *firmware* no libre para el controlador USB FX3), mientras que el fichero `/etc/apt/sources.list` del equipo únicamente tenía habilitados los componentes `main` y `non-free-firmware`. La solución consistió en añadir `contrib`

a las líneas correspondientes del fichero, tras lo cual la instalación se completó sin más incidencias.

D.2. Rama por defecto del repositorio archivado de srsRAN Project

El primer intento de compilación de srsRAN Project (Sección 5.3), clonando el repositorio sin especificar rama, falló con `CMake Error: The source directory ... does not appear to contain CMakeLists.txt`. La inspección del directorio clonado reveló que este contenía únicamente un fichero `README.md`, pese a que el propio `git clone` había finalizado sin errores. La causa se identificó ejecutando `git status`, que mostró `On branch archive`: al haberse archivado el repositorio de srsRAN Project en GitHub (con motivo de su transición a OCUDU, según se explica en la Sección 4.1.4), la rama por defecto (*HEAD*) del repositorio pasó a ser una rama `archive` que contiene únicamente un aviso de la migración, en lugar del código fuente completo.

La solución consistió en clonar explícitamente la rama `main` histórica, donde el código sigue disponible íntegro aunque sin recibir ya actualizaciones. Con esto, la compilación se completó sin errores, obteniendo la versión **25.10.0** (*commit* 4bf1543936), verificada mediante `./gnb --version`.

D.3. Incidencias múltiples en la integración del gNB con la bladeRF

A partir del fichero de configuración de partida (Sección 5.5), el arranque del gNB con la bladeRF pasó por una serie de incidencias encadenadas, cada una en una capa distinta de la pila (parseo de configuración, comportamiento interno de UHD, y temporización de los búferes de radio), que se documentan a continuación en el orden en que se presentaron.

D.3.1. Valor vacío no admitido en `device_driver`

El primer intento de arranque falló en la fase de parseo, antes incluso de intentar abrir la radio:

```
1 INI was not able to parse ru_sdr.device_driver.++
```

El Anexo B del trabajo predecesor indicaba dejar el campo `device_driver` vacío para permitir la autodetección del dispositivo [1]; sin embargo, la versión de srsRAN empleada en este trabajo (25.10.0, sensiblemente más reciente que la utilizada en aquel) ya no admite un valor vacío en dicho campo. La solución consistió en establecer `device_driver: uhd` de forma explícita, dejando en su lugar el argumento `type=` de `device_args` sin especificar, para que sea UHD quien autodetecte el dispositivo disponible, que es, en realidad, el mecanismo de autodetección que empleaba el trabajo predecesor, solo que aplicado al parámetro correcto.

D.3.2. Fuente de sincronización no soportada

Superada la anterior incidencia, la apertura de la radio fallaba con:

```
1 Error: couldn't set sync source: Invalid time source internal. Supported:
2 srsRAN ERROR: Unable to create radio session.
```

Se trata del mismo problema que motivó uno de los parches del Anexo B del trabajo predecesor [1]; no obstante, la investigación reveló que dicho parche, pensado para versiones de UHD anteriores a la 3.14, no era aplicable a la versión empleada en este trabajo (UHD 4.8.0), que sigue una rama de código distinta dentro del propio fichero `radio_uhd_device.h` de srsRAN (seleccionada mediante una directiva de preprocesador `#if UHD_VERSION <3140099`). Se aplicó el parche equivalente sobre la rama correspondiente a esa versión de UHD:

```
1 #else
```

```

2   return safe_execution([this, &clock_source]() {
    ↪   usrp->set_clock_source(clock_source); });
3 #endif

```

Sin embargo, el error persistió de forma idéntica tras recompilar, lo que llevó a descartar que el origen estuviera en el código de srsRAN. La investigación del código fuente de UHD reveló que su implementación genérica de `multi_usrp::set_clock_source()` gestiona internamente, de forma conjunta, una propiedad combinada de sincronización (*sync source*) que involucra tanto la fuente de reloj como la fuente de tiempo, con independencia de cuál de los dos métodos invoque la aplicación cliente. Dado que SoapyBladeRF no sobrescribe los métodos `listTimeSources/getTimeSource/setTimeSource` de la clase base de SoapySDR (al no existir en la bladeRF un concepto de fuente de tiempo independiente de la fuente de reloj), la lista de fuentes de tiempo soportadas que UHD recibe a través de SoapyUHD está vacía, de ahí el **Supported:** en blanco del mensaje de error.

La solución consistió en añadir a SoapyBladeRF una implementación de compatibilidad para estos tres métodos, reportando `“internal”` como única fuente de tiempo válida (sin efecto real sobre el hardware, ya que la bladeRF no dispone de una entrada de sincronización externa de este tipo):

```

1  std::vector<std::string> bladeRF_SoapySDR::listTimeSources(void) const
2  {
3      std::vector<std::string> sources;
4      sources.push_back("internal");
5      return sources;
6  }
7
8  void bladeRF_SoapySDR::setTimeSource(const std::string &source)
9  {
10     // No-op: no existe una fuente de tiempo real que configurar.
11     (void)source;

```

```

12 }
13
14 std::string bladeRF_SoapySDR::getTimeSource(void) const
15 {
16     return "internal";
17 }

```

Al no poder modificarse un paquete binario ya instalado, fue necesario compilar SoapYBladeRF desde código fuente con este parche aplicado, sustituyendo así al paquete `soapysdr-module-bladerf` de Debian empleado hasta entonces.

D.3.3. *Timeouts* de los búferes de transmisión y recepción

Resuelta la incidencia anterior, el gNB llegó a completar el registro contra el AMF (N2: Connection to AMF on 127.0.0.5:38412 completed) y a arrancar (==== gNB started ===), pero de inmediato comenzó a generar de forma continua el siguiente error:

```

1 [ERROR @ host/libraries/libbladeRF/src/streaming/sync.c:352] wait_for_buffer:
2 Timed out waiting for buf_ready after 1 ms

```

Se trata de la misma incidencia recogida en el Anexo B del trabajo predecesor [1], relativa a los valores por defecto de *timeout* de los búferes de transmisión y recepción en el driver UHD de srsRAN, insuficientes para el flujo de muestras real de la bladeRF. La solución, verificando primero las líneas exactas en la versión empleada (que no coincidían con las indicadas en el Anexo B, al tratarse de una versión de srsRAN distinta), consistió en fijar ambos valores a cero:

```
1 // lib/radio/uhd/radio_uhd_tx_stream.h
2 static constexpr double TRANSMIT_TIMEOUT_S = 0;
3
4 // lib/radio/uhd/radio_uhd_rx_stream.h
5 static constexpr double RECEIVE_TIMEOUT_S = 0.0f;
```

D.4. El terminal no detecta la celda (bladeRF)

Con el APN configurado en el terminal (un Xiaomi POCO X6, con soporte confirmado de 5G SA en banda n78) y una búsqueda manual de operador (necesaria, ya que el PLMN de pruebas no aparece en un escaneo automático), el terminal no detectó en ningún momento la celda emitida por el gNB con la bladeRF, ni siquiera con el dispositivo físicamente pegado a la antena de transmisión. El registro del AMF no mostraba ningún intento de conexión durante las pruebas, lo que sitúa el problema en la capa de radio, antes de llegar al núcleo.

Se descartaron las siguientes causas:

- **Antena y potencia:** se confirmó la conexión física de la antena en el puerto correcto, y se elevó `tx_gain` al máximo del rango accesible desde `srsRAN`.
- **Emisión de RF real:** ante la imposibilidad de usar un analizador de espectro, se validó mediante la propia bladeRF, generando manualmente (con `bladeRF-cli`) un tono de prueba y capturándolo con las antenas de transmisión y recepción próximas entre sí. La magnitud media de la señal recibida en la banda n78 (3489,42 MHz) resultó muy superior al ruido de fondo medido en las mismas condiciones sin transmitir, confirmando que la tarjeta emite señal real en la frecuencia correcta.

D.5. *Timeouts* de transmisión con la USRP B210

Al arrancar el gNB con la B210 por primera vez (Sección 5.8), aparecía de forma inmediata y continua el error:

```
1 Error: exceeded maximum number of timed out transmissions.
```

Se llevó a cabo un descarte de varias causas, cada una verificada con evidencia concreta antes de descartarla:

1. **Velocidad de enlace USB:** confirmada SuperSpeed (USB 3.0) mediante `lsusb -d 2500:` y contraste con el propio registro del kernel (`dmesg`).
2. **Gestión de energía de la CPU:** aplicado el script `srsran_performance`, que fija el gobernador de CPU a *performance* y desactiva el *polling* DRM/KMS.
3. **Límite de memoria de usbfs:** incrementado de 16 MB (valor por defecto del kernel) a 1000 MB, un ajuste habitual documentado para dispositivos USB de alto caudal.
4. ***Autosuspend* USB:** descartado, ya que el dispositivo ya contaba con gestión de energía en modo *on*.
5. **Tamaño de los búferes de transporte y tasa de muestreo:** se probaron incrementos y reducciones (`num_recv_frames/num_send_frames`, y una reducción del ancho de banda de canal de 20 a 10 MHz), sin efecto sobre el error.
6. **Contención del controlador USB físico:** siguiendo un caso reportado en el propio repositorio de srsRAN, en el que el problema desapareció al desconectar periféricos que compartían controlador USB con la USRP [32], se aisló la B210 en un puerto sin otros dispositivos colgando del mismo controlador. Sin efecto.
7. **Fallo de hardware o de enlace USB:** descartado mediante inspección del propio registro del kernel (`dmesg`) durante el fallo, que no mostraba ninguna desconexión, reinicio o error de controlador durante los episodios de *timeout*.

D.5.1. Réplica del entorno del trabajo predecesor

Al agotarse las causas anteriores, se consultó la propia memoria del trabajo predecesor para comparar el entorno exacto empleado: Ubuntu 20.04/22.04, con UHD instalada desde el repositorio oficial de Ettus (`ppa:ettusresearch/uhd`), en su versión **4.6.0** [1], frente a la versión **4.8.0** empaquetada por Debian Trixie empleada en este trabajo. Se descartó reinstalar el sistema operativo completo (una acción reportada como solución, aunque no concluyente, en otro hilo de la comunidad de srsRAN [33]) por su elevado coste, y en su lugar se llevó a cabo otra prueba: compilar UHD 4.6.0 desde código fuente, en un directorio propio (`/opt/uhd-4.6.0`) sin sustituir el paquete de Debian, y recompilar srsRAN enlazando explícitamente contra ella. Tras verificar mediante `ldd` que el binario resultante cargaba correctamente la biblioteca nueva, el resultado fue el mismo error, con idéntico patrón.

D.5.2. Aislamiento final de la causa: `benchmark_rate`

Descartada la versión de UHD, se empleó la utilidad oficial `benchmark_rate`, distribuida con la propia UHD, que ejercita el flujo completo de transmisión y recepción de muestras sin intervención de srsRAN, permitiendo aislar si el problema residía en el sistema (kernel, USB, UHD) o en el propio código de srsRAN:

```
1 benchmark_rate --args="type=b200" --rx_rate 15.36e6 --tx_rate 15.36e6
   ↪ --duration 30
```

El resultado, tras 30 segundos de transmisión y recepción simultáneas a la misma tasa de muestreo empleada por el gNB, no mostró ningún error:

```
1 Num overruns detected:    0
2 Num underruns detected:   0
3 Num sequence errors (Tx): 0
4 Num sequence errors (Rx): 0
5 Num timeouts (Tx):        0
6 Num timeouts (Rx):        0
```

Este resultado fue muy importante, puesto que descartaba de raíz el sistema operativo, el kernel, el controlador USB y la propia UHD como origen del problema, situando la causa en cómo srsRAN empleaba la UHD.

D.5.3. Causa raíz: parche heredado de la integración de la bladeRF

La revisión del propio código fuente reveló el origen de la incidencia: los valores `TRANSMIT_TIMEOUT_S/RECEIVE_TIMEOUT_S`, fijados a 0 en la Sección 5.5 como parte de la integración de la bladeRF, seguían aplicados sobre el mismo árbol de código fuente empleado para compilar la prueba con la B210. Un valor de *timeout* de 0 segundos, necesario para el camino de *streaming* de la bladeRF a través de SoapyUHD, resulta excesivamente agresivo para el temporizado normal de una USRP nativa, haciendo fallar la transmisión ante cualquier variación mínima de tiempo del sistema, con independencia de que el resto de la cadena estuviera sana, que es exactamente lo que confirmaba `benchmark_rate`. Una vez revertidos ambos valores a los originales del proyecto (0.001/0.2f) y recompilado en un árbol de código fuente independiente del de la bladeRF, el gNB arrancó y se mantuvo estable de forma indefinida, sin ningún error de *timeout*.

D.6. Corrupción de la red de Docker tras reinicios parciales

Durante las pruebas de verificación de la Sección 5.11, se produjo una incidencia de red: dos de las funciones de red que sí dependen directamente de una conexión a MongoDB (PCF y UDR, las únicas dos NFs del despliegue con acceso directo a la base de datos) quedaron en un bucle continuo de reinicio, con el mensaje `Failed to connect to server [mongodb://db.open5gs.org/open5gs]` en su registro, pese a que el propio contenedor de MongoDB se encontraba en ejecución y sin errores.

La causa de fondo fue identificada mediante un diagnóstico por descarte entre contenedores: se comprobó que ni siquiera un contenedor sano del propio despliegue (el NRF) era capaz de resolver el nombre `db` a través de la red interna de Docker (`getent hosts db` sin resultado), lo que descartaba un problema específico de PCF/UDR en favor de una incidencia de la propia red *bridge* del despliegue. Esta corrupción se originó, con toda probabilidad, por una sucesión de intentos de arranque y parada incompletos sufridos durante la misma sesión de trabajo, motivados a su vez por un conflicto de puerto recurrente entre el MongoDB nativo (Sección 5.4) y el MongoDB del propio despliegue Docker, ambos publicados por defecto en el puerto 27017 del host.

La solución no consistió en repetir el ciclo habitual de `docker compose down/up`, que no fue capaz de resolver la incidencia por sí solo, sino en forzar la reconstrucción completa de la red de Docker (`docker network prune -f` entre ambos comandos, ver Sección 5.13). Tras esta reconstrucción, la resolución de nombres interna se restableció correctamente y las diez funciones de red permanecieron estables.

D.7. Conexión SCTP entre el gNB y el AMF en el despliegue Docker

Al intentar verificar `PUT /gnb/config` de extremo a extremo (incluyendo el reinicio real del gNB) contra el AMF ya desplegado en Docker (Sección 5.12), se puso de manifiesto

que esta combinación nunca se había probado hasta ese punto, todas las conexiones gNB-AMF documentadas hasta entonces en este trabajo se habían realizado contra el despliegue nativo de Open5GS. Al apuntar el gNB a la dirección publicada del AMF en Docker, la conexión N2 falló de forma persistente con el error `Network is unreachable`, con un tiempo de espera de 0 ms.

D.7.1. Primera causa: reenvío de direcciones internas en el *multi-homing* de SCTP

Una investigación con más detalle reveló un fallo conocido y documentado del propio proyecto Docker: cuando se publica un puerto SCTP mediante su proxy de aplicación (`docker-proxy`), este reenvía la dirección IP interna del contenedor dentro de la red *bridge* de Docker como una de las direcciones candidatas del mecanismo de *multi-homing* propio del protocolo SCTP, siendo esta dirección una que, por definición, no es alcanzable desde fuera de la red de Docker [34]. Como medida de mitigación, se desactivó el proxy de aplicación de Docker para utilizar el reenvío nativo mediante `iptables` del núcleo en `/etc/docker/daemon.json`:

```
1 {  
2   "userland-proxy": false  
3 }
```

Una vez realizado ese cambio, se llevó a cabo un reinicio del propio servicio (`sudo systemctl restart docker`). Tras esto, el error persistió de la misma forma, lo que facilitó acotar el problema: no era específico del proxy de aplicación, sino una limitación derivada de que ningún mecanismo de NAT (ni a nivel de aplicación ni de núcleo) puede reescribir el contenido de un paquete SCTP, únicamente sus cabeceras, y la dirección candidata problemática viaja dentro del propio paquete, como parte del protocolo.

D.7.2. Segunda causa: `bind_addr` obsoleto

Ante la persistencia del fallo, se optó por evitar el problema de NAT por completo, conectando el gNB directamente a la dirección interna del contenedor del AMF dentro de la propia red *bridge* de Docker, alcanzable sin traducción alguna desde el equipo anfitrión. El fallo persistió igualmente, con el mismo error, lo que llevó a descartar la hipótesis de NAT/SCTP y a verificar, mediante una conexión SCTP en bruto desde Python (sin ningún componente de srsRAN de por medio), que la red en sí funcionaba correctamente, confirmando así que el problema residía en cómo el propio gNB gestionaba su extremo local del socket, no en la infraestructura de red.

La revisión completa del fichero de configuración del gNB reveló la causa real: el campo `bind_addr` (la dirección local desde la que el gNB origina su conexión SCTP hacia el AMF) seguía fijado a `127.0.0.1`, un resto de la configuración original empleada contra el Open5GS nativo, nunca actualizado en ninguno de los cambios de dirección del AMF realizados a lo largo del trabajo. Un socket vinculado a la interfaz de *loopback* no puede, por definición, originar conexiones hacia ninguna dirección que no sea ella misma, con independencia de la dirección de destino que se intente alcanzar. Esto explica completamente que el error se reprodujera de forma idéntica frente a cualquier dirección de AMF probada durante esta incidencia.

Al corregir `bind_addr` a la dirección del equipo anfitrión dentro de la propia red de Docker (`10.33.33.1`, la puerta de enlace del *bridge*), se produjo una nueva incidencia relacionada: un conflicto con el puerto 2152 (interfaz N3, GTP-U), ya en uso por el UPF del propio despliegue Docker, al intentar el gNB reservar dicho puerto en todas las interfaces (`0.0.0.0`) en lugar de en una dirección específica. Restringir `bind_addr` a la dirección concreta del *bridge*, en lugar de a todas las interfaces, resolvió también este conflicto.

D.7.3. Confirmación final, y una lección sobre el propio proceso de depuración

Tras aplicar la corrección anterior, el registro de arranque capturado por el mecanismo de orquestación (`gnb_ctl.sh`, que solo captura la salida estándar del proceso) no mostraba ningún mensaje de éxito ni de error, generando la falsa impresión de que el proceso se había quedado bloqueado. La causa de esta ambigüedad fue, de nuevo, una configuración obsoleta: el nivel de registro del gNB (`all_level: debug`) dirige la práctica totalidad de la actividad del proceso a un fichero de registro independiente y más detallado (`/tmp/gnb.log`), no a la salida estándar capturada por el script de orquestación. La revisión de dicho fichero confirmó que la conexión se había establecido correctamente desde el primer intento tras la corrección:

```
1 N2: Connection to AMF on 10.33.33.6:38412 was established
2 Connected to AMF. Supported PLMNs: 90170
```

Esta confirmación quedó respaldada, de forma independiente, por el propio registro del AMF en Docker:

```
1 [Added] Number of gNBs is now 1
```

No se observó ningún mensaje posterior de desconexión. Esta incidencia deja una lección metodológica relevante: al depurar un sistema con múltiples fuentes de registro, conviene verificar explícitamente cuál de ellas refleja realmente el nivel de detalle configurado, en lugar de asumir que la primera fuente consultada es representativa del estado real del proceso.

Bibliografía

- [1] A. J. Hidalgo Fernández, “Desarrollo de aplicación de monitorización para redes privadas 5g basadas en open source,” Trabajo de Fin de Grado, Universidad de Extremadura, 2024, tutor: David Miguel Cortés Polo.
- [2] ITU-R, “Recommendation ITU-R M.2083-0: IMT vision – framework and overall objectives of the future development of IMT for 2020 and beyond,” 2015. [Online]. Available: https://www.itu.int/dms_pubrec/itu-r/rec/m/R-REC-M.2083-0-201509-I!!PDF-E.pdf
- [3] 3GPP, “3GPP meets IMT-2020.” [Online]. Available: <https://www.3gpp.org/technologies/3gpp-meets-imt-2020>
- [4] 5G-ACIA, “5g frequency ranges.” [Online]. Available: <https://5g-acia.org/insight/5g-user-guide/5g-frequency-ranges/>
- [5] 3GPP, “5G system overview.” [Online]. Available: <https://www.3gpp.org/technologies/5g-system-overview>
- [6] Informa TechTarget, “5G NSA vs. SA: How do the deployment modes differ?” [Online]. Available: <https://www.techtarget.com/searchnetworking/feature/5G-NSA-vs-SA-How-does-each-deployment-mode-differ>
- [7] W. Azariah, F. A. Bimo, C.-W. Lin, R.-G. Cheng, N. Nikaein, and R. Jana, “A survey on open radio access networks: Challenges, research directions, and open source approaches,” *Sensors*, vol. 24, no. 3, p. 1038, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/3/1038>

- [8] R. P. Alves *et al.*, “Experimental comparison of 5G SDR platforms: srsran x openairinterface,” arXiv:2406.01485, 2024. [Online]. Available: <https://arxiv.org/abs/2406.01485>
- [9] M. Motaung, N. Mthethwa, B. Nokane, and M. Masonta, “Comparative analysis of 5G SDR-based open-source software platforms: Openairinterface and srsran,” in *IEEE Conference Publication*, 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10992595/>
- [10] Software Radio Systems, “OCUDU 26.04: The next chapter for srsran,” 2026. [Online]. Available: https://www.srslte.com/press_releases/srsran_becomes_ocudu/
- [11] Lime Microsystems, “LimeSDR.” [Online]. Available: <https://limemicro.com/boards/limesdr/>
- [12] Great Scott Gadgets, “HackRF one — HackRF documentation.” [Online]. Available: https://hackrf.readthedocs.io/en/latest/hackrf_one.html
- [13] Prometheus Authors / Cloud Native Computing Foundation, “Prometheus: Overview.” [Online]. Available: <https://prometheus.io/docs/introduction/overview/>
- [14] M. Barbosa, M. Silva, E. Cavalcanti, and K. L. Dias, “Open-source 5g core platforms: A low-cost solution and performance evaluation,” arXiv:2412.21162, 2024. [Online]. Available: <https://arxiv.org/abs/2412.21162>
- [15] A. Chen, R. Preatoni, A. Brighente, M. Conti, and C. Nita-Rotaru, “Cross-service token: Finding attacks in 5g core networks,” arXiv:2509.08992, 2025. [Online]. Available: <https://arxiv.org/pdf/2509.08992>
- [16] Ettus Research, “USRP b200 USB software defined radio (SDR).” [Online]. Available: <https://www.ettus.com/all-products/ub200-kit/>
- [17] Nuand LLC, “bladerf 2.0 micro.” [Online]. Available: <https://www.nuand.com/bladerf-2-0-micro/>

- [18] Ettus Research, “B200/B210/B200mini/B205mini/B206mini – ettus knowledge base.” [Online]. Available: <https://kb.ettus.com/B200/B210/B200mini/B205mini/B206mini>
- [19] Open5GS, “Metrics monitoring system, issue #1559,” GitHub. [Online]. Available: <https://github.com/open5gs/open5gs/issues/1559>
- [20] —, “Get data about sessions, discussion #4068,” GitHub. [Online]. Available: <https://github.com/open5gs/open5gs/discussions/4068>
- [21] —, “Openapi calls with open5gs, discussion #2408,” GitHub. [Online]. Available: <https://github.com/open5gs/open5gs/discussions/2408>
- [22] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [23] Borjis131, “docker-open5gs: Open5GS 5g core container images, docker compose deployments and helm charts,” GitHub. [Online]. Available: <https://github.com/Borjis131/docker-open5gs>
- [24] Open5GS, “Open5gs v2.7.7 release notes,” GitHub. [Online]. Available: <https://github.com/open5gs/open5gs/releases/tag/v2.7.7>
- [25] —, “Metrics with prometheus,” 2026. [Online]. Available: <https://open5gs.org/open5gs/docs/tutorial/04-metrics-prometheus/>
- [26] DeepWiki, “Nrf (network repository function) — open5gs/open5gs,” DeepWiki. [Online]. Available: [https://deepwiki.com/open5gs/open5gs/6.2-nrf-\(network-repository-function\)](https://deepwiki.com/open5gs/open5gs/6.2-nrf-(network-repository-function))
- [27] Open5GS, “Smf and multiple upfs in network, discussion #3630,” GitHub. [Online]. Available: <https://github.com/open5gs/open5gs/discussions/3630>
- [28] Allbesmart, *OAI BOX 5G Lab Manual*, v2026-01-07 ed. [Online]. Available: <https://dashboard.oaibox.com>

- [29] Open5GS, “Some metrics of open5gs/prometheus are not being collected, issue #3100,” GitHub. [Online]. Available: <https://github.com/open5gs/open5gs/issues/3100>
- [30] Ettus Research, *USRP Hardware Driver and USRP Manual: USRP B2x0 Series*. [Online]. Available: https://uhd.readthedocs.io/en/uhd-4.2/page_usrp_b200.html
- [31] International Telecommunication Union, “Recommendation ITU-T E.212: The international identification plan for public networks and subscriptions.” [Online]. Available: <https://www.itu.int/rec/T-REC-E.212>
- [32] srsRAN Project contributors, “exceeded maximum number of timed out transmissions, issue #727,” GitHub. [Online]. Available: https://github.com/srsran/srsRAN_Project/issues/727
- [33] —, “B210 streaming errors, discussion #550,” GitHub. [Online]. Available: https://github.com/srsran/srsRAN_Project/discussions/550
- [34] moby/moby contributors, “SCTP: docker-proxy forwards container’s bridge network IP addresses as potential candidates for multi-homing, issue #42762,” GitHub. [Online]. Available: <https://github.com/moby/moby/issues/42762>